

BIG DATA · NIVEAU 1

Spark Python développer des applications pour le big data

Traiter en parallèle des données qui ne tiennent plus sur une machine, en écrivant du Python.

DURÉE	HORAIRES	LIEU	FORMATEUR
3 jours · 21 heures	09h00 – 17h30	Paris La Défense	Mehdi Tajmouati

Qui est dans la salle

Trois questions, deux minutes chacun :

1. Votre métier : vous **développez**, vous **analysez** des données, vous **exploitez** une plateforme ?
2. Votre Python aujourd'hui : scripts, pandas, notebooks, rien de tout cela ?
3. Quel volume de données vous a fait venir ici, et qu'est-ce qui a cassé ?

Pourquoi la troisième question

On vient rarement à Spark par curiosité. On y vient parce qu'un script pandas a saturé la mémoire, qu'un traitement de nuit ne tient plus dans la nuit, ou qu'un projet l'a imposé. Le point de rupture dit ce qu'il faudra approfondir.

Ce que vous saurez faire vendredi soir

1 Expliquer ce que fait Spark, et où s'exécute votre code

Driver, exécutateurs, partitions, transformations paresseuses : le modèle mental sans lequel rien ne se débogue.

2 Manipuler des données avec les RDD, puis avec les DataFrames et Spark SQL

Dans un notebook Jupyter, du plus bas niveau au plus productif, jusqu'à l'API pandas de Spark pour qui vient de pandas.

3 Entraîner et évaluer un modèle de classification avec MLlib

Un pipeline complet, de la donnée brute au modèle sauvegardé.

4 Calculer des statistiques en temps réel sur un flux

Fenêtres, agrégations, données en retard, et le modèle entraîné appliqué au fil de l'eau.

5 Analyser un réseau avec les graphes

PageRank et plus court chemin, et la visualisation du résultat.

Les trois jours d'un coup d'œil

MATIN · 09H00 – 12H30

APRÈS-MIDI · 13H30 – 17H30

J1 · mer.
30/09**Module 1** · Découvrir Apache Spark
TP 1 : l'environnement et un premier script**Module 2** · Programmer avec les RDD
TP 2 : analyser le journal brut des bornes**J2 · jeu.**
01/10**Module 3** · Spark SQL et DataFrames
TP 3 tutoré : nettoyer, joindre, interroger**Module 4** · Machine learning avec MLlib
TP 4 : une classification supervisée**J3 · ven.**
02/10**Module 5** · Analyser en temps réel
TP 5 : statistiques et prédictions sur un flux**Module 6** · Théorie des graphes
TP 6 : PageRank, plus court chemin, visualisation · **bilan**

Le fil est **progressif** : chaque module réutilise le précédent. Le modèle entraîné au TP 4 sert au TP 5, et les trajets nettoyés au TP 3 deviennent le graphe du TP 6.

Comment lire ces supports

Base

Le socle du programme. Vu par tout le monde, c'est ce sur quoi vous serez à l'aise en sortant.

Avancé

Un approfondissement, hors programme contractuel. Traité selon le rythme du groupe.

Si le temps le permet

Une slide qui approfondit un sujet **déjà livré ailleurs** : la sauter ne retire rien au programme.

Le support vous est remis

En PDF, en fin de session, avec le code de tous les TP et leurs corrigés. Vous n'avez rien à recopier : écoutez, essayez, et notez ce qui vous concerne.

Le Python dont on aura besoin

Fonctions, et fonctions anonymes

```
def carre(x):  
    return x * x  
  
carre = lambda x: x * x      # la même chose  
  
list(map(lambda x: x * 2, [1, 2, 3]))  
# [2, 4, 6]
```

Tuples, et déballage

```
paire = ("S017", 42)  
station, nombre = paire
```

Compréhensions

```
[x * 2 for x in range(5) if x % 2 == 0]  
# [0, 4, 8]
```

pandas, en trois lignes

```
import pandas as pd  
df = pd.read_csv("trajets.csv")  
df.groupby("type_usager")["duree_min"].mean()
```

Spark reprend trois de ces idées : des **fonctions passées en argument**, des **paires clé-valeur**, des **tables**. Le reste est nouveau.

Vélo'Cité, le jeu de données des trois jours

Un réseau **fictif** de vélos en libre-service : 60 stations, 8 quartiers, six mois d'activité.

`trajets.csv`

Environ 300 000 trajets : départ, arrivée, durée, distance, abonné ou occasionnel

`journal-bornes.log`

Environ 600 000 lignes de texte brut, telles que les bornes les émettent

`stations.csv`

Nom, quartier, position, capacité

`meteo.csv`

Température et pluie, jour par jour

Un fil, cinq questions

Quelles bornes tombent en panne ? Qui roule, quand, combien de temps ? Peut-on deviner le type d'utilisateur ? Que se passe-t-il *maintenant* ? Quelles stations sont au cœur du réseau ?

Volontairement imparfait

Doublons, durées négatives, lignes tronquées : environ 0,5 % de défauts, comme en production.

Ce que cette formation n'est pas

Pas un cours de Scala

Spark est écrit en Scala, mais tout se fait ici en Python. On lira quelques lignes de Scala dans la documentation, on n'en écrira pas.

Pas un cours d'administration

On comprend comment un cluster exécute un job, on n'en installe pas. Les TP tournent en mode local, sur votre poste.

Pas un cours de data science

Le module 4 montre comment Spark entraîne un modèle. Le choix du modèle et sa théorie sont survolés, pas enseignés.

Ce qui s'écrit en local sur votre portable **tourne tel quel sur un cluster de cent machines**. Ce qui change, c'est une ligne de configuration, pas le code.

Comment on travaille

Pendant les exposés

- **Coupez-moi.** Une question posée au moment où elle se pose vaut mieux qu'une question gardée pour la fin.
- Le code des slides est fait pour être **tapé et exécuté**, dans un notebook ouvert à côté.
- Aucune question n'est trop simple. « C'est quoi une partition ? » est une question de la salle, pas une faute.

Pendant les TP

- Tout est dans le dépôt github.com/WyTaSoft-Academy/pyspark-hands-on-labs : les TP, les exercices guidés, le générateur de données.
- Chaque TP a un **énoncé**, un **notebook de départ** et un **corrigé** projeté en fin de TP.
- Un poste qui ne suit pas n'est pas un problème : le code de chaque étape est projeté, on repart de là.
- Les étapes **bonus** sont pour ceux qui ont fini, pas pour tout le monde.

Trois questions pour situer la salle

Q1 Un fichier CSV de 50 Go : que se passe-t-il si on l'ouvre avec `pd.read_csv` sur un portable de 16 Go ?

Au mieux une erreur mémoire, au pire une machine qui passe dans le swap pendant une heure. pandas charge tout en mémoire, sur une seule machine.

Q2 Pour compter les mots d'un texte, avez-vous besoin de voir le texte entier en même temps ?

Non. Chaque morceau peut être compté séparément, puis les comptes additionnés. C'est l'idée de map/reduce, et c'est tout Spark.

Q3 Si une machine sur cent tombe en panne au milieu d'un calcul, que doit-on refaire ?

Idéalement, seulement la part de cette machine. Spark le fait en se souvenant de la façon dont chaque morceau a été calculé plutôt qu'en le copiant.

MODULE 1

Découvrir Apache Spark

D'où vient Spark, de quoi il est fait, et comment un calcul se répartit sur plusieurs machines.

JOUR 1 · MODULE 1

Découvrir Apache Spark

D'où vient Spark, de quoi il est fait, et où s'exécute le code que vous écrivez.

DURÉE

≈ 1 h 45 d'exposé + TP 1 h

À LA FIN

Savoir dire ce qui s'exécute, où, et quand

TP

L'environnement et un premier script

Ce que couvre le module 1

1 D'où vient Spark

Le problème qu'il résout, et plus de quinze ans d'histoire en une frise.

2 De quoi Spark est fait

Le cœur, les quatre composants, et ce qui existe en Python.

3 Les concepts de programmation

Driver, exécuteurs, partitions, transformations, actions, shuffle.

4 Exécuter en distribué

Du mode local au cluster, `spark-submit`, l'interface web.

5 TP : l'environnement et un premier script

Installer, vérifier, lire 300 000 trajets, regarder Spark travailler.

La question de la matinée

Quand j'écris une ligne de code Spark, **qui l'exécute, sur quelle machine, et à quel moment ?**

Ceux qui savent y répondre déboguent leurs jobs. Les autres attendent qu'ils finissent.

SÉQUENCE 1

D'où vient Spark

Le problème d'une machine qui ne suffit plus, et comment on l'a résolu deux fois.

Quand une machine ne suffit plus

Grossir la machine

Plus de mémoire, plus de cœurs, plus de disque. Simple : le code ne change pas.

Mais le prix monte plus vite que la puissance, il y a un plafond, et une panne arrête tout.

Ajouter des machines

Des machines ordinaires, autant qu'il en faut. Le coût suit la charge, une panne n'en arrête qu'une.

Mais il faut **découper le calcul**, déplacer les données, et survivre aux pannes.

Ce que Spark prend en charge

- Découper les données en **partitions**
- Envoyer le calcul là où sont les données
- Regrouper les résultats intermédiaires
- Recalculer ce qu'une panne a perdu

Vous écrivez le calcul **comme pour une machine**. Spark le répartit.

MapReduce : la bonne idée, et son coût

Google publie MapReduce en 2004 ; Hadoop en est l'implémentation libre. Tout calcul s'écrit en deux temps :

1 map

Chaque machine transforme sa part des données, sans parler aux autres.

2 shuffle

Les résultats sont regroupés par clé, entre machines, par le réseau.

3 reduce

Chaque groupe est agrégé : une somme, un comptage, une moyenne.

La limite

Chaque étape **écrit sur disque** avant la suivante. Un calcul en dix étapes, c'est dix allers-retours disque. Un algorithme itératif — un modèle de machine learning, un PageRank — relit toutes ses données à chaque tour.

Spark garde l'idée et change le support : **la mémoire** plutôt que le disque, entre les étapes.

Plus de quinze ans en une frise



Ce que la frise raconte

Spark est parti des **RDD**, une API bas niveau, puis a déplacé son centre de gravité vers les **DataFrames** et le SQL, que le moteur sait optimiser.

Ce que cela veut dire pour vous

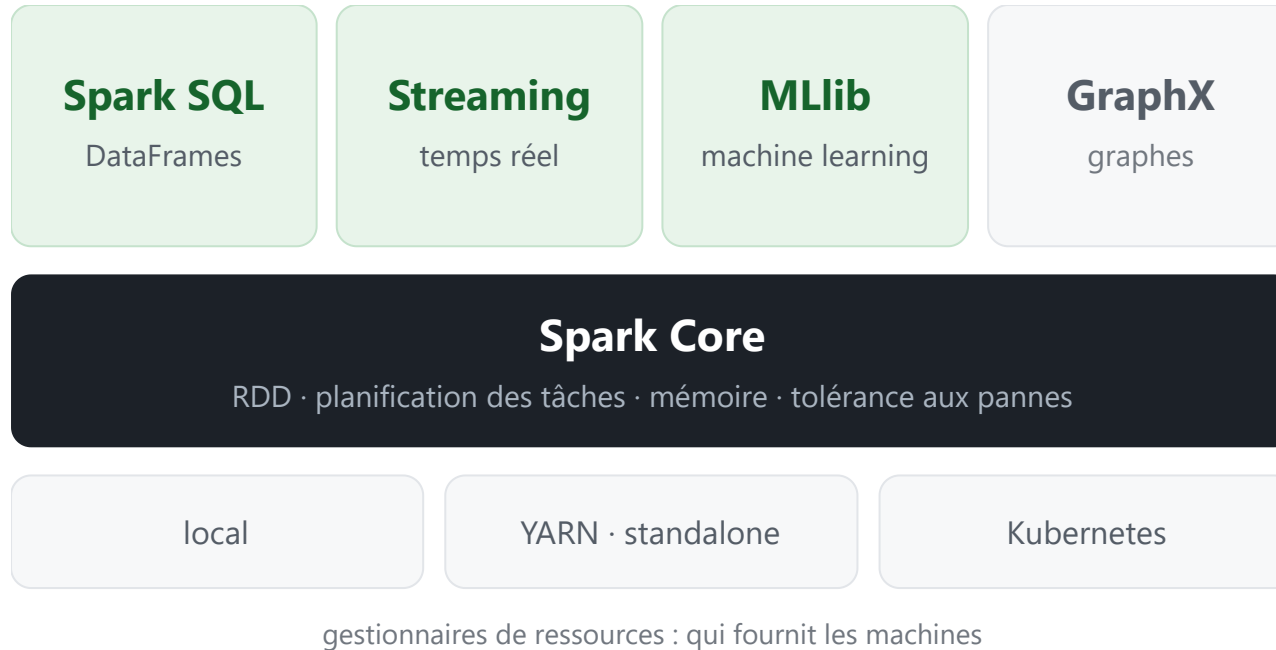
On apprend les RDD cet après-midi pour **comprendre** le moteur. On écrira des DataFrames demain pour **travailler**. Les TP tournent sur **Spark 3.5**.

SÉQUENCE 2

De quoi Spark est fait

Un moteur, quatre bibliothèques, et ce qu'on en voit depuis Python.

Un moteur, quatre composants



Les quatre bibliothèques partagent le **même moteur** : un DataFrame SQL nourrit un modèle MLlib sans export ni copie.

Le moteur ne fournit pas les machines : il les demande à un **gestionnaire de ressources**.

Un module par composant : SQL demain matin, MLlib demain après-midi, Streaming et graphes vendredi.

Ce que chaque composant offre en PySpark

COMPOSANT	EN PYTHON	À RETENIR
Spark SQL	<code>pyspark.sql</code>	Complet, et c'est l'API principale de Spark aujourd'hui
Streaming	<code>pyspark.sql.streaming</code> <code>pyspark.streaming</code>	Structured Streaming est l'API actuelle. Les <i>DStreams</i> , au programme, sont l'ancienne : dépréciée depuis Spark 3.4, toujours présente
MLlib	<code>pyspark.ml</code> <code>pyspark.mllib</code>	La première travaille sur des DataFrames, c'est celle à utiliser. La seconde, sur des RDD, est en maintenance
GraphX	Aucune API Python	On passe par GraphFrames , une bibliothèque séparée, bâtie sur les DataFrames

Le programme cite les anciennes API parce qu'elles existent encore dans beaucoup de code en production. On les verra, et on dira chaque fois ce qui les remplace.

PySpark, Jupyter, et ce qu'est devenu Koalas

PySpark

La bibliothèque Python de Spark. `pip install pyspark` installe aussi le moteur : rien d'autre à télécharger, hormis Java (et, pour l'API pandas, un numpy antérieur à 2).

Jupyter

Le notebook : on exécute cellule par cellule, on voit les résultats au fil de l'eau. L'outil d'exploration, et celui des TP.

Koalas

L'API pandas au-dessus de Spark. Intégré à Spark 3.2 sous le nom **pandas API on Spark** ; le projet Koalas est archivé.

```
import pyspark.pandas as ps

trajets = ps.read_csv("donnees/trajets.csv")
trajets.groupby("type_usager")["duree_min"].mean()    # la syntaxe pandas, exécutée par Spark
```

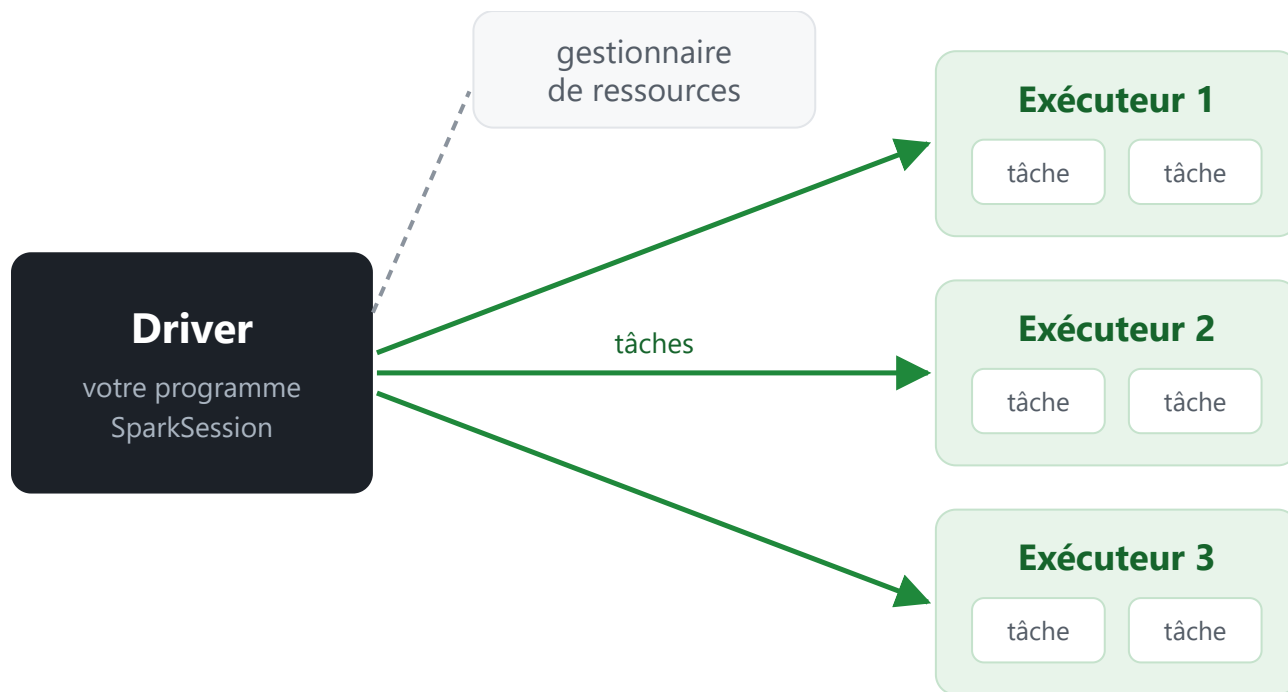
Le code pandas de l'ouverture, presque mot pour mot, mais distribué. On y revient au module 3.

SÉQUENCE 3

Les concepts de programmation

Qui exécute votre code, sur quelle machine, et à quel moment.

Un driver, des exécuteurs



Le **driver** exécute votre programme, découpe le calcul et distribue les tâches.

Les **exécuteurs** sont des processus sur les machines du cluster : chacun exécute des tâches, une par cœur.

Un `print` dans une fonction passée à Spark s'affiche **chez l'exécuteur**, pas dans votre notebook.

SparkSession et SparkContext

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
        .master("local[2]")
        .appName("premier-script")
        .getOrCreate())

sc = spark.sparkContext      # l'ancien point d'entrée

print(spark.version)        # 3.5.4
print(sc.master)            # local[2]
print(sc.defaultParallelism) # 2
```

SparkSession

Depuis Spark 2.0. DataFrames, SQL, lecture de fichiers : **le point d'entrée à utiliser.**

SparkContext

Depuis l'origine. Les RDD, les accumuleurs, le broadcast. Toujours là, dans la session.

`getOrCreate()` réutilise la session existante : dans un notebook, relancer la cellule ne crée pas un second Spark.

Les partitions : l'unité de parallélisme

Spark ne voit jamais « un fichier » : il voit des **partitions**, des morceaux traités indépendamment.

- **Une partition = une tâche**, exécutée par un cœur.
- Trop peu de partitions : des cœurs inoccupés.
- Trop : le coût de lancement des tâches domine.

Ordre de grandeur courant : quelques partitions par cœur disponible, de l'ordre de 100 à 200 Mo chacune.

```
trajets = spark.read.csv("trajets.csv",  
                        header=True,  
                        inferSchema=True)  
  
trajets.rdd.getNumPartitions()  
# 2 : 27 Mo de CSV, deux cœurs  
  
trajets.repartition(8).rdd.getNumPartitions()  
# 8
```

Transformations et actions

	TRANSFORMATION	ACTION
Ce qu'elle fait	Décrit un nouveau jeu de données à partir d'un autre	Demande un résultat : un nombre, un affichage, un fichier
Ce qu'elle renvoie	Un DataFrame ou un RDD, pas encore calculé	Une valeur Python, ou rien (écriture)
Déclenche un calcul	Non	Oui : un ou plusieurs jobs
Exemples	<code>select</code> <code>filter</code> <code>groupBy</code> <code>join</code> <code>map</code>	<code>count</code> <code>show</code> <code>collect</code> <code>first</code> <code>write</code>

La question à se poser devant chaque ligne : **est-ce que je demande un résultat ?** Si non, rien ne s'exécute encore.

Rien ne s'exécute avant une action

```
from pyspark.sql import functions as F

longs = (trajets
        .filter(F.col("duree_min") > 30)
        .select("trajet_id", "type_usager"))
# instantané : 0 job lancé

longs.count()
# 22829 : le calcul a lieu ici
```

Mesuré au TP : `filter` et `select` lancent **0 job** ; `count` en lance au moins un.

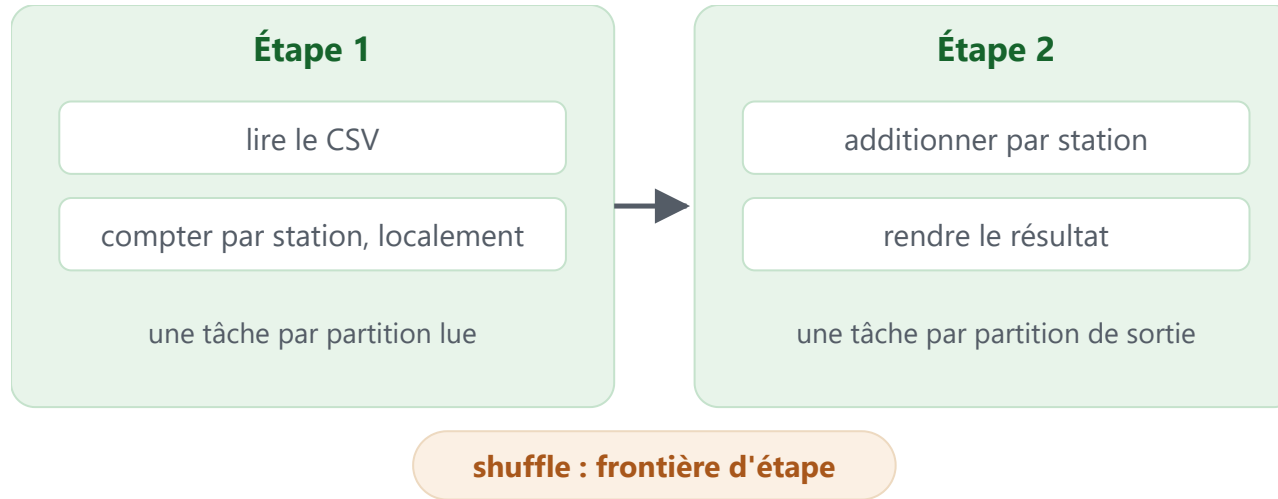
Pourquoi c'est un avantage

Spark voit **toute la chaîne** avant de l'exécuter. Il peut ne lire que les deux colonnes utiles, appliquer le filtre pendant la lecture, et fusionner les étapes.

Pourquoi ça déroute

Une erreur dans le `filter` n'apparaît qu'au `count`, trois cellules plus loin.

Job, étapes, tâches



Job	Ce que déclenche une action
Étape	Une suite d'opérations sans échange entre machines
Tâche	Une étape appliquée à une partition

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[station_depart], ...)
   +- Exchange hashpartitioning(...)
      +- HashAggregate(... partial_count(1))
         +- FileScan csv [station_depart]
```

Le shuffle : quand les données doivent voyager

Dépendance étroite

Chaque partition de sortie ne dépend que d'**une** partition d'entrée. Pas d'échange, pas de réseau.

`select` · `filter` · `withColumn` · `map`

Dépendance large

Une partition de sortie a besoin de lignes venues de **toutes** les partitions. Écriture sur disque, transfert réseau, relecture.

`groupBy` · `join` · `orderBy` · `distinct` · `repartition`

Pourquoi on en parle dès maintenant

Un job lent est presque toujours un job qui **déplace trop de données** : un shuffle de trop, ou un shuffle où une clé concentre tout.

Réflexe : **filtrer et réduire avant** de regrouper ou de joindre. Ce qui ne voyage pas ne coûte rien.

SÉQUENCE 4

Exécuter en environnement distribué

Du portable au cluster : ce qui change, et surtout ce qui ne change pas.

Une ligne décide où tourne le calcul

VALEUR DE MASTER	OÙ S'EXÉCUTE LE CALCUL
<code>local</code>	Sur ce poste, un seul cœur. Utile pour déboguer, jamais pour mesurer.
<code>local[2]</code> · <code>local[*]</code>	Sur ce poste, deux cœurs, ou tous. C'est le mode des TP.
<code>spark://hote:7077</code>	Cluster standalone : le gestionnaire fourni avec Spark.
<code>yarn</code>	Cluster Hadoop : le cas historique des entreprises.
<code>k8s://https://hote:443</code>	Kubernetes : chaque exécuteur est un pod.

Le code du TP d'aujourd'hui tourne sur un cluster de cent machines **sans autre changement que cette valeur** — et le chemin des fichiers, qui doit être visible de toutes les machines.

`spark-submit`, et les deux modes de déploiement

```
spark-submit \  
  --master yarn \  
  --deploy-mode cluster \  
  --num-executors 10 \  
  --executor-cores 4 \  
  --executor-memory 8g \  
  mon_job.py donnees/2026-06
```

Le même script, en local :

```
spark-submit --master "local[2]" mon_job.py
```

MODE	OÙ TOURNE LE DRIVER
client	Sur la machine qui soumet. On voit la sortie, mais si elle s'éteint, le job meurt.
cluster	Sur une machine du cluster. On peut fermer son portable ; la sortie est dans les journaux.

Notebook : toujours **client**. Job de production planifié : **cluster**.

L'interface web, sur le port 4040

ONGLET	LA QUESTION À LAQUELLE IL RÉPOND
Jobs	Qu'est-ce qui a tourné, combien de temps, a-t-il échoué ?
Stages	Quelle étape est lente, et combien de données a-t-elle échangées ?
SQL / DataFrame	Quel plan a été exécuté, avec quels volumes à chaque nœud ?
Storage	Qu'est-ce qui est gardé en mémoire ?
Executors	Qui travaille, qui manque de mémoire ?
Environment	Avec quelle configuration tourne-t-on vraiment ?

Où la trouver

`spark.sparkContext.uiWebViewUrl` donne l'adresse. En local : `http://localhost:4040`, puis 4041 si une seconde session tourne.

Elle disparaît avec la session

Après `spark.stop()`, l'interface ferme. Pour garder l'historique : le *History Server*.

L'environnement et un premier script

≈ 1 h

Individuel

Restitution 10 min

Échauffement : [exercices/serie1-decouvrir/](#)

1

Installer et vérifier

Python 3.8 à 3.11, Java 11 ou 17, `pip install "pyspark==3.5.4"`, puis
`python verifier_env.py` : tout doit être OK.

2

Générer les données

`python donnees/generer_donnees.py` : 300 300 trajets, en moins d'une minute.

3

Premier script

Lire les trajets, le schéma, compter par type d'utilisateur, les stations les plus actives.

4

Regarder Spark travailler

Combien de jobs lance chaque cellule ? Où est le shuffle dans l'interface web ?

Les fichiers

Dépôt [pyspark-hands-on-labs](#), sur GitHub :

`tp1-environnement/enonce.md`

`depart.ipynb` · `verifier_env.py`

Sous Windows

Les pièges, tous dans l'énoncé : `PYSPARK_PYTHON`,
`winutils.exe`, un vieux `SPARK_HOME`.

Cinq questions avant de fermer le module 1

Q1 Vous exécutez `df.filter(...).select(...)` : la cellule rend la main immédiatement. Le filtre est-il faux ?

Non : ce sont deux transformations, rien n'a été calculé. L'erreur éventuelle n'apparaîtra qu'à la première action.

Q2 Un `collect()` sur un gros DataFrame fait planter le job. Qui est tombé, et pourquoi ?

Le driver : `collect` rapatrie toutes les lignes dans sa mémoire, qui est celle d'une seule machine.

Q3 Pourquoi un `groupBy` crée-t-il une nouvelle étape, et pas un `filter` ?

Le regroupement exige que les lignes d'une même clé se retrouvent ensemble : c'est un shuffle, donc une frontière d'étape. Le filtre travaille partition par partition.

Q4 Quelle ligne change entre le TP et un cluster YARN ?

Le master, passé à `spark-submit --master yarn`, à condition de ne pas l'avoir fixé dans le code — et les chemins de fichiers, qui doivent être lisibles par toutes les machines.

Q5 Vous voulez des graphes en Python. Pourquoi pas GraphX ?

GraphX n'a pas d'API Python. On utilise GraphFrames, bâtie sur les DataFrames.

ANNEXES DU MODULE 1 AVANCÉ

Sous le capot du déploiement

D'où vient l'écosystème, ce que coûte le langage, la mémoire d'un exécuter, et un vrai cluster sur votre poste.

L'écosystème Hadoop, dont Spark a hérité

BRIQUE	SON RÔLE
HDFS	Stockage distribué : fichiers découpés en blocs de 128 Mo , copiés sur trois machines. Un <i>NameNode</i> sait où est chaque bloc, les <i>DataNodes</i> les gardent.
YARN	Répartit les cœurs et la mémoire du cluster entre les applications.
MapReduce	Le moteur de calcul d'origine, sur disque entre chaque étape.
Hive	Du SQL sur ces fichiers, et un <i>metastore</i> : le catalogue des tables.
Autour	HBase, Sqoop, Oozie, ZooKeeper... stockage clé-valeur, import, ordonnancement, coordination.

Ce que Spark remplace

MapReduce, et le plus souvent Hive comme moteur d'exécution.

Ce que Spark garde

HDFS comme source, YARN comme gestionnaire, le metastore Hive comme catalogue. Dans le cloud, HDFS cède la place au stockage objet (S3, ADLS, GCS).

Spark ne stocke rien : il **lit et écrit** là où sont les données.

Python, Scala ou Java : un seul moteur

Un calcul écrit avec les **DataFrames** ou en **SQL** produit le même plan, quel que soit le langage : il s'exécute dans la JVM, Python ne fait que le décrire.

Python ne coûte que là où **vos fonctions** s'exécutent : UDF et lambdas des RDD. Chaque ligne part alors vers un processus Python, et revient.

```
== Physical Plan ==              (expression native)
*(1) Project [(duree_min#23 / 60.0) AS h#39]
+- FileScan csv [duree_min#23]

== Physical Plan ==              (fonction Python, UDF)
*(1) Project [pythonUDF0#45 AS h#43]
+- BatchEvalPython [<lambda>(duree_min#23)#42]
   +- FileScan csv [duree_min#23]
```

L'anti-modèle, dans tous les langages

Faire le travail **hors de Spark** : tout rapatrier sur le driver, puis boucler en pandas, en Java ou en Scala. Sur un échantillon, c'est rapide. En production, le driver sature et le cluster attend.

```
# hors de Spark : tout passe par le driver
pdf = trajets.toPandas()
pdf.groupby("type_usager")["duree_min"].mean()

# dans Spark : le calcul reste distribué
trajets.groupBy("type_usager").avg("duree_min")
```

Séparer le client du moteur

Depuis Spark 3.4, le client peut parler au moteur **à distance** : il envoie des plans, le serveur les exécute et renvoie les résultats.

```
pip install "pyspark[connect]==3.5.4"

from pyspark.sql import SparkSession
spark = (SparkSession.builder
        .remote("sc://serveur:15002")
        .getOrCreate())
```

Le reste du code est inchangé : mêmes DataFrames, même SQL.

Ce que ça change

- Le poste n'exécute plus le moteur
- Un client qui plante ne fait pas tomber le moteur
- Client et serveur se mettent à jour séparément

Ce qu'on perd

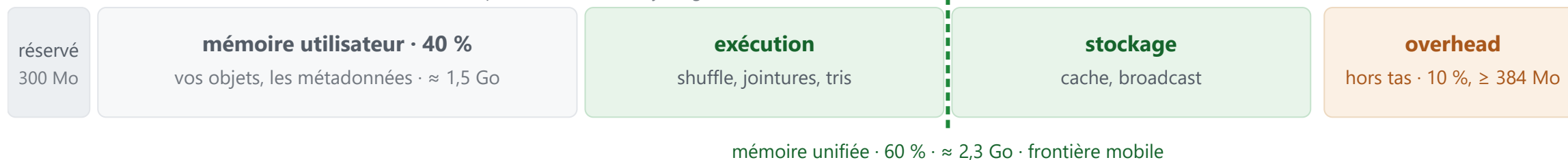
Les RDD et le `SparkContext` : Spark Connect ne parle que DataFrames et SQL.

Ce qu'on cherche dans le détail d'une étape

SYMPTÔME	OÙ ON LE VOIT	CE QUE ÇA VEUT DIRE
Déséquilibre	Tâche max bien au-dessus de la médiane, dans <i>Summary Metrics</i>	Une clé concentre les données : une tâche fait le travail de toutes
Débordement	Colonnes <i>Spill (Memory / Disk)</i> non nulles	Une partition ne tient pas en mémoire : plus de partitions, ou plus de mémoire
Shuffle lourd	<i>Shuffle Read / Write</i> proche de la taille des données	Tout voyage : filtrer ou agréger plus tôt
Ramasse-miettes	<i>GC Time</i> > 10 % de la durée des tâches	Trop d'objets en mémoire : souvent un cache de trop
Trop de petites tâches	Milliers de tâches de quelques millisecondes	Trop de partitions : le lancement coûte plus que le calcul

La mémoire d'un exécuteur

spark.executor.memory = 4g (le tas de la JVM)



RÉGLAGE	DÉFAUT EN 3.5
<code>spark.executor.memory</code>	1g
<code>spark.memory.fraction</code>	0,6 : la part unifiée
<code>spark.memory.storageFraction</code>	0,5 : le cache protégé
<code>spark.executor.memoryOverheadFactor</code>	0,1, avec un minimum de 384 Mo

Et Python ?

Les processus Python des exécuteurs vivent **hors du tas**, dans l'overhead. Une UDF gourmande fait tuer le conteneur par YARN ou Kubernetes, sans erreur Java lisible.

Combien d'exécuteurs, de cœurs, de mémoire ?

Un raisonnement de départ

1. Retirer à chaque machine un cœur et quelques Go pour le système.
2. Viser **4 à 5 cœurs par exécuteur** : au-delà, les écritures disque se gênent.
3. Diviser la mémoire restante par le nombre d'exécuteurs de la machine.
4. Garder environ 10 % pour la mémoire hors tas (**memoryOverhead**).

Puis **mesurer** dans l'interface web, et corriger. Aucune formule ne remplace un essai.

GESTIONNAIRE	QUAND LE CHOISIR
Standalone	Cluster dédié à Spark, simple à monter
YARN	Plateforme Hadoop existante, partagée
Kubernetes	Tout le reste de l'entreprise y est déjà
Managé	Databricks, EMR, Dataproc... : on ne gère plus les machines

Un cluster standalone, sur votre poste

```
spark-class org.apache.spark.deploy.master.Master --host 127.0.0.1 --port 7078 --webui-port 8090
spark-class org.apache.spark.deploy.worker.Worker spark://127.0.0.1:7078 --cores 2 --memory 2g
spark-submit --master spark://127.0.0.1:7078 --total-executor-cores 2 job_standalone.py
```

```
Master: Starting Spark master at spark://127.0.0.1:7078
Master: I have been elected leader! New state: ALIVE
Worker: Starting Spark worker ... with 2 cores, 2.0 GiB RAM
Worker: Successfully registered with master
Master: Launching executor app-...-0000/0 on worker ...
```

```
master : spark://127.0.0.1:7078
|      NULL |   150 |
|   abonne |190269 |
|occasionnel|109881 |
```

Trois processus, trois terminaux

Le **master** distribue les ressources, le **worker** les fournit, **spark-submit** lance le driver. Interface du master sur le port 8090.

Le master écrit dans le code gagne

Un `.master("local[2]")` dans le script l'emporte sur `--master` : le job tourne en local, sans rien dire.

MODULE 2

Programmer avec les RDD

L'API d'origine de Spark : des fonctions Python appliquées à des collections distribuées.

JOUR 1 • MODULE 2

Programmer avec les RDD

La brique de base de Spark : une collection répartie sur plusieurs machines, que l'on transforme avec des fonctions Python.

DURÉE	À LA FIN	TP
≈ 2 h d'exposé + TP 1 h 30	Écrire un map/reduce, et savoir ce qui part sur le réseau	Analyser le journal brut des bornes

Ce que couvre le module 2

- 1 Se connecter à Spark depuis Python**
L'environnement, les contextes, les sessions.
- 2 Les RDD**
Créer, transformer, déclencher.
- 3 Map/reduce**
Paires clé-valeur, lignage, et ce que coûte un *shuffle*.
- 4 Réutiliser, partager, partitionner**
Cache, accumulateurs, broadcast, partitions.
- 5 Notebooks et jobs**
Du notebook au script soumis avec `spark-submit`.

Le fil de l'après-midi

Un seul fichier : `journal-bornes.log`, 600 000 lignes de texte brut.

La question qui structure tout : **quelles bornes tombent en panne, et combien de fois ?** On y répond à la fin avec une dizaine de lignes de code.

SÉQUENCE 1

Se connecter à Spark depuis Python

Un Python pour le driver, un Python pour les exécuteurs, et un point d'entrée pour les relier.

Configurer son environnement Python

PySpark lance **deux sortes de processus Python** : celui du driver, où tourne votre notebook, et ceux des exécuteurs, qui appliquent vos `lambda`.

Ils doivent avoir **la même version de Python**, et les mêmes bibliothèques.

```
python -m venv .venv
.venv\Scripts\activate      # Linux : source .venv/bin/activate
pip install "pyspark==3.5.4"

# le même Python des deux côtés (invite cmd)
set PYSPARK_PYTHON=python
set PYSPARK_DRIVER_PYTHON=python
# PowerShell : $env:PYSPARK_PYTHON="python"
# Linux      : export PYSPARK_PYTHON=python
```

Sous Windows, trois pièges connus

- `PYSPARK_PYTHON` absent : le worker « ne se reconnecte pas » (*failed to connect back*).
- Écrire un fichier demande `winutils.exe` et `HADOOP_HOME`.
- `take()` sur un gros fichier texte peut faire planter le worker Python. Le code est juste ; c'est PySpark sous Windows.

Sur un cluster, les exécuteurs n'ont pas votre `venv` : on l'expédie avec le job. On y revient en séquence 5.

Contexte et session

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
        .master("local[2]")
        .appName("decouverte")
        .getOrCreate())

sc = spark.sparkContext      # le contexte, pour les RDD

print(spark.version, sc.master, sc.defaultParallelism)
# 3.5.4 local[2] 2

spark.stop()                # libère le driver et l'interface web
```

`getOrCreate()` rend la session existante si elle existe : relancer la cellule d'un notebook n'en crée pas une seconde.

SparkContext L'historique (Spark 1). La connexion au cluster, et l'entrée des RDD.

SparkSession Depuis Spark 2. Englobe le contexte, et ouvre SQL et DataFrames.

Un seul contexte par processus

En créer un second lève `ValueError: Cannot run multiple SparkContexts at once`. Pour changer de configuration : `stop()`, puis recréer.

SÉQUENCE 2

Les RDD

Resilient Distributed Datasets : ce que Spark répartit, et comment il s'en souvient.

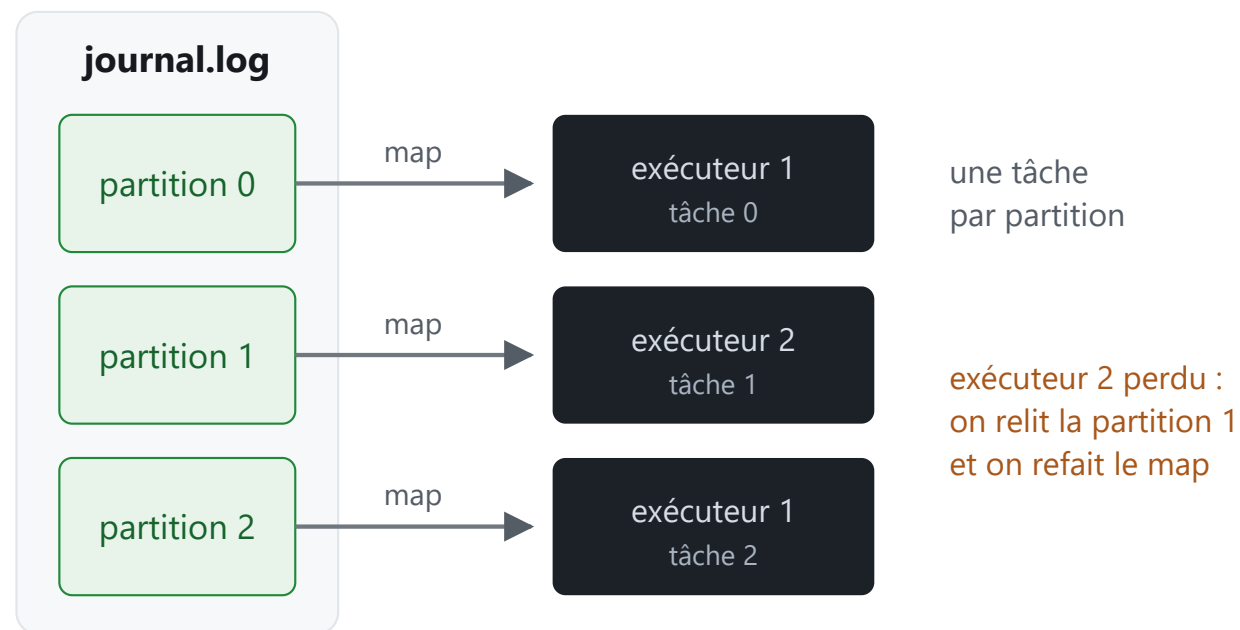
Trois mots, trois propriétés

Distributed Découpé en **partitions**, traitées en parallèle par les exécuteurs.

Dataset Une collection d'objets Python quelconques : lignes, tuples, dictionnaires.

Resilient Une partition perdue se **recalcule** : Spark garde la recette, pas une copie.

Et une quatrième, implicite : un RDD est **immuable**.
Le transformer en crée un nouveau.



Trois façons de créer un RDD

Depuis une collection

```
nombres = sc.parallelize(
    range(1, 11), 4)

nombres.getNumPartitions()
# 4
nombres.glom().collect()
# [[1, 2], [3, 4, 5],
#   [6, 7], [8, 9, 10]]
```

Pour tester, et pour les petites données : la liste part du driver.

Depuis un fichier

```
lignes = sc.textFile(
    "donnees/journal-bornes.log")

lignes.getNumPartitions()
# 2
lignes.count()
# 608411
```

Une ligne de texte par élément. Accepte un dossier, un motif `*.log`, un chemin `hdfs://` ou `s3a://`.

Depuis un DataFrame

```
df = spark.read.csv(
    "donnees/stations.csv",
    header=True)

df.rdd.map(lambda r:
    (r.station_id, r.nom)).take(2)
# [('S001', 'Place du Marché'),
#   ('S002', 'Hôtel de Ville')]
```

Le pont vers le module 3 : chaque ligne devient un objet `Row`.

Transformations et actions

	TRANSFORMATION	ACTION
Renvoie	Un nouveau RDD	Un résultat Python, ou un fichier
S'exécute	Jamais seule : elle s'inscrit dans la recette	Tout de suite : elle déclenche un <i>job</i>
Exemples	map filter flatMap reduceByKey join	count collect take reduce saveAsTextFile

```
carres = nombres.map(lambda x: x * x)
# instantané : rien n'a été calculé

carres.collect() # maintenant, un job
# [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

La paresse permet à Spark de voir **toute la chaîne** avant d'agir, et d'enchaîner les étapes sans écrire de résultat intermédiaire.

map, flatMap, filter

```
phrases = sc.parallelize([
    "le vélo est là",
    "le vélo est parti",
    "il est là"])

phrases.map(lambda p: p.split()).collect()
# [['le', 'vélo', 'est', 'là'],
#  ['le', 'vélo', 'est', 'parti'],
#  ['il', 'est', 'là']]

phrases.flatMap(lambda p: p.split()).collect()
# ['le', 'vélo', 'est', 'là', 'le', 'vélo',
#  'est', 'parti', 'il', 'est', 'là']
```

map

Un élément en entrée, **un** en sortie.

flatMap

Un élément en entrée, **zéro, un ou plusieurs** en sortie : la liste est aplatie.

filter

Garde l'élément si la fonction renvoie vrai.

```
nombres.filter(lambda x: x % 2 == 0).collect()
# [2, 4, 6, 8, 10]
```

Un `flatMap` qui renvoie `[]` pour une ligne invalide, c'est un `map` et un `filter` en un seul passage. On s'en sert au TP.

Les actions, et ce qu'elles ramènent

ACTION	RENVOIE AU DRIVER	À SURVEILLER
<code>count()</code>	Le nombre d'éléments	Toujours sans risque
<code>take(n)</code> <code>first()</code>	Les n premiers éléments	Lit le moins de partitions possible
<code>reduce(f)</code>	Une valeur, combinée deux à deux	<code>f</code> doit être associative et commutative
<code>collect()</code>	Tout le RDD , dans une liste Python	Seulement si le résultat est petit
<code>saveAsTextFile(d)</code>	Rien : écrit un dossier, un fichier par partition	Refuse d'écraser un dossier existant
<code>foreach(f)</code>	Rien : applique <code>f</code> sur les exécuteurs	Un <code>print</code> n'arrive pas dans le notebook

```
nombre.map(lambda x: x * x).reduce(lambda a, b: a + b)
```

`collect()` sur 600 000 lignes les copie toutes dans la mémoire du driver. Sur 600 millions, le driver tombe. On **réduit d'abord**, on ramène ensuite.

SÉQUENCE 3

Map/reduce

Des paires clé-valeur, une réduction par clé, et un échange sur le réseau que l'on apprend à repérer.

Compter les mots, en trois transformations

```
compte = (phrases
  .flatMap(lambda p: p.split())    # un mot par élément
  .map(lambda m: (m, 1))           # une paire (clé, valeur)
  .reduceByKey(lambda a, b: a + b)) # somme par clé

compte.sortBy(lambda kv: kv[1], ascending=False).collect()
# [('est', 3), ('vélo', 2), ('le', 2),
#  ('là', 2), ('il', 1), ('parti', 1)]
```

1 Map

Chaque élément produit des paires (clé, valeur), indépendamment des autres.

2 Shuffle

Les paires de même clé sont regroupées, quelle que soit la partition d'origine.

3 Reduce

Les valeurs d'une clé sont combinées deux à deux.

Toute fonction à deux arguments, associative et commutative, marche dans `reduceByKey` : somme, maximum, minimum, union d'ensembles. Pas la moyenne : on y revient au TP.

La recette d'un RDD

```
compte.toDebugString().decode()

(2) PythonRDD[12] at collect at ...:22 []
| MapPartitionsRDD[11] at mapPartitions ...
| ShuffledRDD[10] at partitionBy ...
+- (2) PairwiseRDD[9] at reduceByKey ...
| PythonRDD[8] at reduceByKey ...
| ParallelCollectionRDD[5] at readRDDFromFile ...
```

Le **lignage** est la chaîne des transformations qui mène au RDD, jusqu'à la source.

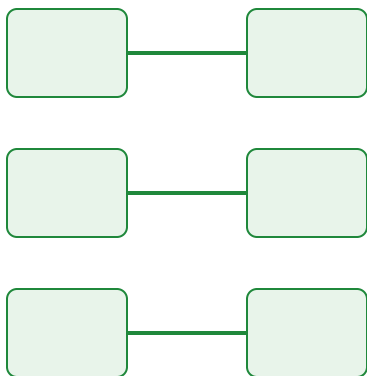
Chaque retrait vers la droite marque une **frontière d'étape** (*stage*) : là où il faut un shuffle.

À quoi il sert

Recalculer une partition perdue, et découper le job en étapes. On retrouve ce même découpage dans l'interface web, onglet *Stages*.

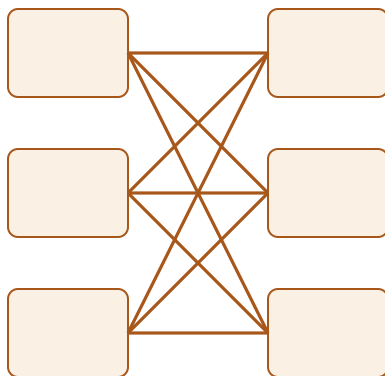
Dépendances étroites, dépendances larges

Étroite · map, filter



sur place, sans réseau

Large · reduceByKey, join



écriture sur disque, puis réseau

Étroite : chaque partition de sortie dépend d'une seule partition d'entrée. Spark les enchaîne dans une même tâche.

Large : chaque partition de sortie dépend de **toutes**. Il faut un *shuffle* : on écrit, on échange, on relit.

Sur un vrai volume, le shuffle est presque toujours **l'étape la plus lente**. Compter les shuffles d'un job, c'est déjà l'optimiser.

groupByKey ou reduceByKey ?

```
paires = sc.parallelize([("S017", 1), ("S023", 1),  
                        ("S017", 1), ("S017", 1)], 2)
```

```
paires.groupByKey().mapValues(sum).collect()  
# [('S017', 3), ('S023', 1)]
```

```
paires.reduceByKey(lambda a, b: a + b).collect()  
# [('S017', 3), ('S023', 1)]
```

Même résultat. Pas le même trajet.

groupByKey

Envoie **chaque** paire sur le réseau, puis réunit toutes les valeurs d'une clé en mémoire.

reduceByKey

Réduit **d'abord dans chaque partition**, puis n'envoie qu'un résultat partiel par clé.

Sur le journal : 9 000 erreurs, 60 stations. `reduceByKey` transporte au plus 60 paires par partition ; `groupByKey` les 9 000.

Le journal des bornes, en map/reduce

```
lignes = sc.textFile("donnees/journal-bornes.log")
champs = (lignes.map(lambda l: l.split(" "))
          .filter(lambda c: len(c) ≥ 3))

champs.map(lambda c: (c[2], 1)) \
    .reduceByKey(lambda a, b: a + b).collect()
# [(' ', 53), ('DEPOT', 299083),
#  ('E', 1), ('ERREUR', 9001),
#  ('R', 25), ('RE', 25), ('RET', 8),
#  ('RETR', 10), ('RETRA', 6),
#  ('RETRAI', 8), ('RETRAIT', 299099)]

(champs.map(lambda c: (c[1], 1))
  .reduceByKey(lambda a, b: a + b)
  .sortBy(lambda kv: kv[1], ascending=False)
  .take(3))
# [('S006', 14909), ('S005', 14811),
#  ('S009', 14519)]
```

Une ligne du journal :

```
2026-01-01T06:03:49 S027 RETRAIT velo=V0696 badge=A-37535
```

Découper sur l'espace donne l'horodatage, la station, le type d'événement : de quoi compter **par type**, puis **par station**.

Un résultat suspect

Des types parasites (' ', 'R', 'RETRA' ...) et 299 099 retraits : les lignes tronquées passent le filtre. Le TP les écarte avec une vraie analyse de ligne.

Joindre deux RDD de paires

```
stations = (sc.textFile("donnees/stations.csv")
    .filter(lambda l: not l.startswith("station_id"))
    .map(lambda l: l.split(","))
    .map(lambda c: (c[0], (c[1], c[2])))) # (id, (nom, quartier))

pannes = (champs.filter(lambda c: c[2] == "ERREUR")
    .map(lambda c: (c[1], 1))
    .reduceByKey(lambda a, b: a + b)) # (id, nombre)

pannes.join(stations).sortByKey().take(2)
# [('S001', (144, ('Place du Marché', 'Centre'))),
#  ('S002', (157, ('Hôtel de Ville', 'Gare')))]
```

La jointure se fait **sur la clé** : les deux RDD doivent être des paires (**clé, valeur**). Le résultat imbrique les deux valeurs : (**clé, (gauche, droite)**).

join

Les clés présentes des deux côtés

leftOuterJoinToutes les clés de gauche ; **None** à droite si absente**rightOuterJoin**

Même principe, à droite, ou des deux côtés

fullOuterJoin**cogroup**

Pour chaque clé, toutes les valeurs de chaque côté

Une jointure est une **dépendance large** : les deux côtés passent par le shuffle. Quand l'un des deux tient dans un dictionnaire, il y a mieux — c'est la séquence suivante.

SÉQUENCE 4

Réutiliser, partager, partitionner

Garder un résultat en mémoire, compter en marge du calcul, envoyer une table à tous, et choisir son découpage.

Sans cache, tout est recalculé

```
evenements = lignes.flatMap(analyser)
evenements.count()  # lit le fichier, analyse
evenements.count()  # relit, réanalyse

evenements.cache()  # marque, ne calcule rien
evenements.count()  # calcule ET remplit le cache
evenements.count()  # lit le cache

evenements.unpersist()  # libère la mémoire
```

MESURÉ AU TP 2

DURÉE

Action sans cache

≈ 5 s

1re action, remplit le cache

≈ 8 s

2e action, lit le cache

≈ 3 s

On met en cache un RDD **réutilisé** par plusieurs actions. Un RDD lu une seule fois n'y gagne rien, et prend de la mémoire.

Les accumulateurs : compter en marge

```
invalides = sc.accumulator(0)

def analyser(ligne):
    morceaux = ligne.split(" ", 3)
    if len(morceaux) < 4:
        invalides.add(1)  # côté exécutateur
    return []
    ...

evenements = lignes.flatMap(analyser)
evenements.count()
invalides.value          # côté driver
# 1246
evenements.count()
invalides.value
# 2492
```

Les exécuteurs **ajoutent**, seul le driver **lit**. C'est le seul moyen de faire remonter un compteur sans action dédiée.

Le piège : 2492 au lieu de 1246

Sans cache, la seconde action rappelle `analyser`, et l'accumulateur compte une deuxième fois. Même effet si une tâche est relancée après une panne.

Dans une **transformation**, aucune garantie : le cache évite le recomptage d'une action à l'autre, pas une éviction ni une tâche relancée. Dans une **action** (`foreach`), Spark garantit un seul comptage par tâche.

Les variables diffusées (*broadcast*)

```
noms = (sc.textFile("donnees/stations.csv")
        .filter(lambda l: not l.startswith("station_id"))
        .map(lambda l: l.split(","))
        .map(lambda c: (c[0], c[1]))
        .collectAsMap())          # un dict, côté driver

noms_diffuses = sc.broadcast(noms)

(evenements
 .filter(lambda e: e[2] == "ERREUR")
 .map(lambda e: noms_diffuses.value[e[1]])
 ... )
```

Une table de référence, **en lecture seule**, envoyée **une fois par exécutateur** au lieu d'une fois par tâche.

Sans broadcast	Le dictionnaire voyage avec chaque tâche, dans la fermeture de la <code>lambda</code> .
Avec broadcast	Il est copié une fois sur chaque exécutateur, puis réutilisé.

Joindre 600 000 événements à 60 stations par un `join`, c'est un shuffle. Par un broadcast, c'est une consultation de dictionnaire.

Choisir le nombre de partitions

```
lignes.getNumPartitions()          # 2
lignes.repartition(8).getNumPartitions() # 8
lignes.coalesce(1).getNumPartitions()   # 1

nombres.repartition(3).glom().map(len).collect()
# [5, 3, 2]
nombres.coalesce(2).glom().map(len).collect()
# [5, 5]
```

Compter par partition, sans rien ramener d'autre :

```
def compter(partition):
    yield sum(1 for _ in partition)

lignes.mapPartitions(compter).collect()
# [304210, 304201]
```

`repartition(n)`

Augmente ou réduit. Redistribue tout : **un shuffle**.

`coalesce(n)`

Réduit seulement, en fusionnant des partitions voisines : **pas de shuffle**.

`mapPartitions`

Une fonction appelée une fois **par partition** : pour une connexion, un modèle chargé une fois.

Repère : 2 à 4 partitions par cœur disponible. Trop peu, des cœurs attendent. Trop, le coût d'ordonnancement domine.

Partitionner par clé avec `partitionBy`

```
par_station = paires.partitionBy(2)
par_station.glom().collect()
# [('S017', 1), ('S017', 1), ('S017', 1)],
# [('S023', 1)]
```

Toutes les paires d'une même clé se retrouvent dans **la même partition**, placée par hachage de la clé.

À quoi ça sert

Un RDD partitionné et mis en cache peut être joint ou réduit par clé **sans nouveau shuffle** : les clés sont déjà à leur place.

Quand s'en passer

Si le RDD n'est utilisé qu'une fois, `partitionBy` ajoute un shuffle au lieu d'en économiser un.

SÉQUENCE 5

Notebooks et jobs

Le notebook pour explorer, le script pour produire, et `spark-submit` entre les deux.

Du notebook au job soumis

Le notebook : explorer

Une session vivante, des résultats cellule par cellule. Idéal pour mettre au point, fragile pour produire : l'ordre d'exécution des cellules n'est écrit nulle part.

Le script : produire

Le même code dans un `.py`, exécuté de bout en bout, planifiable, relisible.

```
if __name__ == "__main__":
    spark = SparkSession.builder \
        .appName("journal").getOrCreate()
    ...
    spark.stop()
```

```
spark-submit \
  --master "local[2]" \
  --py-files mes_modules.zip \
  corrige.py
```

<code>--master</code>	<code>local[2], yarn, spark://hote:7077, k8s:// ...</code>
-----------------------	--

<code>--py-files</code>	Vos modules Python, expédiés aux exécuteurs
-------------------------	---

<code>--archives</code>	Un environnement Python empaqueté, pour les dépendances
-------------------------	---

Ne pas écrire `.master( ... )` dans un script destiné à un cluster : la valeur du code l'emporte sur celle de `spark-submit`.

Les RDD aujourd'hui

Les RDD sont la fondation : tout, dans Spark, finit par s'exécuter sur eux. Mais on les écrit de moins en moins.

	RDD	DATAFRAME
Schéma	Aucun : des objets Python	Colonnes typées
Optimisation	Exécute ce qu'on écrit	Réécrit le plan (Catalyst)
Coût Python	Chaque ligne passe par Python	Reste dans la JVM
Spark Connect	Non pris en charge	Pris en charge

Quand descendre aux RDD

- Du texte sans structure, comme notre journal
- Un contrôle fin du partitionnement
- Du code existant, ou une API qui l'exige

Demain matin, le même calcul en DataFrame : **plus court, et plus rapide.**

Analyser le journal brut des bornes

≈ 1 h 30

Seul ou en binôme

Restitution 15 min

Échauffement : `exercices/serie2-rdd/`

Dossier `tp2-rdd/` : l'énoncé `enonce.md` et le notebook `depart.ipynb`.

1 Analyser chaque ligne, écarter les invalides

Un `flatMap`, un accumulateur, et le piège du double comptage.

2 Mettre en cache, et mesurer

La même action, avec et sans cache.

3 Répondre par map/reduce

Événements par type, dix stations les plus en panne avec leur nom (broadcast), codes d'erreur, batterie moyenne.

4 Écrire, puis soumettre

`saveAsTextFile`, puis le notebook devenu script, lancé par `spark-submit`.

Ce qu'on attend en restitution

La station la plus en panne, le nombre de lignes écartées, et **le gain mesuré du cache** sur votre poste.

Le piège de l'énoncé

La batterie moyenne ne se calcule pas avec `reduceByKey` sur des moyennes. Trouver pourquoi fait partie de l'exercice.

Cinq questions avant de fermer le jour 1

Q1 On écrit `rdd.map(lambda x: 1 / x)` et la cellule passe sans erreur, alors que `rdd` contient un zéro. Pourquoi ?

`map` est une transformation : rien n'est calculé. L'erreur surgira à la première action.

Q2 Qu'est-ce qui distingue une dépendance large d'une dépendance étroite, et pourquoi s'en soucier ?

Une partition de sortie dépend de toutes les partitions d'entrée : il faut un shuffle, donc disque et réseau. C'est presque toujours l'étape la plus lente.

Q3 Pourquoi préférer `reduceByKey` à `groupByKey` pour compter ?

Il réduit dans chaque partition avant le shuffle : seules les sommes partielles voyagent, et aucune clé n'a à tenir entièrement en mémoire.

Q4 Un accumulateur affiche le double de la valeur attendue. Quelle est la cause probable ?

Il est incrémenté dans une transformation, sur un RDD non mis en cache, lu après deux actions : le calcul a été refait.

Q5 Une table de 60 stations doit être consultée pour chaque ligne d'un journal de 600 000 lignes. Que faites-vous ?

Une variable diffusée (`sc.broadcast`) : copiée une fois par exécuteur, sans shuffle, au lieu d'un `join`.

ANNEXES DU MODULE 2 AVANCÉ

Sous le capot des RDD, et la mise en production

Niveaux de stockage et points de reprise, nombre de partitions, clés déséquilibrées, agrégations générales, puis `spark-submit` et l'emballage d'un job.

Niveaux de stockage et points de reprise

```
from pyspark import StorageLevel

evenements.persist(StorageLevel.MEMORY_AND_DISK)
evenements.getStorageLevel()
# Disk Memory Serialized 1x Replicated
```

MEMORY_ONLY	Ce qui ne tient pas est recalculé
MEMORY_AND_DISK	Ce qui ne tient pas déborde sur disque
DISK_ONLY	Tout sur disque local
..._2	Répliqué sur deux exécutants

Couper le lignage

```
sc.setCheckpointDir("sorties/points-de-reprise")
evenements.checkpoint()
evenements.count() # écrit, puis oublie la recette
```

Un **point de reprise** écrit le RDD sur un stockage fiable et remplace son lignage. Utile quand la recette devient très longue : algorithmes itératifs, dizaines de jointures.

En PySpark, les données en cache sont toujours sérialisées : les variantes `_SER` de Scala n'existent pas dans `StorageLevel`.

Combien de partitions ? Trois symptômes

```
rdd = sc.parallelize(range(1000), n)
rdd.map(lambda x: x + 1).sum() # travail négligeable
```

PARTITIONS	DURÉE
2	4,3 s
8	15,3 s
32	54,4 s

Mille éléments, presque aucun calcul : ces durées ne mesurent que le **coût fixe d'une tâche**.

Trop peu

Des cœurs inoccupés, des tâches énormes, et la mémoire qui déborde sur disque

Trop

Le coût fixe par tâche domine le calcul utile, et l'écriture produit une nuée de fichiers minuscules

Mal réparties

Une tâche dure dix fois plus que les autres : c'est le déséquilibre, slide suivante

Repères : **2 à 4 tâches par cœur**, et des partitions de lecture de l'ordre de 100 à 200 Mo. `sc.textFile(chemin, minPartitions=8)` à la lecture, `repartition` ou `coalesce` ensuite.

Partitionneur personnalisé et clés déséquilibrées

```
def par_quartier(station):  
    # stations S001 à S060, 8 quartiers  
    return (int(station[1:]) - 1) % 8  
  
paires.partitionBy(8, par_quartier)
```

La fonction reçoit une clé et renvoie un numéro de partition.
Utile pour regrouper ce qui sera traité ensemble.

Le déséquilibre (*skew*)

Si une clé porte la moitié des données, sa partition porte la moitié du travail : une tâche finit en dix minutes, les autres en dix secondes.

Le remède classique : saler la clé

```
import random  
(paires  
    .map(lambda kv: ((kv[0], random.randrange(8)), kv[1]))  
    .reduceByKey(lambda a, b: a + b) # 8 morceaux par clé  
    .map(lambda kv: (kv[0][0], kv[1]))  
    .reduceByKey(lambda a, b: a + b)) # on recombine
```

aggregateByKey : une moyenne par clé

```
batteries = (evenements
  .filter(lambda e: e[2] == "RETRAIT" and "batterie" in e[3])
  .map(lambda e: (e[1], int(e[3]["batterie"]))))

moyennes = (batteries
  .aggregateByKey(
    (0, 0), # valeur initiale
    lambda acc, v: (acc[0] + v, acc[1] + 1), # dans une partition
    lambda a, b: (a[0] + b[0], a[1] + b[1])) # entre partitions
  .mapValues(lambda s: round(s[0] / s[1], 1)))
```

`reduceByKey` impose que l'entrée et le résultat aient le même type. `aggregateByKey` lève cette contrainte : on accumule un couple `(somme, effectif)` à partir d'entiers.

`combineByKey` est la forme la plus générale : la valeur initiale y est une fonction de la première valeur rencontrée.

Les deux font le même combineur côté carte que `reduceByKey` : pas de `groupByKey` déguisé.

spark-submit : trois blocs, et un fichier

```
spark-submit \
  --master "local[2]" \
  --properties-file conf/prod.conf \
  --conf spark.sql.shuffle.partitions=8 \
  --py-files dist/velocite.zip \
  lancer.py \
  donnees/journal-bornes.log sorties/pannes
```

```
conf/prod.conf
spark.app.name          pannes-prod
spark.sql.shuffle.partitions 64
spark.ui.showConsoleProgress false
```

1 La configuration Spark

Tout ce qui précède le fichier : où tourner, avec quelles ressources, quel code embarquer.

2 Le livrable

Un seul fichier `.py`, le point d'entrée.

3 Les arguments métier

Ce qui suit : lus par votre script dans `sys.argv`.

QUI GAGNE

EXEMPLE MESURÉ

- | | |
|------------------------|---|
| 1. Le code | <code>appName("pannes")</code> l'emporte sur <code>spark.app.name</code> du fichier |
| 2. <code>--conf</code> | <code>=8</code> l'emporte sur les <code>64</code> du fichier |
| 3. Le fichier | Sert quand ni le code ni la ligne de commande ne disent rien |

Un fichier par environnement, versionné avec le code : la commande reste courte, et la recette de production est lisible.

Emballer un job PySpark

```

velocite/
├── pyproject.toml      nom, version, dépendances
├── src/velocite/
│   ├── analyse.py     le métier, testable
│   └── jobs/pannes.py  def main()
└── lancer.py          le point d'entrée
    
```

```

mkdir dist
python -m zipfile -c dist/velocite.zip src/velocite

spark-submit --py-files dist/velocite.zip lancer.py ...
    
```

Le `.zip` est ajouté au chemin Python de **chaque exécuteur** :
`from velocite.analyse import analyser` y fonctionne.

Deux surprises, vérifiées en 3.5.4

- `spark-submit -m paquet.module` n'existe pas :
Unrecognized option: -m. D'où le petit `lancer.py`.
- Une roue `.whl` passée à `--py-files` est copiée mais **pas** ajoutée au chemin des exécuteurs (`ModuleNotFoundError`) : seuls `.zip`, `.egg` et `.py` le sont. Renommer la roue en `.zip` suffit.

Les **dépendances tierces** ne voyagent pas dans ce zip : on les expédie avec `--archives` (un `venv` emballé), ou elles sont déjà installées sur le cluster.

JOUR 2 · MODULE 3

Spark SQL et DataFrames

Donner un schéma aux données, les interroger en SQL, et laisser Spark optimiser à notre place.

JOUR 2 · MODULE 3

Spark SQL et DataFrames

Manipuler des données structurées : des tables avec un schéma, que Spark sait optimiser à votre place.

DURÉE	À LA FIN	TP
≈ 1 h 50 d'exposé + TP 1 h 20	Lire, nettoyer, joindre et interroger une table, en Python comme en SQL	Nettoyer, joindre, interroger

Ce qu'on garde d'hier

Le code part aux exécuteurs

Le driver construit le plan ; les exécuteurs travaillent sur les **partitions**. Une fonction passée à `map` ne s'exécute pas là où on l'a écrite.

Rien ne se passe avant l'action

Les transformations sont **paresseuses** : elles décrivent un calcul. Seule une action (`count`, `collect`, écriture) déclenche un job.

Le shuffle coûte cher

`reduceByKey` et `groupByKey` redistribuent les données entre partitions. C'est là que le temps passe.

Hier, **vous** disiez à Spark *comment* calculer. Aujourd'hui, vous lui direz *quoi* calculer, et il choisira le comment.

Ce que couvre le module 3

- 1 DataFrames et Spark SQL**
Schéma, colonnes, sélection, agrégation, jointure, SQL, fenêtres.
- 2 Sous le capot : Catalyst**
Lire un plan d'exécution. Le mode ANSI.
- 3 Sources et formats**
CSV, JSON, Parquet, ORC, JDBC. Lire, écrire, partitionner.
- 4 RDD et pandas**
Passer d'un monde à l'autre, et l'API pandas de Spark.
- 5 TP 3 tutoré**
Nettoyer, joindre, interroger les trajets.

Le fil de la matinée

On repart des trajets bruts de Vélo'Cité, cette fois avec un **schéma**.

La question qui structure tout : **qu'est-ce que Spark peut faire de mieux quand il sait ce que contiennent vos colonnes ?**

SÉQUENCE 1

DataFrames et Spark SQL

Des tables distribuées, avec des colonnes nommées et typées.

Ce qu'un RDD ne dit pas à Spark

Un RDD de tuples

```
(trajets.map(lambda t: (t[2], t[6]))  
      .filter(lambda p: p[1] > 0)  
      .reduceByKey(lambda a, b: a + b))
```

Pour Spark, `t` est un objet Python opaque. Il ne sait ni ce qu'est `t[6]`, ni ce que fait la `lambda`. Il exécute dans l'ordre écrit.

Un DataFrame

```
(trajets.filter(F.col("duree_min") > 0)  
      .groupBy("station_depart")  
      .agg(F.sum("duree_min")))
```

Spark connaît les colonnes, leurs types, et le sens de `>` et de `sum`. Il peut **réordonner**, ne lire que deux colonnes, filtrer à la lecture.

Le gain ne vient pas d'une syntaxe plus courte. Il vient de ce que Spark **comprend** le calcul, donc peut l'optimiser, et l'exécuter dans la JVM sans passer par Python.

DataFrame, Dataset, Row : ce qu'on a en Python

	SCALA ET JAVA	PYTHON
Dataset[T]	Une collection typée à la compilation : <code>Dataset[Trajet]</code>	N'existe pas : Python n'a pas de typage à la compilation
DataFrame	Un alias : <code>Dataset[Row]</code>	L'API, tout court. C'est aussi un <code>Dataset[Row]</code> sous le capot
Row	Une ligne générique	Une ligne, qui se lit comme un tuple nommé : <code>r.duree_min</code> ou <code>r["duree_min"]</code>
Schéma	Un <code>StructType</code> : la liste des colonnes, avec leur nom, leur type, et si elles acceptent <code>null</code>	

Le programme parle de « DataFrames et datasets ». En Python, c'est **la même chose** : quand la documentation dit Dataset, lisez DataFrame. Le typage que Scala vérifie à la compilation, Python le vérifie à l'exécution, grâce au schéma.

Lire, regarder, compter

```
tr = (spark.read
      .option("header", True)
      .schema(ddl)
      .csv("donnees/trajets.csv"))

tr.printSchema()
tr.show(3)
tr.count()    # 300300
tr.first()
```

Une ligne rendue au driver est un **Row** :

```
Row(trajet_id='T0299341', velo_id='V0696',
     station_depart='S027', station_arrivee='S050',
     debut=datetime.datetime(2026, 1, 1, 6, 3, 49),
     duree_min=5.0, distance_km=1.12,
     type_usager='abonne', electrique=0, ...)
```

```
root
├─ trajet_id: string (nullable = true)
├─ velo_id: string (nullable = true)
├─ station_depart: string (nullable = true)
├─ station_arrivee: string (nullable = true)
├─ debut: timestamp (nullable = true)
├─ fin: timestamp (nullable = true)
├─ duree_min: double (nullable = true)
├─ distance_km: double (nullable = true)
├─ type_usager: string (nullable = true)
├─ electrique: integer (nullable = true)
├─ tarif_eur: double (nullable = true)
```

Trois actions, trois jobs

`show`, `count` et `first` sont des **actions**. `printSchema` n'en est pas une : le schéma est connu sans lire les données.

Le deviner, ou le déclarer

Sans rien dire : tout est texte

```
spark.read.option("header", True).csv(f)
# duree_min: string, debut: string, ...
```

inferSchema : Spark devine

```
(spark.read.option("header", True)
    .option("inferSchema", True).csv(f))
# duree_min: double, debut: timestamp
```

Pour deviner, Spark **lit tout le fichier** une première fois : un job complet avant même votre première action.

Déclarer : une chaîne DDL suffit

```
ddl = """trajet_id STRING, velo_id STRING,
station_depart STRING, station_arrivee STRING,
debut TIMESTAMP, fin TIMESTAMP,
duree_min DOUBLE, distance_km DOUBLE,
type_usager STRING, electrique INT,
tarif_eur DOUBLE"""
```

```
tr = (spark.read.option("header", True)
    .schema(ddl).csv(f))
```

Mesuré sur les 300 300 trajets

Lecture avec `inferSchema` : **34 s** sur un poste chargé. Avec le schéma déclaré : **0,35 s**, car rien n'est lu.

Une colonne est une expression, pas des données

```
from pyspark.sql import functions as F

vitesse = F.col("distance_km") / (F.col("duree_min") / 60)
print(vitesse)
# Column<'(distance_km / (duree_min / 60))'>
```

Rien n'est calculé : `vitesse` est une **formule**, que Spark appliquera à chaque ligne quand une action la demandera.

Trois façons de désigner une colonne

<code>F.col("duree_min")</code>	La plus sûre, marche partout
---------------------------------	------------------------------

<code>tr.duree_min</code>	Courte, mais attachée à <code>tr</code>
---------------------------	---

<code>"duree_min"</code>	Là où une chaîne est acceptée
--------------------------	-------------------------------

Importer `functions` sous l'alias `F` est la convention : `from pyspark.sql.functions import *` écrase `sum`, `max` et `round` de Python.

select, filter, withColumn

```
(tr.select("station_depart", "duree_min",
          "type_usager")
  .filter(F.col("duree_min") > 60)
  .show(3))
```

station_depart	duree_min	type_usager
S053	92.0	occasionnel
S010	74.3	occasionnel
S053	65.9	occasionnel

2 024 trajets dépassent l'heure.

Un DataFrame est **immutable** : `withColumn` renvoie un nouveau DataFrame. Oublier d'affecter le résultat, c'est ne rien avoir fait.

```
tr2 = tr.withColumn(
  "vitesse_kmh",
  F.round(F.col("distance_km")
    / (F.col("duree_min") / 60), 1))
```

<code>select</code>	Choisir ou calculer des colonnes
<code>filter</code> / <code>where</code>	Garder des lignes, deux noms pour la même chose
<code>withColumn</code>	Ajouter ou remplacer une colonne
<code>drop</code>	Retirer des colonnes

groupBy et agg

```
(tr2.groupBy("type_usager")
  .agg(F.count("*").alias("trajets"),
    F.round(F.avg("duree_min"), 1)
      .alias("duree_moy"),
    F.round(F.avg("vitesse_kmh"), 1)
      .alias("vitesse_moy"))
  .orderBy("type_usager")
  .show())
```

type_usager	trajets	duree_moy	vitesse_moy
NULL	150	12.2	13.0
abonne	190269	8.6	17.8
occasionnel	109881	20.4	5.3

Le reduceByKey d'hier

Un `groupBy` suivi d'un `agg` fait le même travail : agrégation partielle dans chaque partition, un shuffle, puis l'agrégation finale.

La ligne NULL

Les 150 trajets sans type d'utilisateur forment **un groupe à part**. Ils sont là, visibles : c'est le premier défaut que le TP devra traiter.

Rattacher le quartier à chaque trajet

```
stations = (spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv("donnees/stations.csv"))

avec_quartier = tr.join(
    stations.select(
        F.col("station_id").alias("station_depart"),
        "quartier"),
    on="station_depart",
    how="left")
```

HOW	GARDE
<code>inner</code>	Les lignes présentes des deux côtés (défaut)
<code>left</code>	Tous les trajets, quartier <code>null</code> si inconnu
<code>left_anti</code>	Les trajets sans station connue : un contrôle de qualité en une ligne
<code>left_semi</code>	Les trajets avec station, sans ajouter de colonne

Joindre sur une condition (`a.x == b.y`) garde les deux colonnes de clé. Renommer avant, et joindre avec `on=`, évite une colonne en double.

Le module functions, en un coup d'œil

FAMILLE	EXEMPLES	SUR VÉLO'CITÉ
Dates	<code>to_date</code> , <code>hour</code> , <code>dayofweek</code> , <code>date_trunc</code>	L'heure de départ, le jour de la semaine
Conditions	<code>when(...).otherwise(...)</code> , <code>coalesce</code>	Classer un trajet court, moyen, long
Valeurs manquantes	<code>isNull</code> , <code>dropna</code> , <code>fillna</code>	Les 300 trajets sans arrivée
Chaînes	<code>regexp_extract</code> , <code>split</code> , <code>upper</code>	Le numéro d'une station
Agrégats	<code>count</code> , <code>countDistinct</code> , <code>percentile_approx</code>	La durée médiane par quartier

Avant d'écrire une fonction Python, **cherchez-la dans `functions`**. Il y en a plus de trois cents, et elles s'exécutent toutes sans quitter la JVM, à la différence des `udf`.

La même question, en SQL

```
tr.createOrReplaceTempView("trajets")

spark.sql("""
    SELECT type_usager,
           COUNT(*) AS trajets,
           ROUND(AVG(duree_min), 1) AS duree_moy
    FROM trajets
    WHERE duree_min > 0
    GROUP BY type_usager
    ORDER BY trajets DESC
""").show()
```

type_usager	trajets	duree_moy
abonne	190090	8.7
occasionnel	109761	20.5
NULL	149	12.3

Une vue, pas une copie

`createOrReplaceTempView` donne un nom au DataFrame pour la durée de la session. Rien n'est écrit ni recopié.

API ou SQL ?

Les deux produisent **le même plan**, donc les mêmes performances. Choisissez selon le lecteur : SQL pour une requête qu'un analyste relira, l'API pour du code qu'on factorise, qu'on paramètre et qu'on teste.

`spark.sql( ... )` renvoie un DataFrame : on peut enchaîner l'API derrière.

Classer sans perdre les lignes

Un `groupBy` **écrase** chaque groupe en une ligne.

Une fenêtre calcule **à côté** de chaque ligne, en regardant ses voisines du même groupe.

<code>rank</code>	Le rang, avec ex æquo
<code>row_number</code>	Le rang, sans ex æquo
<code>lag</code> / <code>lead</code>	La ligne d'avant, d'après
<code>sum().over()</code>	Un cumul

```
from pyspark.sql import Window

w = (Window.partitionBy("quartier")
      .orderBy(F.desc("trajets")))

(par_station
 .withColumn("rang", F.row_number().over(w))
 .filter("rang ≤ 2")
 .show(6))
```

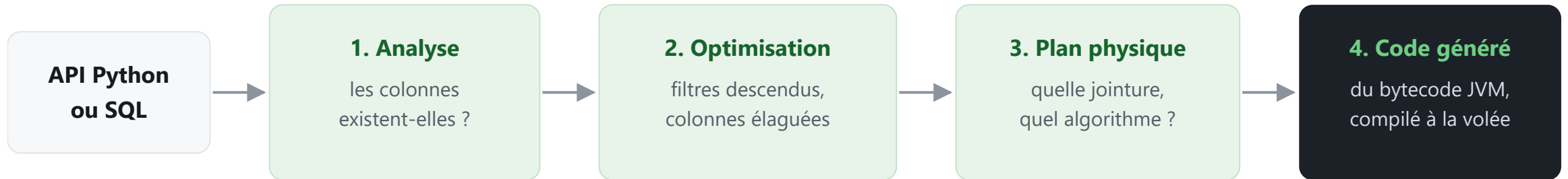
```
+-----+-----+-----+-----+
|quartier|      nom|trajets|rang|
+-----+-----+-----+-----+
|Affaires|Pépinère| 5127| 1|
|Affaires|Tour Horizon| 5066| 2|
|Berges|Halles| 5122| 1|
|Berges|Patinoire| 5099| 2|
|Centre|Carré d'Or| 5107| 1|
|Centre|Musée| 5062| 2|
+-----+-----+-----+-----+
```


SÉQUENCE 2

Sous le capot : Catalyst

Ce que Spark fait de votre requête avant de l'exécuter.

De la requête au plan physique



Tout se passe sur le driver, avant le premier job. Les exécuteurs exécutent le plan physique de l'étape 4.

Pushdown des prédicats : le filtre `duree_min > 0`, écrit après la jointure, est exécuté **à la lecture**, avant.

Élagage des colonnes : sur onze colonnes, seules celles que la requête utilise sont lues.

Lire un plan, de bas en haut

```
q = (tr.filter(F.col("duree_min") > 0)
      .join(stations,
            tr.station_depart
            == stations.station_id)
      .groupBy("quartier")
      .count())
q.explain()
```

1 Scan

2 colonnes lues sur 11, filtre poussé dans la lecture

2 BroadcastHashJoin

les 60 stations envoyées à chaque exécuter

3 partial_count, Exchange, count

le reduceByKey d'hier : un shuffle

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[quartier],
|               functions=[count(1)])
+- Exchange hashpartitioning(quartier, 200)
+- HashAggregate(keys=[quartier],
|               functions=[partial_count(1)])
+- Project [quartier]
+- BroadcastHashJoin
|   [station_depart], [station_id]
:- Project [station_depart]
:   +- Filter (duree_min > 0.0 AND
:   |   isnotnull(station_depart) ...)
:   +- FileScan csv
:       [station_depart, duree_min]
:       PushedFilters: [IsNotNull(duree_min),
:       GreaterThan(duree_min,0.0),
:       IsNotNull(station_depart)]
+- BroadcastExchange
+- Filter isnotnull(station_id)
+- FileScan csv
    [station_id, quartier]
```

Transformations, actions, et cache

TRANSFORMATIONS

`select`, `filter`, `withColumn`, `join`,
`groupBy().agg()`, `orderBy`

Construisent le plan

ACTIONS

`show`, `count`, `collect`,
`first`, `toPandas`,
`write`

Le font exécuter

```
propres = (tr.dropDuplicates()
            .filter("duree_min > 0"))
propres.cache()      # marque, ne calcule pas
propres.count()      # calcule, et garde
propres.groupBy("type_usager").count().show()
                    # relit la mémoire
```

Sans `cache`, chaque action relit le CSV et refait le dédoublonnage.

Spark 3.5 ne se plaint pas : il met NULL

```
spark.sql("SELECT CAST('douze' AS INT)").show()
# +-----+
# |CAST(douze AS INT)|
# +-----+
# |                NULL|
# +-----+

spark.sql("SELECT 10 / 0").show()
# NULL, là aussi
```

Par défaut, `spark.sql.ansi.enabled` vaut `false` : une conversion impossible ou une division par zéro donne `NULL`, **sans erreur ni avertissement.**

Rendre l'erreur visible

```
spark.conf.set("spark.sql.ansi.enabled", "true")
# CAST('douze' AS INT)
# NumberFormatException:
#   [CAST_INVALID_INPUT] ...
# 10 / 0
# [DIVIDE_BY_ZERO] Division by zero.
```

Tolérer, mais le dire

```
F.expr("try_cast(v AS INT)")
F.expr("try_divide(a, b)")
```

Spark 4 inverse le défaut : le mode ANSI y est activé. Du code qui « marche » en 3.5 peut y échouer.

SÉQUENCE 3

Sources et formats

Lire et écrire : le format choisi décide d'une grande partie des performances.

Les formats que Spark lit et écrit

FORMAT	ORGANISATION	SCHÉMA	À RETENIR
CSV	Lignes, texte	Deviné ou déclaré	Universel, lent, fragile : séparateurs, guillemets, dates
JSON	Lignes, texte	Deviné	Une ligne = un objet. Structures imbriquées ; les entiers sortent en <code>Long</code>
Parquet	Colonnes, binaire	Stocké dans le fichier	Le format par défaut de Spark. Compressé, typé, filtrable à la lecture
ORC	Colonnes, binaire	Stocké dans le fichier	Même famille que Parquet, surtout présent dans l'écosystème Hive
JDBC	Une base SQL	Celui de la table	Lecture parallèle si l'on donne une colonne de partition, sinon une seule connexion

```
df = spark.read.format("parquet").load(chemin)      # forme générale
df.write.mode("overwrite").parquet(chemin)         # error, overwrite, append, ignore
```

Ranger par colonne change tout

CSV : ligne après ligne

T01 · V0696 · S027 · S050 · ... · 5.0 · 1.12 · abonne · 0 · 0.0

T02 · V0950 · S044 · S019 · ... · 5.3 · 1.88 · abonne · 0 · 0.0

Pour lire la durée, on lit tout.

```
*(1) Filter (duree_min > 60.0 ...)  
+- *(1) ColumnarToRow  
  +- FileScan parquet [trajet_id, duree_min]  
    PushedFilters: [IsNotNull(duree_min),  
                    GreaterThan(duree_min,60.0)]  
    ReadSchema: struct<trajet_id:string,  
                      duree_min:double>
```

Parquet : colonne après colonne

trajet_id
T01 T02 ...

station
S027 S044 ...

duree_min
5.0 5.3 ...

type_usager
abonne ...

On ne lit que la colonne demandée, et on saute les blocs dont le min et le max excluent le filtre.

trajets.csv

27,2 Mo

Nettoyé, en Parquet (TP 3)

10,3 Mo, soit 2,6 fois moins

partitionBy : un dossier par valeur

```
(tr.withColumn("mois", F.month("debut"))  
  .write.mode("overwrite")  
  .partitionBy("mois")  
  .parquet("sorties/trajets-par-mois"))
```

```
trajets-par-mois/  
mois=1/ part-00000-....parquet  
mois=2/ ...  
...  
mois=6/  
_SUCCESS
```

```
(spark.read.parquet("sorties/trajets-par-mois")  
  .filter("mois = 3")  
  .explain())
```

```
FileScan parquet [... , mois]  
PartitionFilters: [isnotnull(mois),  
                  (mois = 3)]
```

Spark n'ouvre aucun fichier des cinq autres dossiers : c'est l'**élagage de partitions**.

Partitionner par une colonne à forte cardinalité (l'identifiant de trajet) produit des centaines de milliers de petits fichiers : le remède devient le mal.

SÉQUENCE 4

RDD et pandas

Passer d'un monde à l'autre, et savoir ce que ça coûte.

Du DataFrame au RDD, et retour

Descendre

```
tr.rdd.take(1)
# [Row(trajet_id='T0299341', ...)]
```

Remonter

```
rdd = sc.parallelize([("S001", 12),
                      ("S002", 7)])
rdd.toDF(["station", "n"]).collect()
# [Row(station='S001', n=12), ...]

spark.createDataFrame(
    rdd, "station STRING, n INT")
```

Quand descendre au RDD

- Des données **sans structure** : le journal des bornes d'hier, avant qu'on l'ait découpé
- Un contrôle fin du **partitionnement**
- Du code existant écrit avec les RDD

Ce que ça coûte

En passant par `.rdd`, Catalyst ne voit plus rien, et chaque ligne traverse vers Python. Parsez en RDD, puis **remontez en DataFrame au plus tôt**.

Écrire du pandas, exécuter du Spark

```
import pyspark.pandas as ps

psdf = ps.read_csv("donnees/trajets.csv")
psdf.shape
# (300300, 11)

(psdf.groupby("type_usager")["duree_min"]
 .mean())
# occasionnel      20.437061
# abonne           8.645687
```

La ligne du rappel pandas d'hier matin, **au mot près**, sur un DataFrame distribué.

2019

Koalas : projet séparé, de Databricks, pour offrir l'API pandas sur Spark

SPARK 3.2 · 2021

Koalas intégré à Spark sous le nom `pyspark.pandas`. Le projet Koalas est archivé

```
df.pandas_api()      # Spark → pandas-on-Spark
psdf.to_spark()       # pandas-on-Spark → Spark
```

Savoir où vivent vos données

	PANDAS	PANDAS-ON-SPARK	SPARK DATAFRAME
Où sont les données	Mémoire du driver	Exécuteurs	Exécuteurs
Évaluation	Immédiate	Paresseuse	Paresseuse
Taille maximale	La RAM d'une machine	Celle du cluster	Celle du cluster
API	pandas	pandas, presque toute	Spark, et SQL

```
pdf = df.toPandas()      # Spark → pandas
pdf = psdf.to_pandas()   # idem, depuis ps
```

`toPandas` rapatrie **tout** sur le driver. Sur 500 Go, c'est une erreur mémoire garantie.

Le bon usage

Agréger dans Spark, puis `toPandas()` sur le **résultat** : quelques centaines de lignes, pour un graphique ou un export. Apache Arrow accélère ce transfert, une fois

`spark.sql.execution.arrow.pyspark.enabled` à `true` (désactivé par défaut en 3.5).

UDF : le dernier recours

UDF Python, ligne par ligne

```
@F.udf("double")
def vitesse(d, m):
    return d / (m / 60) if m else None
```

UDF pandas, par lots

```
import pandas as pd

@F.pandas_udf("double")
def vitesse_v(d: pd.Series,
              m: pd.Series) → pd.Series:
    return d / (m / 60)
```

Ce qu'une UDF coûte

Catalyst ne voit pas dedans : pas de pushdown à travers. Et chaque valeur part vers un processus Python puis revient.

L'ordre des préférences

1. Une fonction de `F`, ou une expression de colonnes
2. Une UDF pandas : des lots, transmis par Arrow — utile si le calcul par ligne est lourd
3. Une UDF Python, si rien d'autre ne va

Nettoyer, joindre, interroger

≈ 1 h 20

En binôme

Corrigé projeté étape par étape

Échauffement : [exercices/serie3-dataframes/](#)

Dossier `tp3-sql-dataframes/` : ouvrez `depart.ipynb`, l'énoncé est dans `enonce.md`.

1 Lire avec un schéma, compter les défauts

300 300 lignes brutes, puis durées négatives, arrivées et types vides, doublons.

2 Nettoyer et enrichir

Heure, jour, week-end ; quartier de départ et météo du jour.

3 Quatre questions, en API et en SQL

Quartiers, heures de pointe, effet de la pluie, trajets les plus fréquents. Plus une fenêtre, et une question en pandas.

4 Écrire en Parquet

`sorties/trajets-propres` : les TP 4, 5 et 6 en repartent.

Le chiffre de contrôle

Après nettoyage : **299 251 trajets**. Si vous n'avez pas ce nombre, ne passez pas à l'étape 3.

Ce qui est évalué

Compter avant de supprimer, et produire une table que quelqu'un d'autre réutilise sans la relire.

Cinq questions avant de fermer le module 3

Q1 Pourquoi le même calcul est-il en général plus rapide avec un DataFrame qu'avec un RDD ?

Spark connaît le schéma et le sens des expressions : Catalyst peut pousser les filtres, élaguer les colonnes, et tout s'exécute dans la JVM, sans aller-retour vers Python.

Q2 En Python, quelle est la différence entre un Dataset et un DataFrame ?

Aucune : le Dataset typé n'existe qu'en Scala et en Java. En Python, un DataFrame est un `Dataset[Row]`.

Q3 Vous lisez un CSV de 200 Go avec `inferSchema`. Qu'est-ce que ça coûte ?

Une lecture complète du fichier juste pour deviner les types, avant tout calcul. Un schéma déclaré ne coûte rien.

Q4 Une colonne texte contient « N/A » ; on la convertit en entier puis on calcule la moyenne. Que se passe-t-il en Spark 3.5 ?

Les « N/A » deviennent `NULL` sans erreur, et la moyenne les ignore silencieusement. Il faut les compter, ou activer le mode ANSI.

Q5 Dans un plan d'exécution, que signale un `Exchange` ?

Un shuffle : une redistribution des données entre exécuteurs, et une frontière de stage. C'est le premier endroit où chercher le temps perdu.

ANNEXES DU MODULE 3 AVANCÉ

De la table propre à la production

Données illisibles, traitement batch, filtres sur une liste, accès SQL sans Python, puis jointures, plans, UDF et formats de table.

Une ligne illisible : la garder, l'écarter, ou tout arrêter

```
trajet_id,duree_min,type_usager  
T01,5.0,abonne  
T02,N/A,occasionnel  
T03,12.4,abonne
```

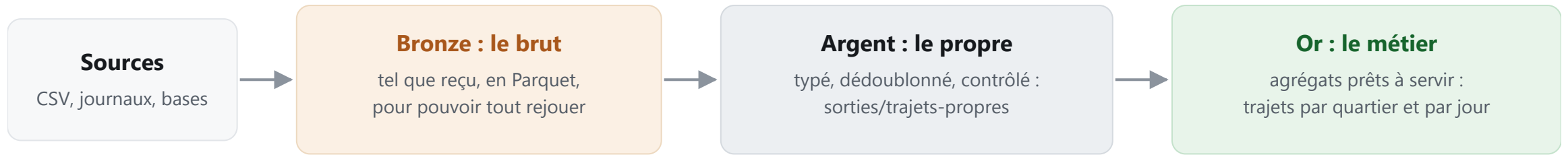
```
ddl3 = "trajet_id STRING, duree_min DOUBLE, type_usager STRING"  
(spark.read.option("header", True)  
    .option("mode", "FAILFAST")  
    .schema(ddl3)  
    .csv("brut.csv"))
```

Le mode ne joue que si le schéma est **déclaré** : sans schéma, tout est texte, et « N/A » est une chaîne valide.

MODE	CE QU'IL FAIT DE T02
PERMISSIVE	Garde la ligne, <code>duree_min</code> vaut <code>NULL</code> . Avec une colonne <code>_corrupt_record</code> déclarée, la ligne brute y est conservée : <code>T02,N/A,occasionnel</code>
DROPMALFORMED	La retire, sans rien dire : 2 lignes lues sur 3
FAILFAST	Arrête le job à la première ligne invalide : <code>MALFORMED_RECORD_IN_PARSING</code> , avec la ligne fautive dans le message

Un traitement qui tourne la nuit, sans personne pour regarder : **FAILFAST**, ou **PERMISSIVE** avec la colonne des rejets, comptée et mise de côté. Jamais un abandon silencieux.

Un traitement batch, en trois couches



```
velocite_batch/  
lecture.py      schémas déclarés, un lecteur par source  
transformations.py  fonctions DataFrame → DataFrame  
requetes.sql   le SQL, versionné à part  
job.py         session, enchaînement, écriture  
tests/  
test_transformations.py
```

- Une transformation est une **fonction** qui prend un DataFrame et en rend un : elle se teste sur trois lignes, sans cluster.
- Le job ne fait qu'**enchaîner** : lire, transformer, écrire, puis `spark.stop()`.
- Les contrôles vivent **entre bronze et argent** : schéma déclaré, rejets comptés.

isin, ou une jointure ?

Une liste courte, écrite dans le code

```
gares = ["S002", "S010", "S018"]  
tr.filter(F.col("station_depart").isin(gares))
```

```
Filter station_depart IN (S002,S010,S018)  
+- FileScan csv  
   PushedFilters: [In(station_depart,  
                     [S002,S010,S018])]
```

15 068 trajets. Le filtre descend jusqu'à la lecture : **ni jointure, ni shuffle**.

Quelques valeurs fixes : **isin**. Une liste qui change, qui vient d'un fichier, ou qui compte des milliers de valeurs : une jointure **left_semi**, que Spark diffuse seul tant qu'elle est petite.

Une liste qui vient d'une table

```
ref = (stations  
      .filter(F.col("quartier") == "Gare")  
      .select(F.col("station_id")  
              .alias("station_depart")))  
tr.join(ref, "station_depart", "left_semi")
```

```
BroadcastHashJoin [station_depart],  
                  [station_depart], LeftSemi
```

40 133 trajets, depuis les 8 stations du quartier. La petite table est diffusée : pas de shuffle non plus.

Spark SQL sans écrire de Python

La ligne de commande `spark-sql`

```
spark-sql --master "local[2]" -f requete.sql
```

```
abonne      189723
occasionnel  109528
Time taken: 77.148 seconds, Fetched 2 row(s)
```

Installée par `pip install pyspark`, à côté de `spark-submit`.

Pour vérifier une table, tester une requête, valider un traitement.

Le serveur JDBC / ODBC

```
spark-submit --class
org.apache.spark.sql.hive.thriftserver.HiveThriftServer2
spark-internal
```

```
beeline -u jdbc:hive2://localhost:10000
```

```
Connected to: Spark SQL (version 3.5.4)
```

```
| occasionnel | 109528 |
| abonne      | 189723 |
```

Un outil de BI, un client SQL ou `beeline` s'y connectent comme à une base, et Spark exécute les requêtes.

Spark n'est pas une base transactionnelle : chaque requête est un job. Plusieurs utilisateurs simultanés se partagent le même cluster, et les droits d'accès sont à organiser à part.

AQE et stratégies de jointure

STRATÉGIE	QUAND
Broadcast hash	Un côté est petit : il est copié sur chaque exécutateur, pas de shuffle
Sort merge	Deux grosses tables : les deux côtés sont shufflés et triés par clé
Shuffle hash	Cas intermédiaire, choisi par l'optimiseur

```
tr.join(F.broadcast(stations),  
        tr.station_depart  
        == stations.station_id)
```

Adaptive Query Execution

Actif par défaut depuis Spark 3.2 : le plan est **révisé pendant l'exécution**, avec les tailles réelles. D'où `AdaptiveSparkPlan isFinalPlan=false` en tête de chaque plan qui contient un shuffle.

- fusionne les petites partitions après un shuffle
- bascule vers un broadcast si un côté s'avère petit
- découpe les partitions déséquilibrées

Seuil de broadcast automatique :

`spark.sql.autoBroadcastJoinThreshold`, **10 Mo** par défaut.

explain(mode="formatted")

Le même plan que tout à l'heure, en deux parties : un **arbre numéroté**, puis le **détail de chaque nœud**.

simple	Le plan physique (défaut)
extended	Les plans logiques, avant et après optimisation
formatted	Arbre numéroté et détails
cost	Les tailles estimées
codegen	Le code généré

La génération de code fusionne les nœuds d'un même stage en une seule fonction, marquée ***(1)**. Elle ne s'affiche pas ici : avec AQE, le plan montré avant exécution n'est pas le plan final.

```
AdaptiveSparkPlan (12)
+- HashAggregate (11)
  +- Exchange (10)
    +- HashAggregate (9)
      +- Project (8)
        +- BroadcastHashJoin Inner BuildRight (7)
          :- Project (3)
            : +- Filter (2)
              : +- Scan csv (1)
            +- BroadcastExchange (6)
              +- Filter (5)
                +- Scan csv (4)

(1) Scan csv
Output [2]: [station_depart, duree_min]
PushedFilters: [IsNotNull(duree_min),
  GreaterThan(duree_min,0.0),
  IsNotNull(station_depart)]
ReadSchema: struct<station_depart:string,
  duree_min:double>

(10) Exchange
Arguments: hashpartitioning(quartier, 200)
```

Le coût d'une UDF, mesuré

```
base = spark.read.parquet(
    "sorties/trajets-propres").cache()

for nom, expr in [
    ("natif", F.col("distance_km")
      / (F.col("duree_min") / 60)),
    ("udf python", vitesse("distance_km",
                          "duree_min")),
    ("pandas udf", vitesse_v("distance_km",
                          "duree_min"))]:
    base.select(F.avg(expr)).collect()
```

VERSION	TEMPS
Expression native	0,9 s
UDF Python	10,6 s
UDF pandas	17,3 s

Sur 300 000 lignes, Spark 3.5.4, `local[2]`, meilleure de trois exécutions. Poste chargé : les valeurs absolues bougent, **l'ordre ne bouge pas.**

Delta Lake et Apache Iceberg

Un dossier Parquet est un tas de fichiers. Si deux traitements y écrivent en même temps, ou si une écriture plante à mi-chemin, **rien ne protège** le lecteur.

Un **format de table** ajoute au-dessus des fichiers Parquet un **journal de transactions** :

- écritures atomiques, lectures cohérentes
- `UPDATE`, `DELETE`, `MERGE`
- retour à une version passée
- évolution du schéma

	DELTA LAKE	ICEBERG
Origine	Databricks	Netflix, puis Apache
Données	Parquet	Parquet, ORC, Avro
Avec Spark	Un paquet à ajouter (<code>--packages</code>), et une extension de session à configurer	

Ils ne remplacent pas Parquet : ils **l'organisent**. Tout ce qu'on a dit de la lecture colonnaire reste vrai.

MODULE 4

Machine learning avec MLlib

Apprendre à deviner le type d'utilisateur à partir d'un trajet, sur la table que vous venez de nettoyer.

JOUR 2 · MODULE 4

Machine learning avec MLlib

Apprendre une règle à partir d'exemples, et le faire sur des données qui ne tiennent pas sur une machine.

DURÉE

≈ 2 h 10 d'exposé + TP 1 h 30

À LA FIN

Entraîner, évaluer et sauvegarder un pipeline MLlib

TP

Abonné ou occasionnel ?

Ce que couvre le module 4

1 Introduction au machine learning

Exemples, variables, cible, entraînement, test.

2 Les classes d'algorithmes

Classifier, prédire un nombre, regrouper, recommander.

3 Présentation de MLlib

Transformateurs, estimateurs, pipelines.

4 Implémenter et évaluer

Classification, régression, clustering, sauvegarde.

5 TP : abonné ou occasionnel ?

Un pipeline complet, de la donnée au modèle.

La question de l'après-midi

Un trajet vient d'avoir lieu : 22 minutes, un samedi à 15 h, départ du quartier Parc. **Est-ce un abonné ou un occasionnel ?**

Personne n'a écrit la règle. On va la faire apprendre à partir de 300 000 trajets dont on connaît la réponse.

SÉQUENCE 1

Introduction au machine learning

Écrire une règle, ou la faire déduire d'exemples dont on connaît la réponse.

Écrire la règle, ou l'apprendre

Programmation classique

```
def type_usager(trajet):  
    if trajet.duree_min > 15:  
        return "occasionnel"  
    return "abonne"
```

Quelqu'un a choisi le seuil de 15 minutes. Il ne regarde qu'un indice, et personne ne sait combien de fois il se trompe.

Apprentissage automatique

```
modele = algorithme.fit(trajets_connus)  
modele.transform(nouveaux_trajets)
```

L'algorithme reçoit des **exemples dont on connaît la réponse**, et cherche lui-même la combinaison d'indices qui la retrouve le mieux. Son taux d'erreur se **mesure**.

Le machine learning n'invente rien : il **résume des exemples**. S'ils sont faux, rares ou biaisés, la règle apprise l'est aussi.

Cinq mots, sur un trajet

Exemple	Une ligne : un trajet passé. On en a environ 300 000.
Variables (features)	Ce qu'on sait du trajet au moment de décider : <code>heure</code> , <code>jour_semaine</code> , <code>duree_min</code> , <code>distance_km</code> , <code>electrique</code> , <code>quartier</code> .
Cible (label)	Ce qu'on veut deviner : <code>type_usager</code> , codé 1 pour occasionnel, 0 pour abonné.
Modèle	La règle apprise. Un objet qu'on sauvegarde, recharge, et applique à de nouveaux trajets.
Prédiction	La réponse du modèle pour un trajet, avec une probabilité : « occasionnel à 81 % ».

Une variable n'est légitime que si on la **connaît au moment de prédire**. Nous y reviendrons : c'est le piège le plus coûteux du domaine.

Entraîner sur une partie, tester sur l'autre

```
train, test = donnees.randomSplit(  
    [0.8, 0.2], seed=42)
```

Le modèle apprend sur 80 % des trajets. On le juge sur les 20 % **qu'il n'a jamais vus**.

Au TP, un découpage identique sur tous les postes :

`pmod(hash(trajet_id), 5) == 0` désigne le test.

Le juger sur ses propres exemples, c'est faire corriger une copie par l'élève qui a eu le corrigé.

Le surapprentissage

Un modèle trop souple apprend **le bruit** des exemples, pas la tendance : excellent sur l'entraînement, médiocre sur le test. L'écart entre les deux scores est le symptôme.

La référence naïve

Répondre « abonné » à chaque trajet donne raison

63,7 % du temps, sans rien apprendre.

Un modèle à 70 % d'exactitude n'est donc pas bon : il fait à peine mieux que rien.

Toujours la calculer d'abord

C'est le score à battre. Sans lui, aucun chiffre ne veut rien dire.

Supervisé ou non supervisé

Supervisé : on connaît la réponse

Chaque exemple porte sa cible. Le modèle apprend à la retrouver, et on mesure combien de fois il y parvient.

Vélo'Cité : deviner le type d'usager, prédire la durée d'un trajet.

Non supervisé : on cherche une structure

Pas de cible. L'algorithme regroupe ce qui se ressemble, et c'est à l'humain de dire ce que les groupes veulent dire.

Vélo'Cité : trouver des familles de stations au comportement proche.

Le programme de cet après-midi est surtout supervisé : c'est le cas le plus fréquent, et le seul qu'on sache **évaluer sans discussion**.

SÉQUENCE 2

Les différentes classes d'algorithmes

On choisit un algorithme d'après la **forme de la réponse** attendue, pas d'après sa réputation.

Cinq formes de réponse, cinq classes

CLASSE	LA RÉPONSE EST...	SUR VÉLO'CITÉ	DANS MLLIB
Classification	Une catégorie	Abonné ou occasionnel ?	<code>LogisticRegression,</code> <code>RandomForestClassifier</code>
Régression	Un nombre	Combien de minutes durera ce trajet ?	<code>LinearRegression,</code> <code>GBRegressor</code>
Clustering	Un groupe, sans cible	Quelles stations se ressemblent ?	<code>KMeans,</code> <code>BisectingKMeans</code>
Recommandation	Un classement d'objets	Quelle station proposer à cet abonné ?	<code>ALS</code>
Réduction de dimension	Moins de colonnes	Résumer 60 stations en 3 axes	<code>PCA</code>

La première question devant un problème n'est pas « quel algorithme ? » mais « **quelle est la forme de la réponse ?** ». Elle élimine d'emblée les quatre cinquièmes du catalogue.

Trois modèles pour une classification

Régression logistique

Une somme pondérée des variables, transformée en probabilité. Rapide, lisible : chaque poids dit dans quel sens pousse une variable.
Ne voit pas qu'une variable peut pousser dans un sens, puis dans l'autre.

Arbre de décision

Une suite de questions : « durée > 14 min ? », puis « week-end ? ». Se lit comme une règle métier.
Seul, il surprend vite.

Forêts et boosting

Des dizaines d'arbres qui votent (`RandomForestClassifier`) ou se corrigent l'un l'autre (`GBClassifier`).
Souvent le meilleur score sur des tables.
Moins lisibles, plus longs à entraîner.

La méthode honnête : commencer par le plus simple, **mesurer**, et ne passer au plus complexe que si l'écart de score le justifie.

SÉQUENCE 3

Présentation de MLlib

Deux verbes, `fit` et `transform`, et un objet qui les enchaîne : le pipeline.

`pyspark.ml` et `pyspark.mllib`

	PYSPARK.ML	PYSPARK.MLLIB
Travaille sur	Des DataFrames	Des RDD
Statut	L'API principale, celle qui évolue	En maintenance : corrections seulement, plus de nouveautés
Organisation	Transformateurs, estimateurs, pipelines	Des fonctions d'entraînement isolées
Quand l'utiliser	Tout projet nouveau	Du code existant, et quelques algorithmes absents de <code>ml</code> , comme <code>StreamingKMeans</code>

« MLlib » désigne les deux dans la documentation. Dans ce module, et dans votre code neuf, c'est `pyspark.ml`.

Transformateur et estimateur

Transformateur : `transform(df)`

Prend un DataFrame, rend un DataFrame avec **une colonne de plus**. N'apprend rien.

`VectorAssembler`, un modèle déjà entraîné.

Estimateur : `fit(df)`

Regarde les données, et rend... **un transformateur** : le modèle.

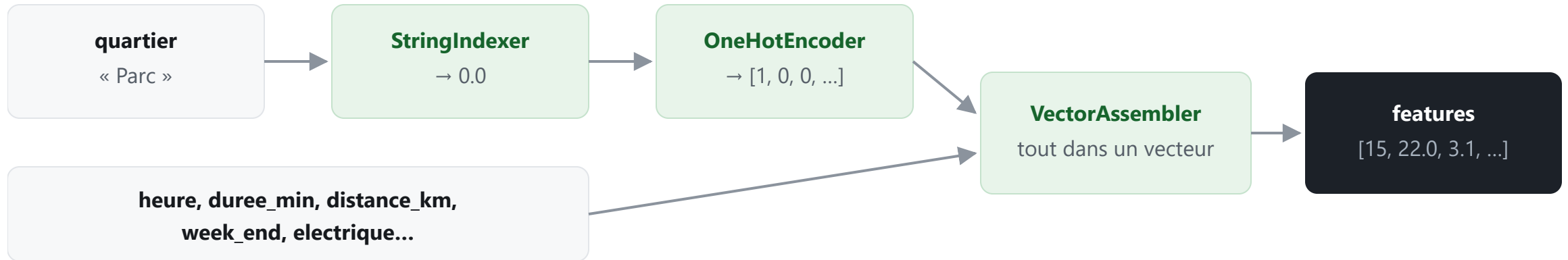
`StringIndexer`, `LogisticRegression`.

```
indexeur = StringIndexer(
    inputCol="quartier",
    outputCol="quartier_idx")
modele = indexeur.fit(df)      # apprend
modele.transform(df).show()   # applique
```

quartier	quartier_idx
Parc	0.0
Gare	1.0
Parc	0.0

`StringIndexer` apprend la liste des quartiers, du plus fréquent au plus rare : sur ces trois lignes, Parc passe en premier ; sur les vrais trajets, il sera le dernier.

Un algorithme ne lit qu'une colonne : un vecteur



StringIndexer

Un texte devient un numéro. Estimateur : il apprend la liste des valeurs.

OneHotEncoder

Un numéro devient une case cochée parmi n . Sans lui, « Gare » (1) serait « plus grand » que « Parc » (0).

VectorAssembler

Toutes les variables rassemblées en une seule colonne vecteur, **features**.

StandardScaler

Met les variables à la même échelle : indispensable aux méthodes à distance (KMeans), inutile aux arbres. Les régressions de MLlib standardisent déjà seules (**standardization=True**).

Le pipeline : une recette qu'on entraîne d'un coup

```
from pyspark.ml import Pipeline

pipeline = Pipeline(stages=[
    indexeur,      # estimateur
    encodeur,      # estimateur
    assembleur,    # transformateur
    normaliseur,   # estimateur
    reglog,        # estimateur
])

modele = pipeline.fit(train)      # PipelineModel
predictions = modele.transform(test)
```

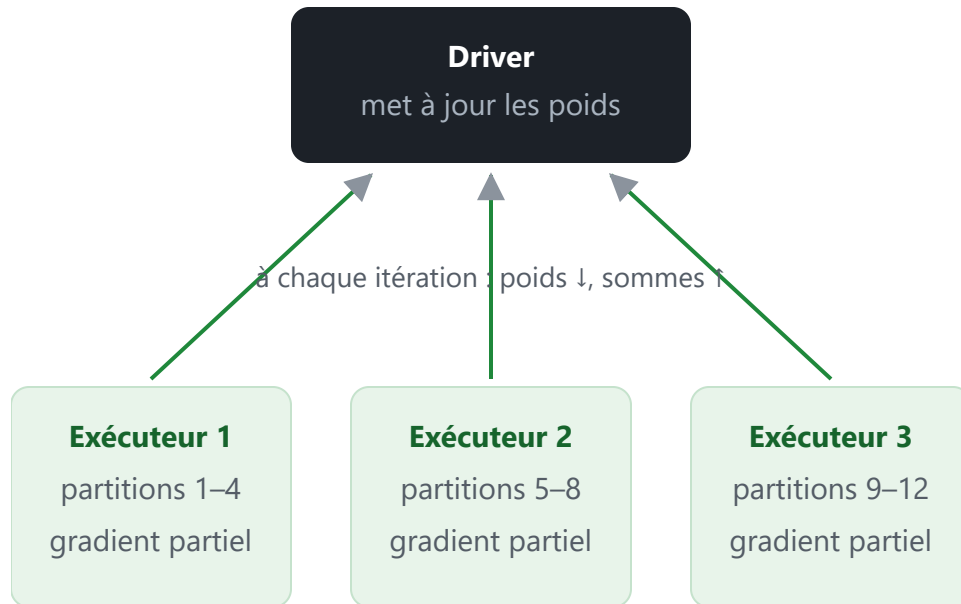
Ce que fait `fit`

Il parcourt les étapes dans l'ordre : chaque estimateur est entraîné sur la sortie de l'étape précédente, puis remplacé par son modèle.

Pourquoi c'est indispensable

Le test, et demain le flux, subissent **exactement** les mêmes préparations, avec les mêmes paramètres appris. Un seul objet à sauvegarder.

Ce qui est distribué, et ce qui ne l'est pas



Distribué

- La **préparation** des colonnes : ce sont des transformations de DataFrame.
- Le **calcul sur les données** : chaque exécuteur traite ses partitions, le driver additionne.

Pas distribué

- Le **modèle** lui-même : il doit tenir dans la mémoire du driver.
- Les **itérations** : elles s'enchaînent une par une. D'où le `cache()` sur les données d'entraînement.

SÉQUENCE 4

Implémenter et évaluer

Classification, mesure du résultat, et deux autres classes d'algorithmes sur les mêmes données.

Au retrait du vélo : une régression logistique

```
AU_RETRAIT = ["heure", "jour_semaine", "week_end",
              "electrique", "quartier_vec"]

indexeur = StringIndexer(inputCol="quartier",
                          outputCol="quartier_idx", handleInvalid="keep")
encodeur = OneHotEncoder(inputCols=["quartier_idx"],
                          outputCols=["quartier_vec"])
assembleur = VectorAssembler(inputCols=AU_RETRAIT,
                              outputCol="brutes")
normaliseur = StandardScaler(inputCol="brutes",
                              outputCol="features")
reglog = LogisticRegression(labelCol="label",
                             featuresCol="features")

modele_lr = Pipeline(stages=[indexeur, encodeur,
                              assembleur, normaliseur, reglog]).fit(train)
predictions_lr = modele_lr.transform(test)
```

Trois colonnes apparaissent dans `predictions_lr` :

<code>rawPrediction</code>	Le score brut
<code>probability</code>	[P(abonné), P(occasionnel)]
<code>prediction</code>	0.0 ou 1.0, au seuil de 0,5

type_usager	heure	quartier	probability	prediction
occasionnel	8	Université	[0.77, 0.23]	0.0
occasionnel	14	Parc	[0.46, 0.54]	1.0
abonne	17	Parc	[0.62, 0.38]	0.0

Exactitude, AUC, et le piège du déséquilibre

```
auc = BinaryClassificationEvaluator(  
    labelCol="label",  
    metricName="areaUnderROC")  
exactitude = MulticlassClassificationEvaluator(  
    labelCol="label", metricName="accuracy")
```

MODÈLE	EXACTITUDE	AUC
Toujours « abonné »	0,637	0,500
Régression logistique	0,677	0,706

L'exactitude

La part de bonnes réponses. Parlante, mais trompeuse quand une classe domine : ici, 63 % des trajets sont des abonnés.

L'AUC

La probabilité qu'un occasionnel tiré au hasard reçoive un score plus haut qu'un abonné tiré au hasard. 0,5 : le hasard. 1 : parfait.

Insensible au déséquilibre, et au choix du seuil.

La matrice de confusion : quelles erreurs ?

```
(predictions_lr
  .groupBy("label")
  .pivot("prediction", [0.0, 1.0])
  .count()
  .orderBy("label")
  .show())
```

```
+-----+-----+-----+
|label|  0.0|  1.0|
+-----+-----+-----+
|  0.0|32178|6218|
|  1.0|13284|8610|
+-----+-----+-----+
```

	PRÉDIT ABONNÉ	PRÉDIT OCCASIONNEL
Vrai abonné	32 178	6 218 fausse alerte
Vrai occasionnel	13 284 occasionnel manqué	8 610

Le modèle manque **61 %** des occasionnels. Si l'objectif est de leur proposer un abonnement, c'est cette case qu'il faut regarder.

Pondérer les classes, plutôt que dupliquer des lignes

```
part_occ = (train.where("label = 1").count()  
            / train.count())          # 0,367
```

```
train = train.withColumn("poids",  
    F.when(F.col("label") == 1, 1 - part_occ)  
    .otherwise(part_occ))
```

```
LogisticRegression(labelCol="label",  
    featuresCol="features", weightCol="poids")
```

OCCASIONNELS	RAPPEL	PRÉCISION	AUC
Sans pondération	0,393	0,581	0,706
Avec pondération	0,637	0,504	0,706

Trois leviers, un seul réglage

Baisser le **seuil**, pondérer les classes, ou rééchantillonner : les trois déplacent presque le même curseur entre rappel et précision.

L'AUC ne bouge pas

Elle juge le **classement** des trajets, pas le point de bascule. Un modèle qui ne s'améliore qu'en rappel n'a rien appris de plus : il décide autrement.

Au retrait : la forêt aléatoire, et ce qu'elle regarde

```
foret = RandomForestClassifier(  
    labelCol="label", featuresCol="brutes",  
    numTrees=30, maxDepth=8, seed=42)  
modele_rf = Pipeline(stages=[indexeur,  
    encodeur, assembleur, foret]).fit(train)
```

MODÈLE	EXACTITUDE	AUC
Régression logistique	0,677	0,706
Forêt aléatoire	0,742	0,809

Importance des variables

heure	0.578
electrique	0.166
week_end	0.104
quartier_vec_Parc	0.045
jour_semaine	0.043

Pas de normalisation : un arbre compare des seuils, l'échelle ne change rien. On branche donc la forêt sur **brutes**.

Au retrait ou au dépôt : l'information bat l'algorithme

```
AU_DEPOT = AU_RETRAIT + ["duree_min",  
                          "distance_km"]
```

AUC	AU RETRAIT	AU DÉPÔT
Régression logistique	0,706	0,992
Forêt aléatoire	0,809	0,985

Exactitude au dépôt : 0,966 pour la logistique, 0,943 pour la forêt.

Deux leçons

Deux colonnes de plus font gagner bien plus que le changement d'algorithme. **Chercher d'abord la bonne information**, ensuite le bon modèle.

Et avec la durée, le modèle simple passe devant : elle pousse toujours dans le même sens, exactement ce que la logistique sait représenter.

Mais ce n'est pas le même usage

Au dépôt, l'offre ne peut plus s'afficher sur la borne de départ. Le moment de prédire se choisit **d'après l'usage**, pas d'après le score.

Régression et clustering, sur les mêmes trajets

Prédire la durée : `LinearRegression`

```
Pipeline(stages=[  
    VectorAssembler(outputCol="features",  
        inputCols=["distance_km", "electrique", "label"]),  
    LinearRegression(labelCol="duree_min"),  
]).fit(train)
```

```
coefficients [4.22, -2.23, 15.18]  
intercept    -0.81  
RMSE 7.67 min      R² 0.51
```

Chaque kilomètre ajoute 4,2 min ; être occasionnel en ajoute 15,2.

Regrouper les stations : `KMeans`

Trois variables par station : part d'occasionnels, durée moyenne, part de départs entre 7 h et 9 h.

```
KMeans(k=3, featuresCol="features").fit(par_station)
```

groupe	stations	part_occ	duree_moy
0	21	0.48	14.5
1	32	0.30	11.5
2	7	0.31	15.0

Personne n'a donné de cible : les groupes sont à **interpréter**.

Sans `StandardScaler`, la durée (≈ 12 min) écrase les parts (0 à 1) : les groupes 1 et 2 ne diffèrent que par elle.

Sauvegarder, recharger, appliquer ailleurs

```
# depot_lr : le modèle « au dépôt » ajusté au TP
depot_lr.write().overwrite().save(
    "sorties/modele-type-usager")

from pyspark.ml import PipelineModel
modele = PipelineModel.load(
    "sorties/modele-type-usager")
modele.transform(nouveaux_trajets)
```

Le dossier contient **tout le pipeline** : la liste des quartiers apprise par l'indexeur, l'encodeur, l'assembleur, le normaliseur et les poids de la régression logistique « au dépôt ».

Le contrat d'entrée

Pour prédire, le DataFrame doit porter les colonnes d'entrée : `heure`, `jour_semaine`, `week_end`, `duree_min`, `distance_km`, `electrique`, `quartier`. Rien de plus, rien de moins. Pas de `type_usager`.

Demain matin

Le TP 5 recharge ce dossier et l'applique à un flux de trajets, en temps réel.

La fuite de cible : un modèle trop beau pour être vrai

Une variable **calculée à partir de la réponse**, ou connue seulement après elle, s'est glissée dans les variables.

PRÉDIRE...	AVEC...	POURQUOI C'EST UNE FUITE
Le départ d'un client	La date de résiliation	Elle n'existe que s'il est parti
Une fraude	Le statut « remboursé »	Posé après l'enquête
Une panne	Le nombre d'interventions	Compté après la panne

Le symptôme

Une AUC proche de 1, obtenue sans effort. En production, le modèle s'effondre : la variable n'est pas encore connue au moment de prédire.

Le réflexe

Pour chaque variable : **la connaît-on au moment où l'on prédit ?** Et si un score est parfait, chercher d'abord la fuite.

Abonné ou occasionnel ?

≈ 1 h 30

Seul ou en binôme

Restitution 15 min

Échauffement : [exercices/serie4-ml/](#)

[tp4-ml/enonce.md](#) · notebook [tp4-ml/depart.ipynb](#)

- 1 Charger les trajets propres, joindre le quartier**
Ceux du TP 3, ou reconstruits si le TP 3 n'a pas abouti.
- 2 Découper, calculer la référence naïve**
Le score à battre, avant tout modèle.
- 3 Au retrait : régression logistique, et l'évaluer**
AUC, exactitude, matrice de confusion.
- 4 Au retrait : forêt aléatoire et importances**
Mieux ? Pourquoi ?
- 5 Au dépôt : ajouter durée et distance, sauvegarder le meilleur**
Dans [sorties/modele-type-usager](#), pour le TP 5.

Ce qui est attendu

Un modèle qui bat nettement la référence naïve, **et la phrase qui dit de combien.**

Bonus

Il reste une colonne numérique dans les trajets.
Ajoutez-la au modèle « au retrait ». Que se passe-t-il ?
Faut-il la garder ?

Cinq questions avant de fermer le jour 2

Q1 Un modèle de détection de pannes affiche 97 % d'exactitude. Que demandez-vous avant de vous en réjouir ?

La part des pannes dans les données : si elle est de 3 %, répondre toujours « pas de panne » fait aussi bien. Puis l'AUC, et la matrice de confusion.

Q2 Pourquoi `StringIndexer` a-t-il une méthode `fit`, alors qu'il n'apprend rien au sens statistique ?

Il doit voir les données pour connaître la liste des valeurs et leur ordre. C'est un estimateur : `fit` rend le transformateur qui applique cette liste.

Q3 Que gagne-t-on à mettre la préparation des colonnes dans le pipeline plutôt qu'avant ?

Les données de test et de production subissent exactement la même préparation, avec les mêmes paramètres appris, et tout se sauvegarde en un seul objet.

Q4 Qu'est-ce que Spark distribue quand il entraîne une régression logistique ?

Le calcul sur les données : chaque exécuteur traite ses partitions et renvoie une somme. Le modèle et l'enchaînement des itérations restent sur le driver.

Q5 Votre modèle atteint une AUC de 0,999 du premier coup. Premier réflexe ?

Chercher une fuite de cible : une variable calculée à partir de la réponse, ou connue seulement après elle.

ANNEXES DU MODULE 4 AVANCÉ

Aller plus loin avec MLlib

Régler les hyperparamètres sans tricher, se passer de modèle, préparer d'autres variables, suivre ses expériences, et savoir quand ne pas utiliser Spark.

Validation croisée et grille de paramètres

```
from pyspark.ml.tuning import (
    CrossValidator, ParamGridBuilder)

grille = (ParamGridBuilder()
    .addGrid(foret.maxDepth, [5, 8, 12])
    .addGrid(foret.numTrees, [20, 50])
    .build())           # 6 combinaisons

pipeline_rf = Pipeline(stages=[indexeur, encodeur,
    assembleur, foret])    # non ajusté

cv = CrossValidator(
    estimator=pipeline_rf,
    estimatorParamMaps=grille,
    evaluator=auc, numFolds=3,
    parallelism=2, seed=42)
meilleur = cv.fit(train).bestModel
```

Pourquoi pas sur le test ?

Choisir les paramètres d'après le score de test, c'est encore faire corriger la copie par l'élève. La validation croisée découpe **l'entraînement** en 3 plis, et garde le test intact pour la fin.

Le coût

6 combinaisons × 3 plis = 18 entraînements, plus un final. `TrainValidationSplit` n'en fait qu'un par combinaison.

Repérer l'anormal avec une fonction de fenêtre

```
groupe = Window.partitionBy("station_depart",
                           "type_usager")

z = (trajets
     .withColumn("moy", F.avg("duree_min").over(groupe))
     .withColumn("ecart", F.stddev("duree_min").over(groupe))
     .withColumn("z", (F.col("duree_min") - F.col("moy"))
                   / F.col("ecart")))

anomalies = z.where(F.abs("z") > 3)
```

anomalies |z| > 3 : 2 616 sur 299 251, extrait

trajet	station	type	durée	moy	z
T0204141	S039	occasionnel	139,7	19,5	8,9
T0104083	S026	occasionnel	136,5	19,5	8,6

Comparer à qui de droit

Le partitionnement de la fenêtre est tout le sujet : chaque trajet est comparé à **son groupe**, ici les occasionnels partant de S039, dont les trajets durent 19,5 minutes en moyenne. Un seuil unique pour tout le réseau jugerait de la même façon une station de parc et une station de gare.

Quand cela suffit

Aucune cible, aucun entraînement, aucun modèle à réentraîner. Sur des mesures à peu près symétriques, c'est souvent la bonne réponse — et la fonction de fenêtre du module 3 y suffit.

Trois outils pour les variables que le TP n'a pas rencontrées

Découper un nombre en tranches

```
tranches = Bucketizer(
    splits=[0, 7, 10, 16, 20, 24],
    inputCol="heure", outputCol="tranche")
```

tranche	plage	trajets	part	occ.
0	0 h – 7 h	3 862		0,209
1	7 h – 10 h	46 079		0,056
2	10 h – 16 h	86 142		0,560
3	16 h – 20 h	91 065		0,351
4	20 h – 24 h	11 813		0,341

La part d'occasionnels monte, chute, remonte : une variable dont l'effet **n'est pas monotone**. Découpée en tranches puis encodée, elle devient lisible par un modèle linéaire : l'AUC de la logistique « au retrait » passe de **0,706 à 0,770**.

Trop de catégories

```
FeatureHasher(inputCols=["station_depart"],
               outputCol="station_hachee",
               numFeatures=32)
```

S060 → (32,[16],[1.0])

60 stations passent encore ; 200 000 références produit font exploser le codage en cases. Le hachage les répartit dans un nombre fixe de cases, au prix de quelques collisions.

Du texte

Tokenizer découpe, **StopWordsRemover** retire les mots vides, **HashingTF** et **IDF** pondèrent. De quoi classer des avis ou des tickets.

Garder la trace de ses expériences

Au bout d'une semaine, la question n'est plus « quel est le meilleur score ? » mais « **avec quelles données, quels paramètres et quel code l'a-t-on obtenu ?** »

MLflow, projet open source, est l'outil le plus répandu pour y répondre : il enregistre paramètres, métriques et modèle à chaque exécution, et sait sauvegarder un `PipelineModel` Spark.

Ce qu'il faut consigner, outil ou pas

- La version des données (un chemin daté, une empreinte)
- Les variables et les hyperparamètres
- La règle du découpage : graine ou hachage
- Les scores, et la référence naïve
- La version de Spark

MLlib ou scikit-learn ?

SITUATION	CHOIX RAISONNABLE
Les données d'entraînement tiennent en mémoire sur une machine	scikit-learn : plus d'algorithmes, plus rapide à ce volume, plus simple à déboguer
Des centaines de millions de lignes, un modèle simple	MLlib : c'est son terrain
Préparation lourde en Spark, petit jeu final	Spark pour préparer et agréger, puis <code>toPandas()</code> et scikit-learn
Appliquer un modèle scikit-learn à des milliards de lignes	Une UDF pandas qui appelle <code>predict</code> , partition par partition
Réseaux de neurones profonds	Ni l'un ni l'autre : des bibliothèques dédiées

Nos 300 000 trajets tiennent dans un portable : en production, scikit-learn serait un choix défendable. On utilise MLlib pour apprendre ce qui

JOUR 3 • MODULE 5

Analyser en temps réel

Les mêmes trajets, mais qui arrivent un par un : statistiques au fil de l'eau, et le modèle d'aujourd'hui appliqué en direct.

JOUR 3 • MODULE 5

Analyser en temps réel avec Spark Streaming

Les mêmes calculs que depuis deux jours, sur des données qui n'ont pas de fin.

DURÉE	À LA FIN	TP
≈ 1 h 50 d'exposé + TP 1 h 20	Savoir écrire, fenêtrer et reprendre une requête de flux	Statistiques et prédictions en temps réel

Ce que couvre le module 5

1 L'architecture du streaming

Flux, micro-lots, heure de l'événement, garanties.

2 Les DStreams

La première API de Spark Streaming, et pourquoi on l'a quittée.

3 Structured Streaming

Sources, puits, modes de sortie, déclencheurs, reprise.

4 Fenêtres et retardataires

Agrégations dans le temps, watermark, jointures.

5 Machine learning en temps réel

Le modèle d'hier, appliqué au fil de l'eau.

Hier soir, vous aviez

Des trajets nettoyés, et un modèle qui devine si un trajet est celui d'un abonné ou d'un occasionnel.

Ce matin, les trajets arrivent **en direct**. La question devient : **combien, où, et qui, dans les cinq dernières minutes ?**

SÉQUENCE 1

L'architecture du streaming

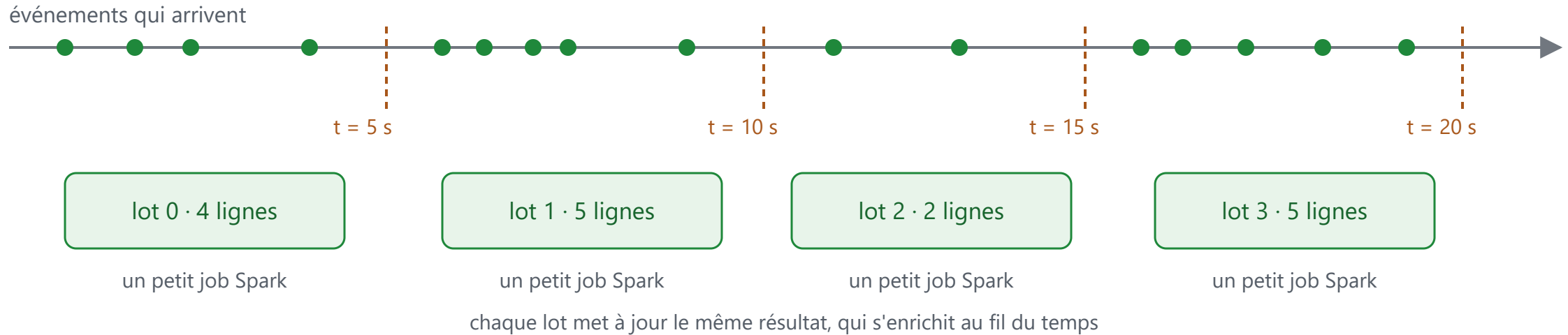
Ce qui change quand les données arrivent en continu, et ce qui ne change pas.

Traitement par lots, traitement de flux

	PAR LOTS (<i>BATCH</i>)	EN FLUX (<i>STREAMING</i>)
Les données	Bornées : un fichier, une table, qui a une fin	Non bornées : elles arrivent sans cesse
Le calcul	Lancé une fois, il se termine	Lancé une fois, il ne se termine pas
Le résultat	Définitif	Provisoire : il change à chaque arrivée
Le délai	Heures : le traitement de nuit	Secondes : ce qui se passe maintenant
Exemple Vélo'Cité	Trajets par station, mois de mars	Trajets par quartier, cinq dernières minutes

Un flux n'est pas un nouveau type de donnée : c'est une table **dont on ne connaît pas encore toutes les lignes**. Structured Streaming est construit sur cette idée.

Le micro-lot : un flux découpé en petits traitements



Latence : le temps entre l'arrivée d'un événement et sa prise en compte. Au mieux, la durée d'un lot.

Débit : le nombre d'événements traités par seconde. Des lots plus gros amortissent mieux le coût fixe d'un job.

L'heure de l'événement n'est pas l'heure d'arrivée

Un vélo est déposé à la station S017 à **10h03**. La borne perd le réseau, et l'événement n'arrive chez Spark qu'à **10h11**.

Heure de l'événement <i>event time</i>	10h03 : quand c'est arrivé. Écrite dans la donnée , colonne <code>fin</code> .
Heure de traitement <i>processing time</i>	10h11 : quand Spark le voit. Dépend du réseau, de la charge, des pannes.

Compter par heure de traitement

C'est facile, et c'est faux : le trajet de 10h03 compte pour la tranche 10h10 – 10h15. Une coupure réseau fabrique un faux pic.

Compter par heure de l'événement

C'est juste, mais cela pose une question : **combien de temps attendre les retardataires** avant de déclarer 10h00 – 10h05 terminée ?

Garanties de livraison et point de reprise

Au plus une fois	Un plantage peut perdre des événements
Au moins une fois	Rien n'est perdu, mais un événement peut être compté deux fois
Exactement une fois	Ni perte, ni doublon : ce que vise Structured Streaming

Les trois conditions de « exactement une fois »

1. Une **source rejouable** : fichiers, Kafka. On peut relire depuis une position.
2. Un **point de reprise** (*checkpoint*) : Spark y note ce qu'il a lu et l'état de ses calculs.
3. Un **puits idempotent** : réécrire un lot déjà écrit ne crée pas de doublon.

Le point de reprise est un **dossier**, un par requête. Deux requêtes qui partagent un dossier, ou un dossier supprimé par erreur, et la garantie tombe.

SÉQUENCE 2

Les Discretized Streams (DStreams)

La première API de flux de Spark : un flux vu comme une suite de RDD.

Un DStream, c'est une suite de RDD



- Un `StreamingContext` se crée sur le `SparkContext`, avec un **intervalle de lot** fixe.
- Chaque intervalle produit **un RDD**. Une opération sur le DStream s'applique à chacun.
- On retrouve donc `map`, `filter`, `reduceByKey` : tout le module 2.
- Sources : `socketTextStream`, `textFileStream`, `queueStream` pour les tests.
- Fenêtres : `window`, `reduceByKeyAndWindow`.
- État entre lots : `updateStateByKey`.

Compter les événements par station, lot par lot

```
from pyspark.streaming import StreamingContext

ssc = StreamingContext(sc, 2) # un lot toutes les 2 s

lots = [sc.parallelize(["S017 RETRAIT", "S017 DEPOT",
                       "S023 RETRAIT"]),
        sc.parallelize(["S023 DEPOT", "S017 RETRAIT"])]

flux = ssc.queueStream(lots) # une file de RDD, pour tester

comptes = (flux.map(lambda l: (l.split()[0], 1))
           .reduceByKey(lambda a, b: a + b))

comptes.pprint()

ssc.start()
ssc.awaitTerminationOrTimeout(60)
ssc.stop(stopSparkContext=False)
```

Time: 2026-09-19 14:15:52

('S017', 2)
('S023', 1)

Time: 2026-09-19 14:15:54

('S017', 1)
('S023', 1)

Ce qu'on lit

Le compte est **par lot**, pas cumulé : le lot suivant repart de zéro. Cumuler demande `updateStateByKey` et un point de reprise.

Une API d'hier, qu'on sait lire

FutureWarning: DStream is deprecated as of Spark 3.4.0. Migrate to Structured Streaming.

BESOIN	DSTREAMS	STRUCTURED STREAMING
Données manipulées	RDD, sans schéma	DataFrames, SQL, optimiseur Catalyst
Fenêtres	Sur l'heure d'arrivée seulement	Sur l'heure de l'événement
Retardataires	Rien de prévu	Watermark
Exactement une fois	À construire soi-même	De bout en bout, avec les bons puits
Même code en lots et en flux	Non	Oui : <code>read</code> devient <code>readStream</code>

Au programme parce qu'on en croise encore dans le code existant. **Pour un nouveau projet : Structured Streaming**, et c'est ce que fait la suite du module.

SÉQUENCE 3

Structured Streaming

Un DataFrame qui ne finit jamais : les mêmes opérations qu'hier, sur une table qui grandit.

Une table à laquelle on ajoute des lignes

table d'entrée, sans fin

S017 · abonne · 10h01	lot 0
S023 · occasionnel · 10h02	
S017 · abonne · 10h04	lot 1 : ajouté
S040 · abonne · 10h04	
... les lignes du prochain lot	

groupBy + count

table de résultat

S017 · 1 → 2
S023 · 1
S040 · 1 (nouvelle)

Spark ne recalcule pas tout :
il garde un **état** (les comptes)
et n'y ajoute que le lot 1.

On écrit la requête **comme si la table était complète**. Spark la transforme en traitement incrémental : c'est lui qui décide ce qu'il faut garder d'un lot à l'autre.

Les sources et les puits

Sources · `spark.readStream`

Fichiers	Un dossier surveillé : CSV, JSON, Parquet. Rejouable. Celle du TP.
Kafka	La source de production. Rejouable. Paquet <code>spark-sql-kafka</code> à ajouter.
Socket	Du texte reçu sur un port réseau. Démonstrations seulement.
Rate	Génère N lignes par seconde. Pour tester un débit.

Puits · `df.writeStream`

Console	Affiche chaque lot. Pour mettre au point.
Memory	Une table en mémoire qu'on interroge en SQL. Celui du TP.
Fichiers	Parquet, JSON, CSV. Exactement une fois.
Kafka	Vers un autre flux.
foreachBatch	Chaque lot comme un DataFrame ordinaire : vers une base, une API...

Lire, agréger, écrire, démarrer

```
trajets = (spark.readStream
    .schema(SCHEMA)          # obligatoire
    .json("flux/entree"))

comptes = trajets.groupBy("station_depart").count()

q = (comptes.writeStream
    .outputMode("complete")
    .format("memory")
    .queryName("comptes_stations")
    .trigger(processingTime="5 seconds")
    .start())                # rend la main

spark.sql("""SELECT * FROM comptes_stations
    ORDER BY count DESC LIMIT 5""").show()
```

```
--- tour 3 : 1640 trajets vus
+-----+-----+
|station_depart|count|
+-----+-----+
|          S011|   39|
|          S009|   35|
|          S032|   35|
|          S027|   33|
|          S053|   32|
+-----+-----+
```

Trois nouveautés, pas plus

`readStream` au lieu de `read`, `writeStream` au lieu de `write`, et `start()`, qui lance la requête **en tâche de fond**.

Que faut-il écrire, à chaque lot ?

MODE	CE QUI EST ÉCRIT	QUAND L'UTILISER
<code>append</code>	Les lignes nouvelles et définitives seulement	Sans agrégation, ou avec fenêtres et watermark. Le seul mode du puits fichier
<code>update</code>	Les lignes modifiées par ce lot	Agrégations qu'on veut suivre en direct
<code>complete</code>	Tout le résultat, à chaque lot	Agrégations dont le résultat reste petit : 60 stations, 8 quartiers

Spark refuse les combinaisons impossibles **dès le** `start()` : `append` sur un comptage sans watermark, par exemple, car aucun compte n'est jamais définitif.

Quand lancer le lot suivant

<i>aucun</i> (par défaut)	Dès que le lot précédent est fini
<code>processingTime="5 seconds"</code>	Toutes les 5 secondes : latence prévisible
<code>availableNow=True</code>	Tout ce qui attend, puis s'arrête
<code>continuous="1 second"</code>	Expérimental, très restreint : à éviter

```
q = (df.writeStream.format("parquet")
    .option("checkpointLocation", "reprise/q1")
    .trigger(availableNow=True)
    .start("sorties/q1"))
q.awaitTermination() # rend la main une fois tout traité
```

`availableNow` : le flux en lots

Lancé toutes les heures par un ordonnanceur, il traite ce qui est arrivé depuis la dernière fois, puis s'arrête. Le point de reprise se souvient d'où il en était. Pas de cluster allumé en permanence, et un code prêt à passer en continu le jour où l'on en a besoin.

Arrêter, relancer, ne rien perdre

```
def lancer_ecriture():  
    return (predits.writeStream  
            .format("parquet")  
            .outputMode("append")  
            .option("path", "sorties/predictions-flux")  
            .option("checkpointLocation",  
                    "flux/points-de-reprise/predictions")  
            .start())
```

```
q = lancer_ecriture()  # ... puis q.stop()  
# l'émetteur continue pendant l'arrêt  
q = lancer_ecriture()  # même dossier de reprise
```

arrêt après le lot 1 : 800 prédictions écrites
reprise au lot 2
après relance : 2000 prédictions, 2000 trajets distincts

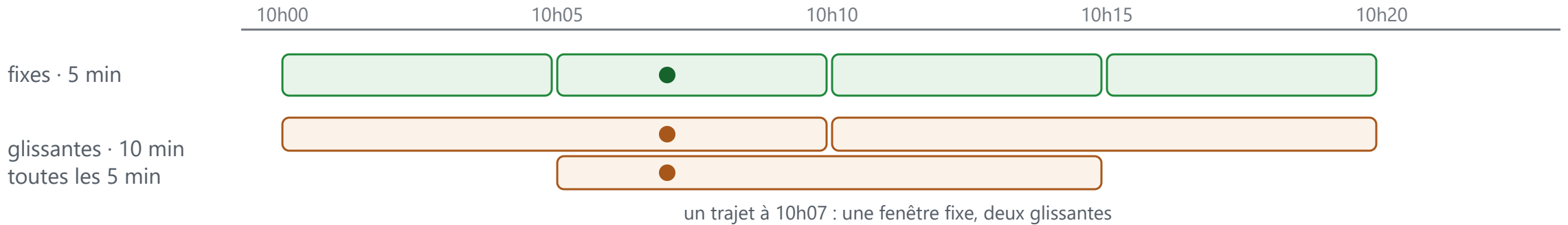
Le dossier de reprise contient les **positions lues** (`offsets/`), les **lots validés** (`commits/`), les **fichiers déjà lus** (`sources/`) ; et l'**état** des agrégations (`state/`) quand la requête agrège.

SÉQUENCE 4

Fenêtres et retardataires

Compter par tranche de temps, et décider jusqu'à quand attendre.

Fixes ou glissantes



```
par_quartier = (enrichis
    .groupBy(F.window("fin", "5 minutes"), "quartier")           # fixes
    .count())

glissant = (enrichis
    .groupBy(F.window("fin", "10 minutes", "5 minutes"), "quartier") # glissantes
    .count())
```

Jusqu'à quand attendre les retardataires

```
par_quartier = (enrichis
    .withWatermark("fin", "10 minutes")
    .groupBy(F.window("fin", "5 minutes"), "quartier")
    .count())
```

watermark = heure d'événement la plus récente vue – 10 minutes

Une fenêtre qui finit avant le watermark est **close** : son état est libéré, et une ligne qui tomberait dedans est **ignorée**.

Ce que le watermark règle

La mémoire : sans lui, Spark garde toutes les fenêtres depuis le démarrage, au cas où. L'état grandit sans fin.

Le mode `append` : une fenêtre close peut être écrite une fois pour toutes.

Ce qu'il coûte

Un retardataire au-delà du délai est perdu, **sans erreur**. Seul un compteur le signale.

Deux retardataires, deux destins

Fenêtres de 5 minutes, watermark de 10 minutes. Quatre lots déposés à la main, `processAllAvailable()` entre chaque.

LOT	ÉVÉNEMENTS REÇUS	WATERMARK APRÈS	FENÊTRE 10H00 – 10H05
1	10h01, 10h03, 10h07, 10h12	10h12 – 10 min = 10h02	2 trajets
2	10h04, en retard	10h02 (inchangé)	fenêtre ouverte (10h05 > 10h02) : accepté, 3 trajets
3	10h31 : le temps avance	10h31 – 10 min = 10h21	3 trajets, fenêtre close
4	10h02, en retard	10h21	fenêtre close (10h05 ≤ 10h21) : ignoré, toujours 3

```
q.lastProgress["stateOperators"][0]["numRowsDroppedByWatermark"] → 1
```

Joindre un flux et une table

```
stations = spark.read.csv("donnees/stations.csv",
                          header=True, inferSchema=True)

enrichis = (trajets
            .join(stations.select(
                F.col("station_id").alias("station_depart"),
                "quartier"),
                "station_depart")
            .withColumn("heure", F.hour("debut"))
            .withColumn("jour_semaine", F.dayofweek("debut"))
            .withColumn("week_end",
                F.dayofweek("debut").isin(1, 7)))
```

Flux × table

Rien de spécial : chaque lot est joint à la table. Le cas du TP.

Flux × flux

Possible, mais il faut un watermark **des deux côtés** et une borne de temps dans la condition, sinon l'état grandit sans fin.

La table statique est relue à chaque lot, sauf si on la met en cache (`stations.cache()`) : pour des stations qui changent rarement, c'est ce qu'on veut.

SÉQUENCE 5

Machine learning en temps réel

Entraîner en lots, prédire en flux.

Le modèle d'hier, appliqué au flux

```
from pyspark.ml import PipelineModel

modele = PipelineModel.load("sorties/modele-type-usager")

predits = (modele.transform(enrichis)  # un DataFrame de flux
           .withColumn("reel", (F.col("type_usager") == "occasionnel")
                        .cast("double")))

part = (predits
        .withWatermark("fin", "10 minutes")
        .groupBy(F.window("fin", "5 minutes"))
        .agg(F.count("*").alias("trajets"),
             F.avg("prediction").alias("part_predite"),
             F.avg("reel").alias("part_reelle")))
```

debut	trajets	part_predite	part_reelle
06:50	16	0.438	0.438
06:55	22	0.409	0.5
07:00	751	0.387	0.409

Aucune ligne nouvelle

`transform` sur un flux, c'est `transform` sur une table. Le pipeline entier s'applique : indexation du quartier, assemblage, normalisation, régression logistique.

La condition

Le flux doit porter **les mêmes colonnes** qu'à l'entraînement, calculées de la même façon. D'où la jointure et les trois colonnes de la slide précédente.

Entraîner sur le flux lui-même

Ce qui existe : l'API RDD

`pyspark.mllib` propose trois algorithmes qui se mettent à jour à chaque lot d'un **DStream** :

- `StreamingKMeans` : des groupes qui dérivent avec les données ;
- `StreamingLinearRegressionWithSGD` : une régression qui s'ajuste ;
- `StreamingLogisticRegressionWithSGD` : une classification, abonné ou occasionnel.

Toujours présents en Spark 3.5, mais liés aux DStreams, et hors de l'API `pyspark.ml` vue hier.

Ce qu'on fait en pratique

1

Prédire en flux

Le modèle chargé, appliqué à chaque lot. C'est ce qu'on vient de voir.

2

Stocker ce qui arrive

Le flux écrit aussi ses lignes en Parquet.

3

Réentraîner en lots

Chaque nuit ou chaque semaine, sur les données fraîches, puis remplacer le modèle.

Statistiques et prédictions en temps réel

≈ 1 h 20

Individuel

Corrigé projeté

Échauffement : `exercices/serie5-streaming/`

Dans un terminal : `python flux/emetteur.py --vider`. Puis `tp5-streaming/depart.ipynb`, énoncé dans `enonce.md`.

1 Lire le flux avec un schéma explicite

Source fichier sur `flux/entree/`.

2 Compter par station de départ

Mode `complete`, puits `memory`, trois lectures successives.

3 Fenêtres de 5 min par quartier

Jointure avec les stations, watermark de 10 minutes.

4 Prédire le type d'usager en direct

Le modèle du TP 4, et la part d'occasionnels par fenêtre.

5 Arrêter, relancer, vérifier

Point de reprise : aucun doublon après la relance.

Ce qui est attendu

Des chiffres qui **changent** entre deux lectures, et une part prédite proche de la part réelle.

Pas de modèle du TP 4 ?

Appelez `modele_de_secours()`, fournie dans le notebook : elle entraîne le même pipeline sur un échantillon, et le TP se fait quand même.

Toujours arrêter

Une requête oubliée tourne jusqu'à la fermeture du noyau. `spark.streams.active` les liste.

Cinq questions avant de fermer le module 5

Q1 Un vélo déposé à 10h03 arrive chez Spark à 10h11. Dans quelle fenêtre de 5 minutes doit-il compter, et que faut-il pour cela ?

10h00 – 10h05 : on fenêtrise sur l'heure de l'événement, la colonne `fin`, et on garde la fenêtre ouverte grâce à un watermark de plus de 6 minutes : elle finit à 10h05, et le temps des événements a atteint 10h11.

Q2 Pourquoi le mode `append` est-il refusé sur `groupBy("station_depart").count()` ?

Parce qu'aucun compte n'est jamais définitif : une station peut recevoir un trajet à tout moment. Il faut `update` ou `complete`.

Q3 Que se passe-t-il si l'on supprime le dossier de reprise et qu'on relance une requête d'écriture Parquet ?

Le résultat n'est plus garanti. Le puits fichier tient son propre journal (`_spark_metadata`) : la nouvelle requête renumérote ses lots à partir de zéro, et saute ceux que le journal a déjà validés. Des lignes peuvent être perdues sans erreur, d'autres dupliquées. Nouveau point de reprise, nouveau dossier de sortie.

Q4 Qu'est-ce qu'un DStream, et pourquoi ne le choisit-on plus ?

Une suite de RDD, un par intervalle de lot. Déprécié depuis Spark 3.4 : pas de schéma, pas d'heure d'événement, pas de watermark.

Q5 Le modèle du TP 4 prédit n'importe quoi sur le flux, sans erreur. Première hypothèse ?

Une colonne d'entrée calculée autrement qu'à l'entraînement : heure, jour de la semaine, quartier. Même préparation, même code.

ANNEXES DU MODULE 5

AVANCÉ

Le streaming en production

Kafka et ses réglages, `foreachBatch`, sessions, état sur mesure, modèle externe, surveillance, et la place de Spark parmi les moteurs de flux.

La source de production : Kafka

```
spark-submit \  
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.4 \  
  job.py
```

```
brut = (spark.readStream.format("kafka")  
  .option("kafka.bootstrap.servers", "kafka:9092")  
  .option("subscribe", "trajets")  
  .option("startingOffsets", "latest")  
  .load())  
  
trajets = (brut  
  .select(F.from_json(F.col("value").cast("string"),  
    SCHEMA).alias("t"))  
  .select("t.*"))
```

- La version du paquet doit être **celle de Spark**, et le suffixe celle de Scala (**2.12** pour PySpark 3.5).
- Kafka livre des octets : **value** est à décoder, ici du JSON avec le même schéma qu'au TP.
- Les positions lues sont dans le point de reprise, pas dans Kafka : c'est ce qui permet l'« exactement une fois ».
- Vers Kafka en sortie : au moins une fois seulement. Prévoir des consommateurs qui tolèrent un doublon.

Trois options à régler avant la mise en service

OPTION	PAR DÉFAUT	CE QU'ELLE DÉCIDE
<code>startingOffsets</code>	<code>latest</code>	Où lire au tout premier démarrage : <code>earliest</code> reprend tout l'historique du topic. Ensuite, c'est le point de reprise qui fait foi : l'option est ignorée.
<code>maxOffsetsPerTrigger</code>	<i>aucune limite</i>	Le nombre maximal de messages par lot. Sans elle, le premier lot après une panne d'une nuit contient toute la nuit, et peut ne jamais finir.
<code>failOnDataLoss</code>	<code>true</code>	Si des messages ont disparu avant d'être lus (rétention dépassée, topic supprimé) : arrêter la requête, ou continuer en les sautant.

`failOnDataLoss` à `false`

La requête survit, mais des événements manquent **sans erreur**. Acceptable pour un tableau de bord de supervision, pas pour de la facturation. C'est un choix métier, à écrire noir sur blanc.

Les positions ne sont pas dans Kafka

Spark garde ce qu'il a lu dans son point de reprise, pas dans le groupe de consommateurs Kafka. Les outils Kafka qui mesurent le retard d'un groupe ne voient donc rien : surveiller par `lastProgress`.

Un lot, plusieurs destinations, aucun doublon

```
def traiter_lot(lot, batch_id):  
    lot.persist() # lu deux fois  
    (lot.withColumn("batch_id", F.lit(batch_id))  
     .write.mode("overwrite") # rejouer = réécrire  
     .parquet(f"sorties/trajets/batch_id={batch_id}"))  
    alertes = lot.filter("duree_min > 90")  
    if not alertes.isEmpty():  
        alertes.write.mode("append").json("sorties/alertes")  
    print(f"lot {batch_id} : {lot.count()} trajets, "  
          f"{alertes.count()} alerte(s)")  
    lot.unpersist()  
  
q = (trajets.writeStream  
    .foreachBatch(traiter_lot)  
    .option("checkpointLocation", "reprise/lots")  
    .start())
```

```
lot 0 : 3 trajets, 1 alerte(s)  
lot 1 : 2 trajets, 1 alerte(s)  
trajets écrits : 5  
alertes : ['T5', 'T2']
```

- Chaque lot arrive comme un **DataFrame ordinaire** : toute l'API de lots s'y applique, y compris les écritures JDBC vers une base.
- `persist()` : sans lui, chaque écriture relit et recalcule le lot.
- La garantie de base est **au moins une fois** : après une panne, le même lot est rejoué avec **le même batch_id**.
- D'où l'idempotence : écrire le lot à un emplacement qui porte son numéro, en `overwrite`. Rejouer, c'est réécrire, pas dupliquer.

La sortie `alertes`, en `append`, n'a pas cette protection : un lot rejoué y ajoute des doublons.

Regrouper par période d'activité

```
sessions = (evenements
    .withWatermark("heure", "10 minutes")
    .groupBy(F.session_window("heure", "30 minutes"),
        "velo")
    .agg(F.count("*").alias("evenements")))
```

	+	+	+	+	+
	+	+	+	+	+
velo debut	fin evenements				
V0077 10:05 10:35				1	
V0077 12:40 13:10				1	
V0423 10:00 10:55				3	
V0423 11:30 12:00				1	

- Une session **reste ouverte** tant que l'écart entre deux événements d'une même clé est sous 30 minutes.
- Elle se ferme 30 minutes après son dernier événement : V0423, vu à 10h00, 10h12, 10h25, forme une session 10h00 – 10h55.
- Sa durée **n'est pas fixée d'avance** : c'est ce qui la distingue des fenêtres fixes et glissantes.

Disponible depuis Spark 3.2. En mode `append`, une session n'est écrite qu'une fois le watermark passé au-delà de sa fin.

Quand **groupBy** ne suffit pas : l'état à la main

```
from pyspark.sql.streaming.state import GroupStateTimeout

def compter(cle, lots, etat):
    n = etat.get[0] if etat.exists else 0
    for pdf in lots:                # des DataFrames pandas
        n += len(pdf)
    etat.update((n,))
    yield pd.DataFrame({"velo": [cle[0]], "total": [n]})

res = velos.groupBy("velo").applyInPandasWithState(
    compter,
    outputStructType="velo STRING, total LONG",
    stateStructType="n LONG",
    outputMode="update",
    timeoutConf=GroupStateTimeout.NoTimeout)
```

```
+---+---+
|velo|total|
+---+---+
|  0 |  175 |
|  1 |  175 |
|  2 |  175 |
+---+---+
```

- Pour une **session** (du retrait au dépôt), une alerte « vélo non rendu depuis 2 h », une déduplication maison.
- Une fonction par clé, appelée à chaque lot avec les nouvelles lignes **et** l'état laissé au lot précédent.
- Minuteries :
`GroupStateTimeout.ProcessingTimeTimeout` ou `EventTimeTimeout`, pour agir quand **rien** n'arrive.

Disponible depuis Spark 3.4. Spark 4 a ajouté une API plus riche, `transformWithStateInPandas`, absente de la 3.5.

Noter les avis en direct : un modèle par pandas UDF

```
from pyspark.sql.functions import pandas_udf

_MODELE = None
def charger_modele():
    global _MODELE
    if _MODELE is None:
        # une fois par processus
        _MODELE = LEXIQUE # en vrai : un modèle de langue
    return _MODELE

@pandas_udf("string")
def sentiment(textes: pd.Series) → pd.Series:
    m = charger_modele()
    return textes.map(lambda t: noter(t, m)) # tout le lot d'un coup

notes = avis.withColumn("sentiment", sentiment("texte"))
```

station	texte	sentiment
S017	Vélo super rapide, merci !	POSITIF
S023	Station vide à 8h, encore	NEGATIF
S023	Pneu crevé, verrou bloqué	NEGATIF
S040	Pratique pour aller à la gare	POSITIF
S040	Rien à signaler	NEUTRE

Deux règles pour un modèle lourd

Le charger une fois par processus Python, pas à chaque ligne ni à chaque lot : un modèle de langue pèse des centaines de Mo.

Le nourrir par lots : une pandas UDF reçoit des milliers de lignes par appel, là où une UDF classique appelle Python une fois par ligne.

Le modèle doit être sur les exécuteurs

Téléchargé à l'avance, ou installé dans l'image : un exécuteur qui télécharge un modèle à chaque démarrage, derrière un proxy, n'en fait rien.

Savoir si une requête va bien

```
q.lastProgress      # le dernier lot, en dictionnaire
q.recentProgress    # les derniers lots
q.status            # en attente ? en train de lire ?
spark.streams.active # toutes les requêtes en cours
```

```
from pyspark.sql.streaming import StreamingQueryListener

class Ecoule(StreamingQueryListener):
    def onQueryStarted(self, e): print("démarrée", e.name)
    def onQueryProgress(self, e):
        p = e.progress
        print(f"lot {p.batchId} : {p.numInputRows} lignes")
    def onQueryIdle(self, e): pass
    def onQueryTerminated(self, e): print("arrêtée")

spark.streams.addListener(Ecoule())
```

Les trois indicateurs à suivre

`inputRowsPerSecond` vs `processedRowsPerSecond` Si on traite moins vite qu'il n'arrive, le retard s'accumule

`durationMs` Un lot plus long que l'intervalle du déclencheur : la latence dérive

`numRowsDroppedByWatermark` Des retardataires perdus : le délai est trop court

L'onglet **Structured Streaming** de l'interface web trace les mêmes courbes.

Spark, Flink, Kafka Streams : lequel pour quoi ?

	SPARK STRUCTURED STREAMING	APACHE FLINK	KAFKA STREAMS
Nature	Un moteur de traitement, lots et flux	Un moteur de flux, qui sait aussi les lots	Une bibliothèque Java, dans votre application
Modèle	Micro-lots : de la centaine de millisecondes à quelques secondes	Événement par événement : de l'ordre de la milliseconde	Événement par événement
Sources et puits	Fichiers, Kafka, tables, bases...	Nombreux connecteurs	Kafka vers Kafka seulement
Infrastructure	Un cluster Spark, souvent déjà là	Un cluster Flink à exploiter	Aucune : elle tourne dans l'application
En Python	PySpark, la même API qu'en lots	PyFlink, moins répandu	Pas d'API Python officielle

Spark s'impose quand la seconde suffit et qu'on veut **un seul code et une seule équipe** pour les lots, le flux et le machine learning. Flink, quand chaque milliseconde compte ou que l'état est au cœur du métier.

MODULE 6

Théorie des graphes

Les stations comme des nœuds, les trajets comme des arêtes : quelles stations sont au cœur du réseau ?

JOUR 3 • MODULE 6

Théorie des graphes

Quand ce qui compte n'est plus la ligne, mais le lien entre deux lignes.

DURÉE

≈ 1 h 20 d'exposé + TP 1 h 15 +
bilan

À LA FIN

Savoir modéliser un réseau et en extraire les nœuds
clés

TP

PageRank, plus court chemin,
visualisation

Ce que couvre le module 6

1 Qu'est-ce qu'un graphe

Nœuds, arêtes, sens, poids, degré, chemin, connexité.

2 Les algorithmes qui comptent

Parcours en largeur, PageRank, composantes, triangles, communautés.

3 GraphX et GraphFrames

Ce qui existe dans Spark, ce qui est utilisable en Python, et le plan B.

4 TP : le réseau Vélo'Cité

PageRank, plus court chemin, connexité, carte du réseau.

Le fil de l'après-midi

Depuis mercredi, une ligne de `trajets.csv` était un fait isolé.

Cet après-midi, c'est un **lien** entre deux stations.

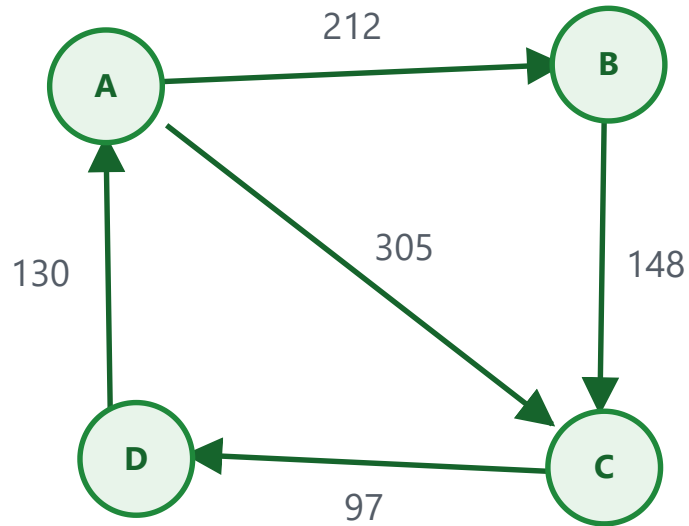
La question change : non plus « combien ? », mais « **qui est relié à qui, et par où passe-t-on ?** »

SÉQUENCE 1

Qu'est-ce qu'un graphe

Le vocabulaire minimal, et une décision de modélisation qui change tous les résultats.

Des nœuds, des arêtes, un sens, un poids



Nœud	Une entité : ici, une station . On dit aussi sommet (<i>vertex</i>).
Arête	Un lien entre deux nœuds : ici, des trajets de A vers B (<i>edge</i>).
Orienté	$A \rightarrow B$ n'est pas $B \rightarrow A$. Le matin, les vélos convergent vers la Gare ; le soir, ils en repartent.
Pondéré	L'arête porte une valeur : un nombre de trajets, une distance, un coût.

Degré, chemin, connexité

Degré	Le nombre d'arêtes d'un nœud. En orienté, on distingue le degré entrant et le degré sortant .
Chemin	Une suite d'arêtes qui se suivent. Sa longueur : le nombre de sauts, ou la somme des poids.
Plus court chemin	Le chemin de longueur minimale entre deux nœuds. « Minimal » dépend du poids choisi : sauts, kilomètres, minutes.
Composante connexe	Un groupe de nœuds reliés entre eux, sans lien avec le reste. Faible si on ignore le sens des arêtes, forte si chacun peut atteindre chacun en respectant le sens.
Triangle	Trois nœuds reliés deux à deux. Beaucoup de triangles : un groupe soudé.

Vélo'Cité en graphe : une décision d'abord

```
aretes = (trajets
  .where("station_depart <> station_arrivee")
  .groupBy(F.col("station_depart").alias("src"),
    F.col("station_arrivee").alias("dst"))
  .agg(F.count("*").alias("trajets")))
```

Stations (nœuds)	60
Couples départ → arrivée	3 540
Boucles écartées	28 796 trajets

$$3\,540 = 60 \times 59$$

Toutes les paires existent : chaque station est à un saut de toutes les autres. Un graphe **complet** ne dit rien.

On garde les liaisons régulières

Au moins **100 trajets** en six mois : il reste **914 arêtes**. Le seuil est un choix métier, et il change tous les résultats qui suivent.

Quand penser « graphe »

QUESTION	VÉLO'CITÉ	AILLEURS	ALGORITHME
Qui est central ?	Les stations par lesquelles tout passe	Le compte pivot d'un réseau de fraude	PageRank, degrés
Comment aller de A à B ?	L'itinéraire d'un rééquilibrage	Une tournée, une chaîne de fournisseurs	Plus court chemin
Qu'est-ce qui est isolé ?	Des stations que rien ne relie	Un sous-réseau informatique coupé	Composantes connexes
Quels groupes se forment ?	Des quartiers qui vivent entre eux	Des clients qui achètent ensemble	Communautés, triangles
Quel schéma se répète ?	Les allers-retours de navetteurs	De l'argent qui tourne en rond	Recherche de motifs

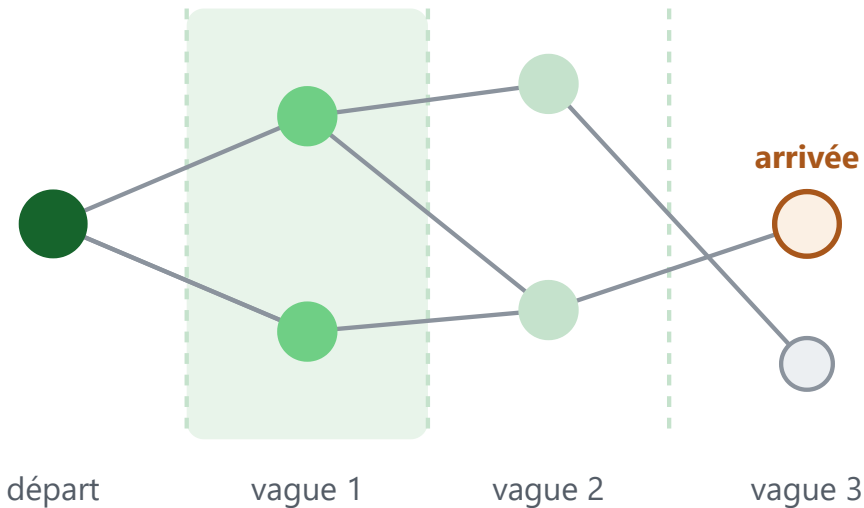
Le signal : la réponse dépend des **relations entre les lignes**, et pas des lignes elles-mêmes. En SQL, cela se traduit par une jointure d'une table sur elle-même, répétée un nombre inconnu de fois.

SÉQUENCE 2

Les algorithmes qui comptent

Tous itératifs : on répète la même étape jusqu'à ce que plus rien ne change.

Le plus court chemin : avancer par vagues



À chaque tour : les voisins de la frontière, moins les stations déjà vues.

```
# esquisse : sans garde-fou si l'arrivée est inatteignable
frontiere = vus = depart # (id, chemin)
while arrivee not in frontiere:
    suivants = (frontiere
                .join(aretes, frontiere.id == aretes.src)
                .select(F.col("dst").alias("id"),
                       F.concat("chemin", F.array("dst"))
                       .alias("chemin"))
                .join(vus.select("id"), "id", "left_anti")
                .dropDuplicates(["id"]))
    vus = vus.unionByName(suivants)
    frontiere = suivants
```

Une vague, c'est **une jointure**. Un chemin de 5 sauts coûte au moins 5 jointures, donc au moins 5 shuffles.

PageRank : un cycliste qui erre au hasard

Un cycliste part d'une station au hasard. À chaque station, il prend une liaison sortante au hasard, **ou**, avec une probabilité de 15 %, se téléporte n'importe où.

Le PageRank d'une station, c'est **la part du temps qu'il y passe** à long terme.

$$\text{rang}(s) = 0,15 / n + 0,85 \times \sum_{v \rightarrow s} \text{rang}(v) / \text{sorties}(v)$$

L'intuition

Une station est importante si des stations importantes y mènent. Être atteint par beaucoup de stations compte ; être atteint par des stations **centrales** compte davantage.

Pourquoi les 15 %

À chaque pas, 15 fois sur 100, le cycliste se **téléporte** n'importe où : il ne tourne jamais sans fin dans une boucle. 15 % est la probabilité de téléportation (**resetProbability**) ; 0,85, le **facteur d'amortissement**. Une station sans sortie demande une règle en plus : on s'y téléporte à coup sûr.

PageRank en DataFrames, et ce qu'il dit

```
for i in range(20):
    recu = (liens.join(rangs, liens.src == rangs.id)
            .groupBy("dst")
            .agg(F.sum(F.col("rang") / F.col("nb_sorties"))
                .alias("recu")))
    rangs = (noeuds.select("id")
            .join(recu, noeuds.id == recu.dst, "left")
            .select("id",
                    (0.15 / n + 0.85 *
                     F.coalesce("recu", F.lit(0.0)))
                    .alias("rang")))
```

STATION	QUARTIER	RANG
Quai Sud	Université	0,0555
Gare Centrale	Affaires	0,0555
Pont Neuf	Vieille Ville	0,0553
...	...	...
Musée	Centre	0,0549
Jardin des Plantes	Faubourg	0,0549

Dix stations autour de 0,055 ; les cinquante autres, **0,009** en moyenne. Un tirage uniforme donnerait 0,017.

Composantes connexes : le réseau tient-il ?

SEUIL	LIAISONS	COMP. FAIBLES	COMP. FORTES
100 trajets	914	1	1
150 trajets	306	1	36
200 trajets	48	18	59

À 100 trajets, le réseau tient d'un seul bloc, dans les deux sens. À 150, il reste d'un seul morceau, mais on ne peut plus **revenir** : les liaisons fortes mènent aux carrefours et n'en repartent pas. À 200, il se disloque.

Comment ça se calcule

Chaque nœud prend pour étiquette le plus petit identifiant entre le sien et ceux de ses voisins, sans tenir compte du sens, et on recommence jusqu'à stabilité. À la fin, une étiquette = une composante **faible**.

Une composante sans sortie

Des vélos y entrent, aucun n'en ressort régulièrement : c'est là que les stations débordent.

Triangles et communautés

Compter les triangles

Pour chaque nœud, le nombre de triangles dont il fait partie. Beaucoup de triangles : ses voisins sont reliés entre eux, le groupe est **soudé**.

Usage typique : repérer un groupe de comptes qui se virent de l'argent entre eux, et à peu près à personne d'autre.

Propagation d'étiquettes

Chaque nœud adopte l'étiquette la plus fréquente chez ses voisins, pendant quelques tours. Des groupes émergent : des **communautés**.

Sur Vélo'Cité, on s'attend à retrouver les quartiers. Si ce n'est pas le cas, c'est une information : les usagers ne découpent pas la ville comme l'urbaniste.

Aucun des deux n'a de résultat « juste » : ce sont des outils d'**exploration**. Ils proposent des groupes, le métier dit s'ils ont un sens.

SÉQUENCE 3

GraphX et GraphFrames

Ce que Spark fournit, ce qui est accessible depuis Python, et comment s'en passer.

Deux bibliothèques, un seul choix en Python

	GRAPHX	GRAPHFRAMES
Livré avec Spark	Oui	Non : paquet séparé, à charger
Construit sur	Les RDD	Les DataFrames
Langages	Scala et Java seulement	Python, Scala, Java
Modèle de calcul	Pregel : des messages entre voisins	Des jointures, et GraphX en coulisses pour certains algorithmes
Recherche de motifs	Non	Oui : <code>find("(a)-[]→(b)")</code>
Requêtes SQL sur le graphe	Non	Oui : nœuds et arêtes sont des DataFrames

Charger GraphFrames

Deux morceaux : un JAR et un module Python

```
pip install graphframes-py

spark = (SparkSession.builder
    .config("spark.jars.packages",
        "io.graphframes:graphframes-spark3_2.12:0.12.2")
    .getOrCreate())

from graphframes import GraphFrame
g = GraphFrame(noeuds, aretes)
```

Colonnes imposées : **id** pour les nœuds, **src** et **dst** pour les arêtes.

La coordonnée dépend de Spark

spark3_2.12 pour Spark 3.5, notre version ;
spark4_2.13 pour Spark 4. Avec la mauvaise, la session démarre, puis `GraphFrame( ... )` échoue sur un `NoClassDefFoundError`.

Le JAR vient d'Internet

Téléchargé depuis Maven Central au démarrage de la session. Derrière un proxy d'entreprise, c'est souvent là que ça bloque.

L'API GraphFrames : un algorithme, une ligne

```
g.inDegrees      # id, inDegree
g.outDegrees     # id, outDegree

g.pageRank(resetProbability=0.15, maxIter=20)
g.shortestPaths(landmarks=["S037"])
g.bfs("id = 'S016'", "id = 'S037'")

g.connectedComponents()
g.stronglyConnectedComponents(maxIter=10)
g.triangleCount(StorageLevel.MEMORY_AND_DISK)
g.labelPropagation(maxIter=5)

g.find("(a)-[e]→(b); (b)-[e2]→(a)")
```

Tout rend des DataFrames

Le résultat se filtre, se joint aux noms des stations, s'écrit en Parquet : tout le module 3 s'applique.

Deux pièges

`connectedComponents()` exige un répertoire de points de reprise : `sc.setCheckpointDir(...)`.

`pageRank` rend des rangs dont la somme vaut **n**, pas 1 : diviser par le nombre de nœuds pour comparer.

Le triplet : une arête et ses deux extrémités

```
t = g.triplets      # src, edge, dst

(t.select(
  F.when(F.col("src.quartier")
        == F.col("dst.quartier"),
        "même quartier")
  .otherwise("autre quartier")
  .alias("liaison"),
  "edge.trajets")
.groupBy("liaison")
.agg(F.count("*").alias("liaisons"),
     F.sum("trajets").alias("trajets"))
.show())
```

	liaison	liaisons	trajets
autre quartier	522	72628	
même quartier	392	59458	

Une ligne par arête, avec ses deux nœuds

Trois colonnes structurées : `src`, `edge`, `dst`. Toute question qui mêle une liaison et les attributs de ses stations s'écrit dessus.

Sans GraphFrames

Deux jointures des arêtes avec les nœuds, une par extrémité. Le triplet les écrit pour vous.

Sans GraphFrames : deux chemins qui suffisent

Le graphe est gros : DataFrames seuls

PageRank et parcours en largeur s'écrivent en une dizaine de lignes de jointures, comme vu plus haut. Pas de paquet, pas de téléchargement, et le calcul reste distribué.

Le graphe est petit : networkx

60 stations et 914 liaisons tiennent dans la mémoire du driver. On les rapatrie avec `collect()`, et on dispose de toute la bibliothèque : Dijkstra, centralités, dessin.

```
G = nx.DiGraph()
G.add_edges_from((r["src"], r["dst"], {"trajets": r["trajets"]}))
for r in aretes.collect()
nx.pagerank(G, alpha=0.85, weight=None)
```

Le piège est le `collect()` : sur un vrai graphe d'un milliard d'arêtes, il fait tomber le driver. La règle : **Spark pour construire et réduire le graphe, networkx pour l'explorer quand il est devenu petit.**

PageRank, plus court chemin et visualisation

≈ 1 h 15

Individuel ou en binôme

tp6-graphes/

Échauffement : [exercices/serie6-graphes/](#)

1

Construire le graphe

Nœuds depuis `stations.csv`, arêtes depuis les trajets propres du TP 3, seuil de 100 trajets.

2

Degrés et PageRank

Les dix stations centrales, avec leur nom. Vérifier contre networkx.

3

Plus court chemin

De « Roseraie » à « Docks », en sauts, par parcours en largeur.

4

Connexité

Composantes faibles et fortes, aux seuils de 100, 150 et 200.

5

La carte du réseau

Positions réelles, couleur par quartier, taille selon le PageRank.

Pour démarrer

Ouvrir `depart.ipynb`, l'énoncé est dans `enonce.md`.
La première cellule détecte seule si GraphFrames est disponible.

Bonus

Le plus court chemin **en kilomètres** : est-ce le même que celui qui a le moins de sauts ?

Cinq questions avant le bilan

Q1 Pourquoi le graphe brut des trajets Vélo'Cité est-il inutilisable tel quel ?

Il est complet : les 3 540 couples possibles existent. Tout est à un saut de tout, les algorithmes ne distinguent plus rien. Il faut un seuil, qui est un choix métier.

Q2 Qu'est-ce qui distingue le PageRank d'une station de son degré entrant ?

Le degré compte les liaisons ; le PageRank les pondère par l'importance de la station d'origine et par le nombre de sorties qu'elle partage.

Q3 Pourquoi un plus court chemin de 6 sauts coûte-t-il cher dans Spark ?

Chaque saut est une vague du parcours, donc au moins une jointure, donc un shuffle. Six sauts, six shuffles au minimum.

Q4 On a fait `pip install graphframes-py`, et `GraphFrame( ... )` échoue. Que manque-t-il ?

Le JAR côté JVM : `spark.jars.packages` avec la coordonnée qui correspond à la version de Spark.

Q5 Quand a-t-on le droit de faire `collect()` sur les arêtes ?

Quand le graphe est devenu petit : Spark l'a déjà agrégé et filtré. Jamais sur les données brutes.

ANNEXES DU MODULE 6 AVANCÉ

Aller plus loin avec les graphes

Le modèle Pregel et ses messages, la recherche de motifs, les communautés, GraphX dans le texte, et les limites de Spark sur les graphes.

Penser comme un sommet

Pregel, publié par Google en 2010, est le modèle de calcul repris par GraphX. Un algorithme s'écrit du point de vue d'**un seul nœud**, en super-étapes :

- 1 **Recevoir**
Les messages envoyés par les voisins à l'étape précédente.
- 2 **Calculer**
Mettre à jour son propre état : un rang, une distance, une étiquette.
- 3 **Envoyer**
Des messages le long de ses arêtes, si son état a changé.

On s'arrête quand plus personne n'envoie rien.

PageRank en Pregel

Message : mon rang divisé par mes sorties.

Calcul : $0,15/n + 0,85 \times \text{somme reçue}$.

C'est exactement la boucle de jointures du TP, écrite autrement.

En Python ?

GraphFrames expose une API Pregel, `g.pregel`, construite sur des DataFrames.

Un tour de messages, sans boucle

```
from graphframes.lib import AggregateMessages as AM

venu_d_ailleurs = F.when(
    AM.src["quartier"] != AM.dst["quartier"],
    AM.edge["trajets"]).otherwise(F.lit(0))

recus = g.aggregateMessages(
    F.sum(AM.msg).alias("venus_d_ailleurs"),
    sendToDst=venu_d_ailleurs)
# puis : rapport au total reçu, et les noms → part
```

Chaque arête **envoie** un message à sa station d'arrivée ;
chaque station **agrège** ce qu'elle reçoit. C'est une super-étape
de Pregel.

id	nom	quartier	part
S003	Gare Centrale	Affaires	0.85
S008	Jardin des Plantes	Faubourg	0.85
S002	Hôtel de Ville	Gare	0.84
...			
S011	Tour Horizon	Affaires	0.0

Deux mondes

Les **10 carrefours** reçoivent 81 à 85 % de leurs trajets réguliers
d'un autre quartier. Les **50 autres stations** : aucun.

Chercher un schéma dans le graphe

```
aller_retour = g.find(
    "(a)-[e1]→(b); (b)-[e2]→(a)")

(aller_retour
    .where("a.id < b.id")
    .select("a.nom", "b.nom",
            "e1.trajets", "e2.trajets")
    .orderBy(F.desc("e1.trajets"))
    .show(5))
```

Chaque groupe entre parenthèses est un nœud, chaque groupe entre crochets une arête. Le point-virgule veut dire « et ».

Ce que ça trouve

239 couples de stations reliées régulièrement **dans les deux sens** : des trajets de navette. En tête : Place du Marché et Musée, 204 et 200 trajets.

Et **436** liaisons sans retour régulier : les flux à rééquilibrer en camionnette.

Motif négatif

"(a)-[]→(b); !(b)-[]→(a)" : les liaisons sans retour.

Les usagers découpent-ils la ville comme l'urbaniste ?

```
carrefours = [f"S{i:03d}" for i in range(1, 11)]
sans = aretes.where(~F.col("src").isin(carrefours)
                  & ~F.col("dst").isin(carrefours))

(GraphFrame(noeuds, sans)
 .labelPropagation(maxIter=5)
 .where(~F.col("id").isin(carrefours))
 .groupBy("label")
 .agg(F.count("*").alias("stations"),
      F.countDistinct("quartier")
      .alias("quartiers"))
 .show())
```

Sur le graphe entier : une seule communauté

60 stations, 8 quartiers. Les dix carrefours relient tout le monde : ils absorbent tout.

Sans les carrefours : 8 communautés

Et chacune correspond à **exactement un quartier**. Les usagers découpent la ville comme l'urbaniste, mais on ne le voit qu'une fois les carrefours retirés.

Lire GraphX dans la documentation

Ce que vous lirez, en Scala

```
val graph = Graph(vertices, edges)
val ranks = graph.pageRank(0.0001).vertices
val cc     = graph.connectedComponents().vertices
graph.triplets
graph.aggregateMessages( ... )
graph.partitionBy(PartitionStrategy.EdgePartition2D)
```

Couper les arêtes, pas les nœuds

GraphX répartit les **arêtes** entre partitions ; un nœud très connecté est **recopié** dans chaque partition qui en a besoin. Un nœud à des millions de liaisons ne tombe donc pas sur une seule machine. La stratégie de découpage se choisit :

`EdgePartition2D`, `RandomVertexCut` ...

GRAPHX

GRAPHFRAMES, EN PYTHON

`Graph(vertices, edges)`

`GraphFrame(noeuds, aretes)`

`pageRank(0.0001)`

`pageRank(tol=0.0001)`

`triplets`, `aggregateMessages`

Mêmes noms

Spark n'est pas une base de données de graphes

BESOIN	OUTIL ADAPTÉ
Analyser tout le graphe , en lot : PageRank, communautés, composantes	Spark et GraphFrames : c'est leur terrain
Explorer un petit graphe , dessiner, tester un algorithme	networkx, sur le graphe réduit par Spark
Répondre vite à une question ponctuelle : « les voisins de ce compte, à trois sauts »	Pas Spark : une base orientée graphe, interrogée en millisecondes
Parcours très profonds , des dizaines de sauts	Pas Spark : chaque saut est un shuffle

Schéma courant : la base de graphes répond au quotidien, Spark recalcule chaque nuit les indicateurs globaux.

CLÔTURE

Bilan des trois jours

Ce qui a été couvert, les questions de mercredi matin, et la suite.

Le programme, et où il a été traité

OBJECTIF DU PROGRAMME	OÙ	CE QUI A ÉTÉ PRODUIT
Les concepts fondamentaux de Spark	Module 1	L'environnement, un premier job
Utiliser les RDD	Module 2	L'analyse du journal brut des bornes
Exploiter les données avec Spark SQL , PySpark comme pandas	Module 3	Les trajets propres, en Parquet
Le machine learning avec Spark	Module 4	Un modèle de type d'usager
L'analyse temps réel avec Spark Streaming	Module 5	Des statistiques et des prédictions en continu
La théorie des graphes	Module 6	La carte du réseau et ses stations clés

Les trois questions de mercredi matin

Q1 Un fichier CSV de 50 Go sur un portable de 16 Go : que faire ?

Le lire avec Spark, qui le découpe en partitions et ne charge jamais tout en mémoire. Et l'écrire aussitôt en Parquet, pour ne plus jamais relire le CSV.

Q2 Pour compter les mots d'un texte, faut-il voir le texte entier ?

Non : `map` sur chaque morceau, `reduceByKey` pour additionner. C'est aussi la structure du PageRank de cet après-midi : distribuer, puis agréger par clé.

Q3 Une machine tombe au milieu du calcul : que refait-on ?

Seulement ses partitions, recalculées depuis le lignage. Et pour les calculs très longs, comme les itérations de graphe, un point de reprise coupe ce lignage.

Après la formation

Les références

- **spark.apache.org/docs/latest** : la documentation officielle, dont les guides de programmation SQL, Structured Streaming et MLlib.
- La référence de l'API Python, **PySpark**, sur le même site : chaque fonction, avec un exemple.
- **graphframes.io** : la documentation de GraphFrames.

Les exercices, et la suite

Le dépôt github.com/WyTaSoft-Academy/pyspark-hands-on-labs : six énoncés, cinquante exercices guidés, et le générateur de données. Tout tourne hors connexion, GraphFrames excepté : le TP 6 a son plan B.

Le deck « **Optimiser un job Spark** » ([08-optimiser.html](#), **Avancé**) : hors programme, à lire quand un job devient lent.

Les plateformes gérées

En production, Spark tourne rarement sur un cluster installé à la main : on le trouve chez les grands fournisseurs de cloud et sur des plateformes spécialisées. Le code écrit ici y tourne tel quel ; ce qui change, c'est la configuration de la session.

Le premier pas lundi

Reprendre un script pandas qui souffre, et le réécrire en PySpark. C'est l'exercice le plus rentable.

Évaluation et attestation

1

L'évaluation de la formation

Le questionnaire de l'organisme, à remplir maintenant, avant de partir. Dix minutes.

2

La feuille de présence

Signée pour la dernière demi-journée.

3

L'attestation de fin de formation

Objectifs, durée, et résultat de l'évaluation des acquis. Transmise par l'organisme.

4

Le support

Tous les decks en PDF, dont « Optimiser un job Spark », avec le dossier des TP.

Vos retours servent vraiment

Ce qui a été trop long, trop court, trop rapide : c'est ce qui change la session suivante. Une phrase précise vaut mieux qu'une note.

Merci

Questions, retours, et bonne route avec Spark.

Mehdi Tajmouati · Support de cours transmis en fin de session.

 mehdi.tajmouati@wytasoft.com  www.wytasoft.com  Udemy  Medium

FIL PRATIQUE · LES TROIS JOURS

Les six TP pas à pas

Le code de chaque étape, tel qu'il doit finir, dans l'ordre des corrigés.

QUAND

Pendant chaque TP, et à la restitution

SOURCE

`tpN-.../corrige.py`

VERSION

Spark 3.5.4

Comment se servir de ce deck

TP	MODULE	DOSSIER
TP 1	L'environnement et un premier script	tp1-environnement
TP 2	Le journal brut des bornes, en RDD	tp2-rdd
TP 3	Nettoyer, joindre, interroger	tp3-sql-dataframes
TP 4	Abonné ou occasionnel ?	tp4-ml
TP 5	Statistiques et prédictions en temps réel	tp5-streaming
TP 6	PageRank, plus court chemin, visualisation	tp6-graphes

Chaque dossier contient

`enonce.md` pour les consignes, `depart.ipynb` pour démarrer, `corrige.py` et `corrige.ipynb` pour la solution. Les slides qui suivent en reprennent le code, étape par étape.

Les TP s'enchaînent, sans dépendre l'un de l'autre

Le TP 3 écrit les trajets nettoyés, le TP 4 le modèle, repris aux TP 5 et 6. Si une sortie manque, le TP la reconstruit lui-même : on peut rejoindre la formation en cours de route.

TP 1 · MODULE 1

L'environnement et un premier script

Installer, vérifier, lire 300 000 trajets, et compter les jobs que Spark lance.

Installer et vérifier le poste

```
python -m venv .venv
.venv\Scripts\activate
pip install "pyspark==3.5.4"
pip install "numpy<2" "pandas<2.3"
pip install pyarrow jupyter

cd tp
python tp1-environnement/verifier_env.py
python donnees/generer_donnees.py
```

Java 11 ou 17 pour Spark 3.5, **pas 21**, avec `JAVA_HOME` défini.

```
Un vrai job Spark
[OK] Job RDD      somme des carrés = 338350
[OK] Job DataFrame somme = 4999999500000
[OK] Interface web http://localhost:4040

Poste prêt pour les TP.
```

SYMPTÔME

REMÈDE

Accept timed out

`PYSPARK_PYTHON` = chemin de `python.exe`

'JavaPackage' object is not callable

Supprimer un `SPARK_HOME` d'une autre version

L'écriture échoue

`winutils.exe` et `HADOOP_HOME`

Créer la session, lire les trajets

```
from pyspark.sql import SparkSession, functions as F

spark = (SparkSession.builder
        .master("local[2]")
        .appName("tp1-premier-script")
        .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

trajets = spark.read.csv(
    os.path.join(DONNEES, "trajets.csv"),
    header=True, inferSchema=True)

trajets.printSchema()
trajets.count()           # 300300
trajets.rdd.getNumPartitions() # 2
```

```
root
├─ trajet_id: string
├─ velo_id: string
├─ station_depart: string
├─ station_arrivee: string
├─ debut: timestamp
├─ fin: timestamp
├─ duree_min: double
├─ distance_km: double
├─ type_usager: string
├─ electrique: integer
└─ tarif_eur: double
```

La lecture seule lance déjà **2 jobs** : `inferSchema` parcourt le fichier pour deviner les types.

Transformation ou action : la preuve

```
suivi = spark.sparkContext.statusTracker()

def nb_jobs():
    return len(suivi.getJobIdsForGroup())

avant = nb_jobs()
longs = (trajets
        .filter(F.col("duree_min") > 30)
        .select("trajet_id", "type_usager"))
print(nb_jobs() - avant)          # 0

avant = nb_jobs()
print(longs.count())              # 22829
print(nb_jobs() - avant)          # 2
```

Ce qu'on vient de mesurer

`filter` et `select` : **0 job**. Ils décrivent le calcul.
`count` : le calcul a lieu.

Pourquoi 2 et pas 1

L'exécution adaptative lance d'abord l'étape avant le shuffle, mesure, puis la suite : une action, deux jobs dans l'onglet **Jobs**.

Qui roule, d'où, et le plan

```
(trajets.groupBy("type_usager")
  .agg(F.count("*").alias("trajets"),
    F.round(F.avg("duree_min"), 1)
    .alias("duree_moy"))
  .orderBy(F.desc("trajets"))
  .show())
```

type_usager	trajets	duree_moy
abonne	190269	8.6
occasionnel	109881	20.4
NULL	150	12.2

```
(trajets.groupBy("station_depart")
  .count()
  .orderBy(F.desc("count"))
  .show(5))
# S052 5143 · S051 5127 · S006 5122
```

```
trajets.groupBy("station_depart") \
  .count().explain()
```

```
HashAggregate(keys=[station_depart], count(1))
+- Exchange hashpartitioning(station_depart, 200)
   +- HashAggregate(partial_count(1))
      +- FileScan csv [station_depart]
```

TP 2 · MODULE 2

Analyser le journal brut des bornes

Quelles bornes tombent le plus en panne ? Neuf étapes, uniquement avec des RDD.

Charger, analyser, compter les invalides

```
lignes = sc.textFile(JOURNAL)           # 2 partitions, 608 411 lignes

TYPES = {"RETRAIT": {"velo", "badge"},
         "DEPOT": {"velo", "duree"},
         "ERREUR": {"code", "message"}}
PAIRE = re.compile(r'(\w+)=(("[^"]*"|\'[^\']*\'|\S+))')
STATION = re.compile(r"^S\d{3}$")
invalides = sc.accumulator(0)

def analyser(ligne):
    morceaux = ligne.split(" ", 3)
    if len(morceaux) < 4:
        invalides.add(1)
        return []
    horodatage, station, type_, reste = morceaux
    champs = {k: v.strip('"') for k, v in PAIRE.findall(reste)}
    if (len(horodatage) != 19 or not STATION.match(station)
        or type_ not in TYPES or not TYPES[type_] <= champs.keys()
        or reste.count('"') % 2):
        invalides.add(1)
        return []
    return [(horodatage, station, type_, champs)]

evenements = lignes.flatMap(analyser)
evenements.count()      # 607165
invalides.value          # 1246
```

Pourquoi flatMap

Renvoyer `[]` fait disparaître la ligne : analyse et filtre en un seul passage.

Seconde action : 2492

Sans cache, `analyser` est rappelée, et l'accumulateur compte deux fois.

Aperçu sous Windows

`spark.read.text(JOURNAL).show(3)` plutôt que `lignes.take(3)`.

Mettre en cache, et mesurer

```
def chrono(action):  
    debut = time.perf_counter()  
    action()  
    return time.perf_counter() - debut  
  
invalides = sc.accumulator(0)  
evenements = lignes.flatMap(analyser).cache()  
  
chrono(lambda: lignes.flatMap(analyser).count())  
chrono(evenements.count)    # remplit le cache  
chrono(evenements.count)    # lit le cache
```

Le second `count()` sur le RDD en cache est **plusieurs fois plus rapide** : il ne relit plus le fichier et ne rappelle plus Python.

Avec le cache, l'accumulateur ne compte plus aux actions suivantes. Ici il vaut **2 492** : l'appel sans cache, puis le remplissage du cache.

Les durées dépendent du poste. Ce qui doit se retrouver partout, c'est le rapport entre la première et la seconde action.

Par type, par station, par code

```
(evenements.map(lambda e: (e[2], 1))
      .reduceByKey(lambda a, b: a + b).collect())
# DEPOT 299083 · ERREUR 9001 · RETRAIT 299081

noms = (sc.textFile(STATIONS)
      .filter(lambda l: not l.startswith("station_id"))
      .map(lambda l: l.split(","))
      .map(lambda c: (c[0], c[1]))
      .collectAsMap())
noms_diffuses = sc.broadcast(noms)

top_pannes = (evenements
      .filter(lambda e: e[2] == "ERREUR")
      .map(lambda e: (e[1], 1))
      .reduceByKey(lambda a, b: a + b)
      .sortBy(lambda kv: (-kv[1], kv[0]))
      .map(lambda kv: (kv[0], noms_diffuses.value[kv[0]], kv[1])))
top_pannes.take(10)
```

STATION	ERREURS
S016 Roseraie	173
S014 Bibliothèque	172
S008 Jardin des Plantes	171
S033 Parvis	170
S044 Vieux Pont	168

```
(evenements.filter(lambda e: e[2] == "ERREUR")
      .map(lambda e: ((e[3]["code"], e[3]["message"]), 1))
      .reduceByKey(lambda a, b: a + b)
      .sortByKey().collect())
# E12 2275 · E27 2159 · E42 2258 · E51 2309
```


Une moyenne, un fichier, un job

```
somme, effectif = (evenements
    .filter(lambda e: e[2] == "RETRAIT"
              and "batterie" in e[3])
    .map(lambda e: (int(e[3]["batterie"]), 1))
    .reduce(lambda a, b: (a[0] + b[0], a[1] + b[1])))
somme / effectif      # 57.0 %, sur 112 427 retraits

cible = os.path.join(SORTIES, "tp2-top-pannes")
shutil.rmtree(cible, ignore_errors=True)
(sc.parallelize(top_pannes.take(10))
 .map(lambda t: f"{t[0]};{t[1]};{t[2]}")
 .coalesce(1)
 .saveAsTextFile(cible))
```

On réduit des couples `(somme, effectif)` : additionner des moyennes partielles serait faux dès que les partitions n'ont pas le même nombre de retraits.

`coalesce(1)` : dix lignes, un seul fichier `part-00000`.

```
spark-submit --master "local[2]" corrige.py
```

Sur un cluster, seul `--master` change, **une fois le `.master( ... )` retiré du code** : sinon il l'emporte (voir `job_standalone.py`).

TP 3 · MODULE 3

Nettoyer, joindre, interroger

Des trajets bruts à une table propre, enrichie, interrogée deux fois, et écrite en Parquet.

Lire avec un schéma, compter, nettoyer

```
brut = (spark.read.option("header", True)
        .schema(schema)
        .csv(os.path.join(DONNEES, "trajets.csv"))
        .withColumn("debut", F.to_timestamp("debut"))
        .withColumn("fin", F.to_timestamp("fin")))

brut.count() # 300300
```

```
brut.select(
    F.sum((F.col("duree_min") ≤ 0).cast("int"))
      .alias("duree_negative"),
    F.sum(F.col("station_arrivee").isNull()
          .cast("int")).alias("sans_arrivee"),
    F.sum(F.col("type_usager").isNull()
          .cast("int")).alias("sans_type"),
).show()

brut.count() - brut.dropDuplicates().count()
```

```
+-----+-----+-----+
|duree_negative|sans_arrivee|sans_type|
+-----+-----+-----+
|          300|          300|        150|
+-----+-----+-----+

doublons exacts : 300
```

```
propres = (brut
            .dropDuplicates()
            .filter(F.col("duree_min") > 0)
            .dropna(subset=["station_arrivee",
                             "type_usager"])
            .withColumn("heure", F.hour("debut"))
            .withColumn("jour_semaine",
                        F.dayofweek("debut"))
            .withColumn("week_end",
                        F.col("jour_semaine").isin(1, 7)))

propres.cache()
propres.count() # 299251
```

Joindre, puis poser les questions deux fois

```
enrichis = (propres
    .join(stations.select(
        F.col("station_id").alias("station_depart"),
        F.col("quartier")),
        on="station_depart", how="left")
    .withColumn("date", F.to_date("debut"))
    .join(meteo, on="date", how="left"))

(enrichis.groupBy("quartier")
    .agg(F.count("*").alias("trajets"))
    .orderBy(F.desc("trajets"))).show()

enrichis.createOrReplaceTempView("trajets")
spark.sql("""
    SELECT CASE WHEN pluie_mm = 0 THEN 'sec'
        WHEN pluie_mm < 5 THEN 'pluie faible'
        ELSE 'pluie forte' END AS temps,
        COUNT(DISTINCT date) AS jours,
        ROUND(COUNT(*) / COUNT(DISTINCT date))
        AS trajets_par_jour
    FROM trajets GROUP BY 1
    ORDER BY trajets_par_jour DESC""").show()
```

quartier	trajets
Gare	40004
Centre	39963
Université	39878
Affaires	39833
Faubourg	35215
Vieille Ville	34948
Berges	34942
Parc	34468

temps	jours	trajets_par_jour
sec	118	1727.0
pluie faible	60	1538.0
pluie forte	3	1050.0

Une fenêtre, et l'API pandas

```
par_station = (enrichis
    .groupBy("quartier", "station_depart").count())
fenetre = (Window.partitionBy("quartier")
    .orderBy(F.desc("count")))
(par_station
    .withColumn("rang", F.rank().over(fenetre))
    .filter("rang = 1")
    .orderBy("quartier").show())
```

quartier	station_depart	count	rang
Affaires	S051	5113	1
Berges	S006	5094	1
Centre	S041	5086	1
Faubourg	S048	5086	1
Gare	S058	5083	1
Parc	S053	5006	1
Université	S052	5120	1
Vieille Ville	S023	5101	1

```
psdf = (enrichis
    .select("type_usager", "duree_min")
    .pandas_api())
(psdf.groupby("type_usager")["duree_min"]
    .mean().sort_index())
```

```
type_usager
abonne          8.662582
occasionnel     20.484214
Name: duree_min, dtype: float64
```

Même résultat qu'avec `groupBy().agg()` : c'est le même plan Spark, écrit en pandas.

Écrire en Parquet, et mesurer

```
cible = os.path.join(SORTIES, "trajets-propres")
propres.coalesce(4).write.mode("overwrite").parquet(cible)
```

```
def chrono(lire, n=3):
    temps = []
    for _ in range(n):
        t = time.perf_counter()
        (lire().groupBy("type_usager")
         .agg(F.avg("duree_min")).collect())
        temps.append(time.perf_counter() - t)
    return temps
```

```
def lire_csv():
    return (spark.read
            .option("header", True)
            .option("inferSchema", True)
            .csv(os.path.join(DONNEES, "trajets.csv")))

min(chrono(lire_csv))
min(chrono(lambda: spark.read.parquet(cible)))
```

trajets.csv

27,2 Mo

trajets-propres/

10,3 Mo en 4 fichiers

Le contrat pour les TP suivants

Les onze colonnes du CSV, typées, plus `heure`, `jour_semaine` et `week_end`. Pas les colonnes de jointure : chaque TP joint ce dont il a besoin.

Les temps mesurés dépendent du poste : comparez l'écart CSV / Parquet sur **votre** machine, pas d'un binôme à l'autre.

TP 4 · MODULE 4

Abonné ou occasionnel ?

Une classification supervisée, de la donnée nettoyée au modèle sauvegardé pour le TP 5.

Joindre le quartier, découper, mesurer la référence

```
trajets = spark.read.parquet(PROPRES)
stations = spark.read.csv(
    os.path.join(DONNEES, "stations.csv"),
    header=True, inferSchema=True)

donnees = (trajets
    .join(stations.select(
        F.col("station_id").alias("station_depart"),
        "quartier"), "station_depart")
    .withColumn("label",
        (F.col("type_usager") == "occasionnel")
        .cast("double")))
```

```
dans_test = F.pmod(F.hash("trajet_id"), 5) == 0
train = donnees.where(~dans_test).cache()
test = donnees.where(dans_test).cache()

part_abonnes = (test.where("label = 0").count()
    / test.count())
```

```
entraînement : 238961  test : 60290
référence « toujours abonné » : exactitude 0.637
```

Le hachage de l'identifiant désigne toujours les mêmes 20 % :
mêmes chiffres sur tous les postes.

Au retrait : la logistique, et son évaluation

```
AU_RETRAIT = ["heure", "jour_semaine", "week_end",
               "electrique", "quartier_vec"]
indexeur = StringIndexer(inputCol="quartier",
                          outputCol="quartier_idx", handleInvalid="keep")
encodeur = OneHotEncoder(inputCols=["quartier_idx"],
                          outputCols=["quartier_vec"])
assembleur = VectorAssembler(inputCols=AU_RETRAIT,
                              outputCol="brutes")
normaliseur = StandardScaler(inputCol="brutes",
                              outputCol="features")
reglog = LogisticRegression(labelCol="label",
                             featuresCol="features")
modele_lr = Pipeline(stages=[indexeur, encodeur,
                              assembleur, normaliseur, reglog]).fit(train)
predictions_lr = modele_lr.transform(test)
```

```
auc = BinaryClassificationEvaluator(
    labelCol="label", metricName="areaUnderROC")
exactitude = MulticlassClassificationEvaluator(
    labelCol="label", metricName="accuracy")

(predictions_lr.groupBy("label")
 .pivot("prediction", [0.0, 1.0])
 .count().orderBy("label").show())
```

```
logistique au retrait      AUC 0.706    exactitude 0.677
+-----+-----+-----+
|label|  0.0|  1.0|
+-----+-----+-----+
|  0.0|32178|6218|
|  1.0|13284|8610|
+-----+-----+-----+
```

Au retrait : la forêt aléatoire et ses importances

```
foret = RandomForestClassifier(
    labelCol="label", featuresCol="brutes",
    numTrees=30, maxDepth=8, seed=42)
modele_rf = Pipeline(stages=[indexeur, encodeur,
    assembleur, foret]).fit(train)
predictions_rf = modele_rf.transform(test)

attrs = (predictions_rf.schema["brutes"]
    .metadata["ml_attr"]["attrs"])
noms = sorted((a["idx"], a["name"]))
for groupe in attrs.values() for a in groupe)
importances = (modele_rf.stages[-1]
    .featureImportances.toArray())
```

forêt au retrait	AUC 0.809	exactitude 0.742
heure	0.578	
electrique	0.166	
week_end	0.104	
quartier_vec_Parc	0.045	
jour_semaine	0.043	

Les noms des variables sont dans les **métadonnées** de la colonne `brutes`, posées par l'assembleur.

Au dépôt : ajouter la durée, sauvegarder le meilleur

```
AU_DEPOT = AU_RETRAIT + ["duree_min",  
                          "distance_km"]  
assembleur_depot = VectorAssembler(  
    inputCols=AU_DEPOT, outputCol="brutes")  
depot_lr = Pipeline(stages=[indexeur, encodeur,  
    assembleur_depot, normaliseur, reglog]).fit(train)  
depot_rf = Pipeline(stages=[indexeur, encodeur,  
    assembleur_depot, foret]).fit(train)  
  
depot_lr.write().overwrite().save(MODELE)  
recharge = PipelineModel.load(MODELE)
```

logistique au dépôt	AUC 0.992	exactitude 0.966
forêt au dépôt	AUC 0.985	exactitude 0.943
modèle rechargé	AUC 0.992	exactitude 0.966

Avec la durée, la logistique repasse devant : c'est elle qu'on sauvegarde pour le TP 5.

La fuite de cible, et le déséquilibre

```
assembleur_fuite = VectorAssembler(
    inputCols=AU_RETRAIT + ["tarif_eur"],
    outputCol="brutes")
fuite = Pipeline(stages=[indexeur, encodeur,
    assembleur_fuite, normaliseur,
    reglog]).fit(train)
```

```
au retrait + tarif      AUC 1.000    exactitude 1.000
```

Le tarif est **calculé d'après le type d'usager** : c'est une fuite. Un score parfait obtenu sans effort se soupçonne d'abord.

```
part_occ = (train.where("label = 1").count()
            / train.count())
train_pondere = train.withColumn("poids",
    F.when(F.col("label") == 1, 1 - part_occ)
    .otherwise(part_occ))
LogisticRegression(labelCol="label",
    featuresCol="features", weightCol="poids")
```

```
sans pondération  rappel 0.393  précision 0.581
avec pondération  rappel 0.637  précision 0.504
AUC 0.706 dans les deux cas
```

TP 5 · MODULE 5

Statistiques et prédictions en temps réel

Un flux de trajets, des comptes qui bougent, un modèle qui prédit au fil de l'eau, et une requête qui reprend sans rien perdre.

Lire le flux, compter par station

```
SCHEMA = """
    trajet_id STRING, velo_id STRING,
    station_depart STRING, station_arrivee STRING,
    debut TIMESTAMP, fin TIMESTAMP,
    duree_min DOUBLE, distance_km DOUBLE,
    type_usager STRING, electrique INT,
    tarif_eur DOUBLE
"""

trajets = (spark.readStream
    .schema(SCHEMA)
    .option("maxFilesPerTrigger", 10)
    .json(ENTREE))
```

```
comptes = trajets.groupBy("station_depart").count()

q_comptes = (comptes.writeStream
    .outputMode("complete")
    .format("memory")
    .queryName("comptes_stations")
    .trigger(processingTime="5 seconds")
    .start())

spark.sql("""SELECT station_depart, count
    FROM comptes_stations
    ORDER BY count DESC LIMIT 5""").show()
```

complete : un compte déjà écrit change encore. Relancer le **SELECT** dix secondes plus tard : les nombres ont grandi.

Joindre les stations, fenêtrer par quartier

```
enrichis = (trajets
    .join(stations.select(F.col("station_id").alias("station_depart"), "quartier"),
        "station_depart")
    .withColumn("heure", F.hour("debut"))
    .withColumn("jour_semaine", F.dayofweek("debut"))
    .withColumn("week_end", F.dayofweek("debut").isin(1, 7)))

par_quartier = (enrichis
    .withWatermark("fin", "10 minutes")
    .groupBy(F.window("fin", "5 minutes"), "quartier")
    .agg(F.count("*").alias("trajets"),
        F.round(F.avg("duree_min"), 1).alias("duree_moy")))

q_quartiers = (par_quartier.writeStream.outputMode("update")
    .format("memory").queryName("par_quartier")
    .trigger(processingTime="5 seconds").start())
```

Le puits **memory** en **update** **empile** les versions d'une même fenêtré : pour lire le dernier état, regrouper par fenêtré et garder le plus grand compte.

Le modèle du TP 4, sur le flux

```
modele = (PipelineModel.load(MODELE)
         if os.path.exists(MODELE)
         else modele_de_secours())

predits = (modele.transform(enrichis)
          .withColumn("reel",
                      (F.col("type_usager") == "occasionnel")
                      .cast("double")))

part_predite = (predits
               .withWatermark("fin", "10 minutes")
               .groupBy(F.window("fin", "5 minutes"))
               .agg(F.count("*").alias("trajets"),
                   F.round(F.avg("prediction"), 3)
                   .alias("part_predite"),
                   F.round(F.avg("reel"), 3)
                   .alias("part_reelle")))
```

modele_de_secours()

Si le TP 4 n'a pas été terminé : le même pipeline (régression logistique « au dépôt »), entraîné sur 20 % de l'historique, avec les mêmes colonnes d'entrée. Le code est dans le corrigé.

Attendu

Sur la tranche en cours, une part prédite très proche de la part réelle : 0,387 pour 0,409 à la préparation.

Arrêter, relancer, compter les doublons

```
def lancer_ecriture():  
    return (predits.select("trajet_id", "fin",  
                           "quartier", "type_usager", "prediction")  
            .writeStream  
            .format("parquet")  
            .outputMode("append")  
            .option("path", PREDICTIONS)  
            .option("checkpointLocation",  
                    os.path.join(REPRISE, "predictions"))  
            .trigger(processingTime="5 seconds")  
            .start())
```

```
q = lancer_ecriture()  
time.sleep(10); q.stop()  
print(q.lastProgress["batchId"]) # après stop()  
  
time.sleep(10) # l'émetteur continue  
q = lancer_ecriture()  
time.sleep(10)  
print(q.recentProgress[0]["batchId"])  
q.stop()  
  
ecrits = spark.read.parquet(PREDICTIONS)  
print(ecrits.count(),  
      escrits.select("trajet_id").distinct().count())
```

Le premier lot relancé suit le dernier lot validé, et les deux comptes sont **égaux** : aucun doublon.

TP 6 · MODULE 6

PageRank, plus court chemin et visualisation

Le réseau Vélo'Cité en graphe : construire, classer, parcourir, dessiner.

Des trajets au graphe

```
noeuds = (spark.read.csv(os.path.join(DONNEES, "stations.csv"),
                                   header=True, inferSchema=True)
          .withColumnRenamed("station_id", "id"))

toutes = (trajets
          .where("station_depart <> station_arrivee")
          .groupBy(F.col("station_depart").alias("src"),
                  F.col("station_arrivee").alias("dst"))
          .agg(F.count("*").alias("trajets"),
              F.round(F.avg("distance_km"), 2)
              .alias("distance_km")))

SEUIL = 100
aretes = toutes.where(F.col("trajets") ≥ SEUIL).cache()
```

299251 trajets
60 nœuds · 3540 arêtes
28796 boucles écartées
914 liaisons régulières

Les noms `id`, `src` et `dst` sont imposés par GraphFrames. Les garder aussi sans GraphFrames : le code reste le même dans les deux cas.

Degrés, PageRank, et la vérification

```
entrants = (aretes.groupBy(F.col("dst").alias("id"))
            .agg(F.count("*").alias("degre_entrant")))

if GRAPHFRAMES:
    g = GraphFrame(noeuds, aretes)
    pr = g.pageRank(resetProbability=0.15,
                   maxIter=20).vertices
    rangs = pr.select("id", (F.col("pagerank")
                           / noeuds.count()).alias("rang"))
else:
    rangs = pagerank_dataframes(noeuds, aretes)

# G : le même graphe dans networkx (nx.DiGraph, voir le corrigé)
reference = nx.pagerank(G, alpha=0.85, weight=None)
ecart = max(abs(reference[r["id"]] - r["rang"])
            for r in rangs.collect())
```

id	nom	quartier	rang
S004	Quai Sud	Université	0.0555
S003	Gare Centrale	Affaires	0.0555
S007	Pont Neuf	Vieille Ville	0.0553
S010	Cathédrale	Gare	0.0553
S002	Hôtel de Ville	Gare	0.0553

écart maximal avec networkx : 0.000000

Dix stations ont un degré entrant de **58 ou 59** : presque toutes les autres y mènent. Ce sont elles qui dominent le PageRank.

Le plus court chemin, par vagues

```
def plus_court_chemin(aretes, depart, arrivee, max_sauts=10):
    vus = spark.createDataFrame([(depart, [depart])],
                                "id string, chemin array<string>")
    frontiere = vus
    for _ in range(max_sauts):
        suivants = (frontiere
                    .join(aretes, frontiere.id == aretes.src)
                    .select(F.col("dst").alias("id"),
                           F.concat("chemin", F.array("dst"))
                               .alias("chemin"))
                    .join(vus.select("id"), "id", "left_anti")
                    .dropDuplicates(["id"]))
        if suivants.isEmpty():
            return None
        trouve = suivants.where(F.col("id") == arrivee).first()
        if trouve:
            return trouve["chemin"]
        vus = vus.unionByName(suivants).localCheckpoint()
        frontiere = suivants
    return None
```

2 sauts : Roseraie → Opéra → Docks
networkx : 2 sauts

Avec GraphFrames

`g.shortestPaths(landmarks=[ARRIVEE])` rend la distance, `{S037 → 2}`, pas le chemin.

Bonus, en kilomètres

Roseraie → Planétarium → Opéra → Docks : **6,00 km**, un saut de plus.

La connexité, et la carte du réseau

```
for seuil in (100, 150, 200):
    H = nx.DiGraph()
    H.add_nodes_from(G.nodes)
    H.add_edges_from((r["src"], r["dst"]) for r in
        toutes.where(F.col("trajets") ≥ seuil).collect())
    print(seuil,
        nx.number_weakly_connected_components(H),
        nx.number_strongly_connected_components(H))

nx.draw_networkx_nodes(G, pos, ax=ax,
    node_size=[rang_de[n] * 30000 for n in G.nodes],
    node_color=[couleur[G.nodes[n]["quartier"]]
        for n in G.nodes])
fig.savefig(IMAGE, dpi=120, bbox_inches="tight")
```

SEUIL	LIAISONS	FAIBLES	FORTES
100	914	1	1
150	306	1	36
200	48	18	59

L'image est dans `sorties/graphe-stations.png` : les dix carrefours ressortent, un par quartier ou presque.

FIN DU FIL PRATIQUE

Six TP, un seul jeu de données

Du journal brut au graphe du réseau, étape par étape. Les énoncés et les notebooks de départ sont dans le dépôt `pyspark-hands-on-labs` ; ce deck, qui en montre les corrigés, part avec le support.

HORS PROGRAMME · AVANCÉ

Optimiser un job Spark

Les cinq causes d'un job lent, comment les lire dans l'interface web, et quoi changer — dans cet ordre.

STATUT

Hors programme contractuel

QUAND

Questions de la salle, lecture après la session

VERSION

Spark 3.5.4

Mesurer, localiser, changer une chose, remesurer

1 Mesurer

Un job lent se regarde dans l'interface web, pas dans le code. Onglet Stages : quelle étape dure, et pourquoi.

2 Localiser la cause

Presque toujours l'un des cinq S : spill, shuffle, skew, storage, serialization. La slide suivante dit où voir chacun.

3 Changer une seule chose

Une option de configuration, une jointure, un format. Deux changements à la fois, et on ne sait plus lequel a joué.

4 Remesurer

Sur le même volume, deux fois. Un gain qui ne se remesure pas n'existe pas.

Hors programme

Rien ici n'est évalué. Le socle est dans les modules : les partitions au 1 et au 2, les jointures et AQE au 3. Ce deck les prolonge.

Le premier réflexe

Avant toute option : le code lit-il des colonnes et des lignes dont il n'a pas besoin ? Filtrer et projeter tôt bat tous les réglages.

Les cinq S d'un job lent

CAUSE	CE QUI SE PASSE	OÙ LE VOIR	PREMIER REMÈDE
Spill	Une partition ne tient pas en mémoire : Spark écrit sur disque et relit	Stages : colonnes <i>Spill (Memory)</i> et <i>Spill (Disk)</i>	Des partitions plus petites
Shuffle	Les données traversent le réseau pour être regroupées par clé	Stages : <i>Shuffle Read / Write</i> ; Exchange dans le plan	Diffuser la petite table, agréger plus tôt
Skew	Une clé concentre les données : une tâche fait le travail de toutes	Summary Metrics : tâche max >> médiane	AQE pour les jointures, salage sinon
Storage	Mauvais format, fichiers trop petits ou trop gros, tout est relu	SQL : fichiers lus, <i>PartitionFilters</i> vides	Parquet, fichiers de 128 Mo à 1 Go, partitionner
Serialization	Le temps passe à convertir des objets, pas à calculer	Durée des tâches, <i>GC Time</i> ; SQL : <i>BatchEvalPython</i> , <i>ArrowEvalPython</i>	Pas d'UDF Python, Arrow pour pandas

Ils se tiennent : un shuffle mal découpé fait du spill, une clé déséquilibrée fait du spill sur une seule tâche. On corrige la **cause**, pas le symptôme.

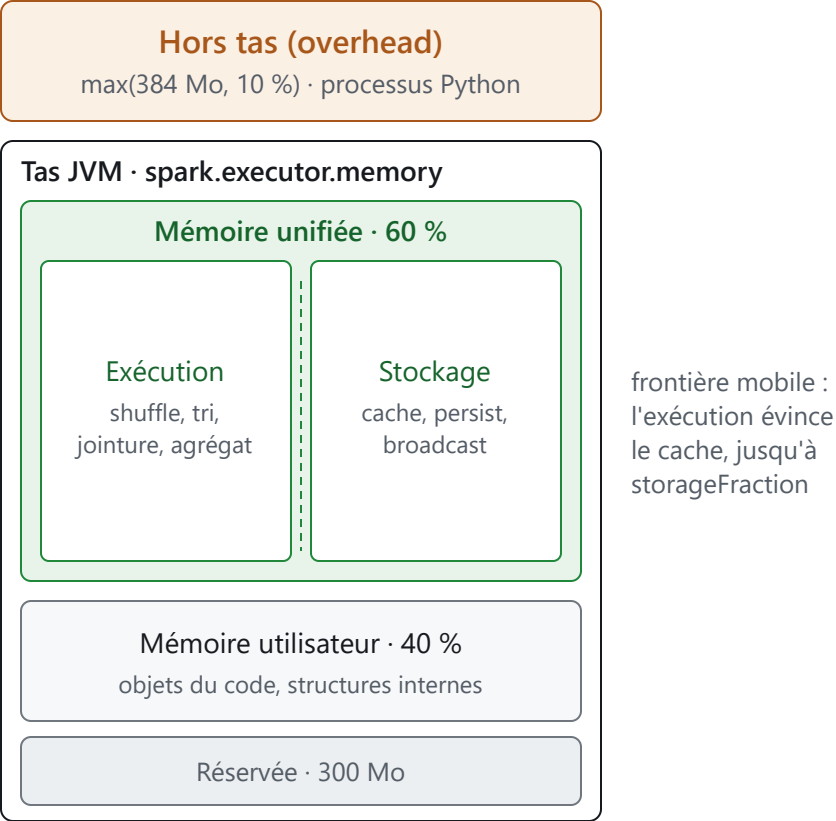
AVANT DE RÉGLER

AVANCÉ

La mémoire d'un exécuteur, et les réglages qui comptent

Ce que Spark fait de la mémoire qu'on lui donne, et les valeurs par défaut de Spark 3.5.4.

Comment un exécuteur range sa mémoire



Les défauts qu'on croise dans ce deck

OPTION	DÉFAUT 3.5.4	RÔLE
<code>spark.sql.shuffle.partitions</code>	200	Partitions après un shuffle
<code>spark.default.parallelism</code>	nb de cœurs	Partitions par défaut des RDD
<code>spark.sql.files.maxPartitionBytes</code>	128 Mo	Partition maximale à la lecture
<code>spark.sql.autoBroadcastJoinThreshold</code>	10 Mo	Seuil de diffusion automatique
<code>spark.sql.adaptive.enabled</code>	true	AQE : plan révisé en cours de route
<code>spark.sql.adaptive.skewJoin.enabled</code>	true	Découpe les jointures déséquilibrées
<code>spark.sql.execution.arrow.pyspark.enabled</code>	false	Arrow pour <code>toPandas()</code>
<code>spark.driver.maxResultSize</code>	1g	Plafond d'un résultat ramené au driver

S · 1 AVANCÉ

Spill

Quand une partition ne tient plus en mémoire, Spark ne plante pas : il écrit sur disque, et ralentit.

Provoquer un spill, et le lire

```
spark = (SparkSession.builder
  .master("local[2]")
  .config("spark.driver.memory", "512m")
  .config("spark.sql.shuffle.partitions", "2")
  .getOrCreate())

df = spark.range(20_000_000).select(
  "id", F.sha1(F.col("id").cast("string"))
    .alias("cle"))
df.orderBy("cle").write.format("noop") \
  .mode("overwrite").save()
```

Vingt millions de lignes triées en **deux** partitions, avec 512 Mo de mémoire.

ÉTAPE	ÉCRIT EN SHUFFLE	SPILL MÉM.	SPILL DISQUE
Lecture et échange	889 Mo	0	0
Tri, 2 tâches	—	1 302 Mo	949 Mo

Durée totale : 258 s sur un poste chargé. Sans la contrainte de mémoire, ce tri ne déborde pas.

Spill (Memory) : la taille des données désérialisées qui ont dû quitter la mémoire. *Spill (Disk)* : ce qu'elles pèsent une fois écrites. Les deux non nuls, c'est un spill.

Supprimer un spill, dans l'ordre

1 Toutes les tâches débordent ? Des partitions plus petites

Plus de `shuffle.partitions` : AQE fusionne les petites partitions, il ne découpe pas celles d'une agrégation. Viser 100 à 200 Mo par tâche après shuffle.

2 Une seule tâche déborde ? C'est un déséquilibre

Mémoire et partitions n'y feront rien : section Skew.

3 Moins de données à trier

Filtrer et projeter avant la jointure ou l'agrégation, pas après.

4 Libérer la mémoire unifiée

Un `cache()` inutile occupe la place des calculs : `unpersist()` dès qu'on n'en a plus besoin.

5 En dernier : plus de mémoire par cœur

Plus de mémoire par exécuteur, ou moins de cœurs par exécuteur. C'est le remède le plus cher.

S · 2 AVANCÉ

Shuffle

Combien de partitions après un shuffle, quelle stratégie de jointure, et comment éviter un shuffle tout court.

200 partitions après un shuffle : trop, ou pas assez

```
(trajets.groupBy("station_depart", "heure")
  .agg(F.count("*"), F.avg("duree_min"))
  .write.format("noop").mode("overwrite")
  .save())
```

RÉGLAGE	TÂCHES	DURÉE
AQE coupé · 200	207	18,5 s
AQE coupé · 8	15	2,3 s
AQE · 200	8	9,3 s
AQE · 8	8	1,5 s

300 000 trajets, `local[2]`, meilleur de trois passages, poste chargé.

Ce que fait AQE

Il lit la taille réelle des sorties du shuffle et **fusionne** les petites partitions : ici, les 200 prévues sont ramenées à **une seule**, et le job passe de 207 à 8 tâches en tout : 7 de lecture, 1 d'agrégation.

Ce qu'il ne fait pas

Il fusionne, il ne découpe pas les partitions trop grosses d'une agrégation. Sur de gros volumes, 200 peut être **trop peu** : c'est là qu'on relève la valeur.

Diffuser la petite table, et le vérifier

```
q = trajets.join(stations,
                 trajets.station_depart
                 == stations.station_id)

spark.conf.set(
    "spark.sql.autoBroadcastJoinThreshold",
    "-1")                # diffusion interdite

trajets.join(F.broadcast(stations),
             trajets.station_depart
             == stations.station_id) # l'indication gagne
```

JOINTURE	STRATÉGIE	DURÉE
Défaut (seuil 10 Mo)	BroadcastHashJoin	2,0 s
Seuil à -1	SortMergeJoin	6,0 s
Seuil à -1 + <code>F.broadcast</code>	BroadcastHashJoin	2,5 s

300 000 trajets joints à 60 stations. Trois fois plus lent dès que la petite table n'est plus diffusée.

Indications disponibles : `broadcast`, `merge`, `shuffle_hash`, `shuffle_replicate_nl`, via `df.hint("...")` ou `/*+ BROADCAST(s) */` en SQL.

Semi, anti, et le produit cartésien involontaire

```
longs = trajets.where("duree_min > 120")

stations.join(longs, stations.station_id
              == longs.station_depart,
              "left_semi")
# stations ayant un trajet de plus de 2 h

stations.join(trajets, stations.station_id
              == trajets.station_depart,
              "left_anti")
# stations sans aucun départ
```

La condition oubliée

```
trajets.join(stations).count()
# 17 955 060 = 299 251 × 60
# BroadcastNestedLoopJoin
```

Depuis Spark 3.0, `spark.sql.crossJoin.enabled` vaut **true** : une jointure sans condition n'est plus refusée. Elle multiplie les lignes, en silence.

QUESTION

RÉPONSE

Stations sans aucun départ

0

Stations avec un trajet de plus de 2 h

13 sur 60

Plan : `BroadcastHashJoin ... LeftSemi` — ni doublon, ni colonne en plus.

Fenêtre ou agrégation puis jointure : mesurer

Les 30 128 trajets deux fois plus longs que la moyenne de leur station de départ, écrits de deux façons :

```
moy = (trajets.groupBy("station_depart")
        .agg(F.avg("duree_min").alias("moy")))
trajets.join(moy, "station_depart") \
    .where(F.col("duree_min")
           > 2 * F.col("moy"))
```

```
w = Window.partitionBy("station_depart")
trajets.withColumn("moy",
    F.avg("duree_min").over(w)) \
    .where(F.col("duree_min")
           > 2 * F.col("moy"))
```

	ÉCHANGES	DURÉE
groupBy + jointure	2 (dont 1 diffusion)	3,4 s
Fenêtre	1	5,1 s

Ici, la fenêtre est **plus lente** : l'agrégat ne fait que 60 lignes, Spark le diffuse, et la jointure ne coûte presque rien — tandis que la fenêtre doit trier les 300 000 trajets par station.

Ce qui ramène vraiment les données au driver

ACTION	CE QUI REVIENT	RISQUE
<code>collect()</code> , <code>toPandas()</code>	tout	Le driver doit tout contenir : erreur mémoire, ou <code>maxResultSize</code> (1 Go) dépassé
<code>take(n)</code> , <code>head(n)</code> , <code>limit(n).collect()</code>	n lignes	Sans risque pour un petit n ; un grand n est un <code>collect</code>
<code>toLocalIterator()</code>	une partition à la fois	Le driver tient la plus grosse partition, pas tout ; lent
<code>count()</code> , <code>reduce()</code> , <code>agg(F.sum...)</code>	une valeur	Aucun pour le driver : le coût est la lecture complète, pas la mémoire
<code>foreach()</code> , <code>foreachPartition()</code>	rien	S'exécutent sur les exécuteurs : un <code>print</code> n'apparaît pas dans le notebook
<code>F.broadcast(df)</code>	la table entière	Collectée par le driver avant d'être diffusée

Réduire **avant** de ramener : agréger, filtrer, `limit`. Ce qui revient au driver doit être un résultat, pas une donnée.

S · 3 AVANCÉ

Skew

Quand une clé concentre les données, une tâche fait le travail de toutes, et le job attend la dernière.

Un déséquilibre fabriqué, et ce qu'AQE en fait

```
skew = trajets.withColumn("station_depart",  
    F.when(F.rand(7) < 0.7, F.lit("S001"))  
    .otherwise(F.col("station_depart"))  
    # 70 % des trajets sur une seule clé  
  
skew.join(autre, skew.station_depart  
    == autre.sd) # sans diffusion
```

Seuils abaissés à 1 Mo pour la démonstration : par défaut, AQE ne considère une partition déséquilibrée qu'au-delà de 256 Mo et de 5 fois la médiane.

ÉTAPE DE LA JOINTURE	TÂCHES	MÉDIANE	MAX
Jointure sur la clé salée à 70 %	28	1,6 s	18,0 s

Onze fois la médiane sur une seule tâche : **c'est la signature.**
Toutes les autres attendent celle-là.

AQE n'est pas intervenu ici (pas de `skew=true` dans le plan) : même avec des seuils abaissés, les partitions de ce jeu de 300 000 lignes restent trop petites pour qu'il les découpe.

Ce qu'AQE ne rattrape pas, et le salage

OPÉRATION	FACE À UNE CLÉ CHAUDE
Jointure par tri-fusion	AQE découpe la partition
<code>sum</code> , <code>count</code> , <code>avg</code>	Pré-agrégation dans chaque tâche : peu de risque
<code>collect_list</code> , fenêtre, <code>distinct</code> par clé	Aucune aide : toute la clé dans une tâche
Jointure avec une table diffusable	Diffuser : plus de shuffle, plus de déséquilibre

```
sel = (F.rand() * 8).cast("int")
partiel = (df.withColumn("sel", sel)
           .groupBy("station_depart", "sel")
           .agg(F.sum("duree_min").alias("s")))
total = (partiel.groupBy("station_depart")
         .agg(F.sum("s").alias("duree_totale")))
```

Le sel découpe la clé chaude en 8 morceaux agrégés en parallèle, puis recombinaison. Ne marche qu'avec une opération **associative**.

S · 4 AVANCÉ

Storage

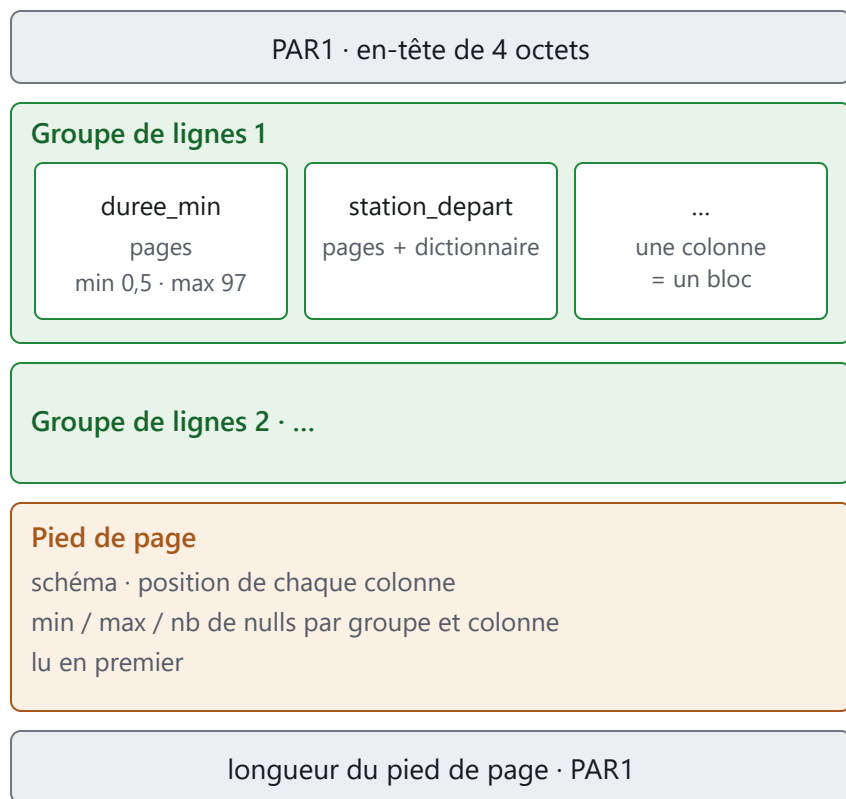
Le format, la compression, le nombre et la taille des fichiers, et ce qu'on évite de lire.

Choisir un format de fichier

	PARQUET	ORC	AVRO	DELTA · ICEBERG
Organisation	Colonnes	Colonnes	Lignes	Tables au-dessus de Parquet, le plus souvent
Fait pour	L'analyse : lire peu de colonnes de beaucoup de lignes	Idem, historiquement Hive	Échanger des messages, écrire ligne à ligne (Kafka)	Tables qu'on modifie : mises à jour, suppressions, versions
Avec Spark 3.5	Intégré, format par défaut	Intégré	Paquet <code>spark-avro</code> à ajouter	Paquet et extension à ajouter
Statistiques min / max	Oui, par groupe de lignes	Oui, et index légers	Non	Oui, et par fichier dans les métadonnées
Évolution du schéma	Fusion à la lecture (<code>mergeSchema</code>)	Limitée	Prévue par le format	Gérée, versionnée

Pour analyser avec Spark : **Parquet**. Pour une table qu'on modifie : Parquet **sous** Delta ou Iceberg. Avro sert aux échanges, pas à l'analyse.

Dans un fichier Parquet



1

Lire la fin du fichier

Le lecteur saute au pied de page : schéma, et statistiques de chaque groupe.

2

Écarter des groupes entiers

`duree_min > 120` : tout groupe dont le max est inférieur ou égal n'est pas lu.
C'est le *predicate pushdown*.

3

Ne lire que les colonnes demandées

Chaque colonne est un bloc contigu : un `select` de deux colonnes lit deux blocs.

Réglages, actifs par défaut en 3.5.4 :

```
spark.sql.parquet.filterPushdown, ...enableVectorizedReader.
```

Cinq codecs, mesurés

```
(trajets.coalesce(1).write  
  .option("compression", "zstd")  
  .parquet("sorties/codec-zstd"))
```

CODEC	TAILLE	ÉCRITURE	LECTURE 1 COLONNE
aucun	13,5 Mo	8,5 s	2,8 s
snappy défaut	9,3 Mo	9,3 s	1,7 s
lz4	9,4 Mo	4,4 s	0,6 s
gzip	7,2 Mo	11,9 s	0,7 s
zstd	6,7 Mo	4,3 s	0,7 s

Le choix par défaut

snappy, le défaut de Spark : compression correcte, et un choix sûr partout. lz4 et zstd le battent ici en vitesse, sur un poste chargé.

zstd quand le stockage coûte : plus compact, décompression rapide.

gzip : plus lent à écrire, rarement justifié en analytique.

Combien de fichiers on écrit

ÉCRITURE	FICHIERS	TAILLE MOYENNE	CE QUI SE PASSE
Après un <code>groupBy</code> , AQE coupé	200	13 ko	Une partition de shuffle = un fichier
Après un <code>groupBy</code> , AQE actif	2	1,1 Mo	AQE a fusionné les partitions vides
Les trajets tels quels	7	1,5 Mo	Autant que de partitions en mémoire
<code>repartition(8)</code>	8	1,3 Mo	Shuffle complet, partitions équilibrées
<code>coalesce(1)</code>	1	9,3 Mo	Fusion sans shuffle — mais un seul cœur écrit
<code>partitionBy("quartier")</code>	56	199 ko	7 tâches × 8 quartiers
<code>repartition("quartier")</code> puis <code>partitionBy</code>	8	1,3 Mo	Un fichier par quartier

Viser des fichiers de **128 Mo à 1 Go**. Trop petits : la lecture passe son temps à ouvrir des fichiers. `coalesce(n)` pour réduire sans shuffle, `repartition(col)` avant un `partitionBy(col)`, `maxRecordsPerFile` pour plafonner.

Ne lire que les partitions utiles

```
(trajets.withColumn("jour", F.to_date("debut"))
  .write.partitionBy("jour")
  .parquet("sorties/trajets-par-jour"))

pluie = (meteo.where("pluie_mm > 3")
  .select(F.col("date").alias("jour")))
spark.read.parquet("sorties/trajets-par-jour") \
  .join(pluie, "jour")
```

ÉLAGAGE DYNAMIQUE

DURÉE

Désactivé

12,8 s

Activé (défaut)

7,0 s

181 dossiers de partition, 17 jours de pluie de plus de 3 mm.

Statique

`where(F.col("jour") == "2026-06-30")` : la valeur est connue avant l'exécution, un seul dossier est lu (`PartitionFilters` dans le plan).

Dynamique (DPP, depuis 3.0)

Les jours viennent d'une **jointure** : Spark calcule d'abord la liste des jours de pluie, puis ne lit que ces dossiers.

Ne marche que si le filtre porte sur la colonne de partition elle-même, sans UDF autour.

Trouver la dernière partition sans lire les données

```
jvm = spark.sparkContext._jvm
conf = spark.sparkContext._jsc.hadoopConfiguration()
racine = jvm.org.apache.hadoop.fs.Path(chemin)
fs = racine.getFileSystem(conf)

jours = sorted(
    s.getPath().getName().split("=")[1]
    for s in fs.listStatus(racine)
    if s.getPath().getName().startswith("jour="))

derniere = (spark.read.parquet(chemin)
    .where(F.col("jour") == jours[-1]))
```

Mesuré

181 dossiers listés, dernier : 2026-06-30. La lecture filtrée ne rend que **1 956 lignes**, et le plan porte `PartitionFilters: [jour = 2026-06-30]`.

À éviter

`df.agg(F.max("jour"))`, puis un filtre : Spark lit toutes les partitions pour trouver le maximum.

Mettre en cache : quand, comment, et quand arrêter

APPEL	NIVEAU EN 3.5.4
<code>df.cache()</code>	Disk Memory Deserialized 1x
<code>df.persist(niveau)</code>	celui qu'on demande
<code>rdd.cache()</code> (PySpark)	Memory Serialized 1x

- Utile si le résultat sert **au moins deux fois** et coûte à recalculer.
- Paresseux : rien n'est stocké avant la première action.
- `unpersist()` dès que c'est fini : le cache prend la place des calculs.

Les niveaux `_SER`

En PySpark, les données d'un RDD sont toujours sérialisées (pickle), et celles d'un DataFrame sont au format binaire de Spark : les variantes `MEMORY_ONLY_SER` de Scala n'ont pas d'intérêt ici.

Le cache n'est pas une sauvegarde

Il disparaît avec la session. Pour réutiliser entre deux jobs : écrire en Parquet.

S · 5 AVANCÉ

Serialization

Convertir des objets en octets, pour le réseau, le disque, ou pour passer de la JVM à Python.

Kryo, pickle, Arrow : qui sérialise quoi

DONNÉE	FORMAT	RÉGLAGE UTILE
DataFrame, entre exécuteurs	Binaire interne de Spark (Tungsten)	Aucun : ni Java ni Kryo n'interviennent
RDD d'objets Python	pickle, dans le processus Python	Aucun côté JVM : Kryo n'y peut rien
RDD d'objets JVM (Scala, Java)	Java par défaut, ou Kryo	<code>spark.serializer=...KryoSerializer</code>
DataFrame ↔ pandas	pickle ligne à ligne, ou Arrow	<code>spark.sql.execution.arrow.pyspark.enabled</code>
UDF Python	pickle, ou Arrow pour les UDF pandas	Fonctions natives, sinon <code>pandas_udf</code>

Pour un utilisateur Python, la sérialisation qui coûte est celle qui traverse la frontière **JVM ↔ Python**. Kryo et `KryoRegistrar` n'ont aucun effet sur du code PySpark.

toPandas() avec et sans Arrow

```
spark.conf.set(
    "spark.sql.execution.arrow.pyspark.enabled",
    "true")
pdf = trajets.toPandas()
```

50 000 LIGNES, 14 COLONNES

DURÉE

Arrow désactivé (défaut)

11,6 s

Arrow activé

0,34 s

Sur 300 000 lignes, la version sans Arrow n'a pas fini dans le temps imparti à la mesure.

À mettre dans chaque session

Désactivé par défaut en Spark 3.5. En cas de type non pris en charge, Spark revient seul à l'ancien chemin (... `arrow.pyspark.fallback.enabled`, actif par défaut).

Ça reste un collect

Arrow accélère la conversion, pas la taille : le résultat doit toujours tenir dans la mémoire du driver.

DIMENSIONNER **AVANCÉ**

Combien d'exécuteurs, de cœurs, de mémoire

Une méthode explicite, et deux cas refaits de bout en bout.

La méthode, en cinq étapes

1 Compter les tâches de l'étape la plus lourde

Lecture : $\text{volume} \div 128 \text{ Mo}$ (`maxPartitionBytes`). Après shuffle : $\text{volume shuffle} \div 100 \text{ à } 200 \text{ Mo}$.

2 Choisir le nombre de vagues acceptable

Une vague = autant de tâches que de cœurs. Deux à quatre vagues, c'est raisonnable ; une seule est un luxe rarement rentable.

3 En déduire les cœurs, puis les exécuteurs

Cœurs = $\text{tâches} \div \text{vagues}$. Exécuteurs de 4 à 5 cœurs chacun.

4 Mémoire par exécuteur

Mémoire unifiée $\approx \text{cœurs} \times 2 \text{ à } 3$ fois la taille d'une partition. Tas = $\text{unifiée} \div 0,6 + 300 \text{ Mo}$.

5 Ajouter l'overhead, puis placer sur les machines

$\max(384 \text{ Mo}, 10 \%)$, davantage en PySpark. Laisser 1 cœur et 1 à 2 Go par machine au système.

Un traitement de 10 Go

ÉTAPE	CALCUL	RÉSULTAT
Tâches à la lecture	10 Go ÷ 128 Mo	80
Vagues visées		2
Cœurs	80 ÷ 2	40
Exécuteurs	40 ÷ 5 cœurs	8
Mémoire unifiée	5 × 3 × 128 Mo	≈ 1,9 Go
Tas	1,9 Go ÷ 0,6 + 0,3	≈ 3,4 → 4 Go
Overhead	max(384 Mo, 10 %)	≈ 410 Mo

```
spark-submit \  
  --num-executors 8 \  
  --executor-cores 5 \  
  --executor-memory 4g \  
  --conf spark.sql.shuffle.partitions=160 \  
  job.py
```

Total : **40 cœurs, ≈ 35 Go**. L'ancienne version de ce cas proposait 75 cœurs et 75 Go pour une seule vague : deux fois plus de ressources pour gagner, au mieux, une vague.

90 Go de transactions chaque nuit, sur Kubernetes

ÉTAPE	CALCUL	RÉSULTAT
Tâches à la lecture	$90 \text{ Go} \div 128 \text{ Mo}$	720
Vagues visées	fenêtre de nuit	4
Cœurs	$720 \div 4$	180
Exécuteurs	$180 \div 5$	36
Tas	$5 \times 3 \times 128 \text{ Mo} \div 0,6 + 0,3$	$\approx 3,4 \rightarrow \mathbf{4 \text{ Go}}$
Overhead PySpark	workers Python	1 Go
Pod	$5 \text{ cœurs} \cdot 5 \text{ Go}$	$\times 36$

Grain fin ou grain gros

36 × 5 cœurs : 180 tâches en parallèle, des pods faciles à placer.

18 × 10 cœurs : même total, moins de pods, mais chaque exécuteur sature plus vite son disque local et son réseau.

Total : **180 cœurs, 180 Go**. Les actions du job (`count`, agrégation, écriture) ne se partagent pas les données : chacune relit tout, sauf cache.

Le ramasse-miettes, sans y passer la journée

- **Symptôme** : *GC Time* au-delà de 10 % de la durée des tâches, dans l'onglet Executors.
- **Cause habituelle** : trop d'objets en mémoire. Un cache de trop, une partition trop grosse, un `collect_list`.
- **Premier remède** : le même que pour le spill. Des partitions plus petites, moins de cache.

Changer de collecteur ?

Depuis Java 9, G1 est le collecteur par défaut. CMS, qu'on voit encore recommandé, a été **retiré** de Java 14 : une option `-XX:+UseConcMarkSweepGC` sur Java 17 empêche la JVM de démarrer.

```
--conf "spark.executor.extraJavaOptions=
      -XX:+UseG1GC"    # utile sous Java 8 seulement
```

LAKEHOUSE **AVANCÉ**

Des tables transactionnelles au-dessus de Parquet

Ce qu'apportent Iceberg et Delta, et comment Iceberg range ses métadonnées.

Un dossier Parquet n'est pas une table

PROPRIÉTÉ	DOSSIER PARQUET	ICEBERG · DELTA
Atomicité	Une écriture qui plante laisse des fichiers à moitié	Tout ou rien : un seul pointeur change
Isolation	Un lecteur voit l'écriture en cours	Chaque lecture voit un instantané
Modification	Réécrire le dossier	UPDATE, DELETE, MERGE
Historique	Aucun	Versions consultables et restaurables
Schéma	Déduit des fichiers	Imposé, et évolutif

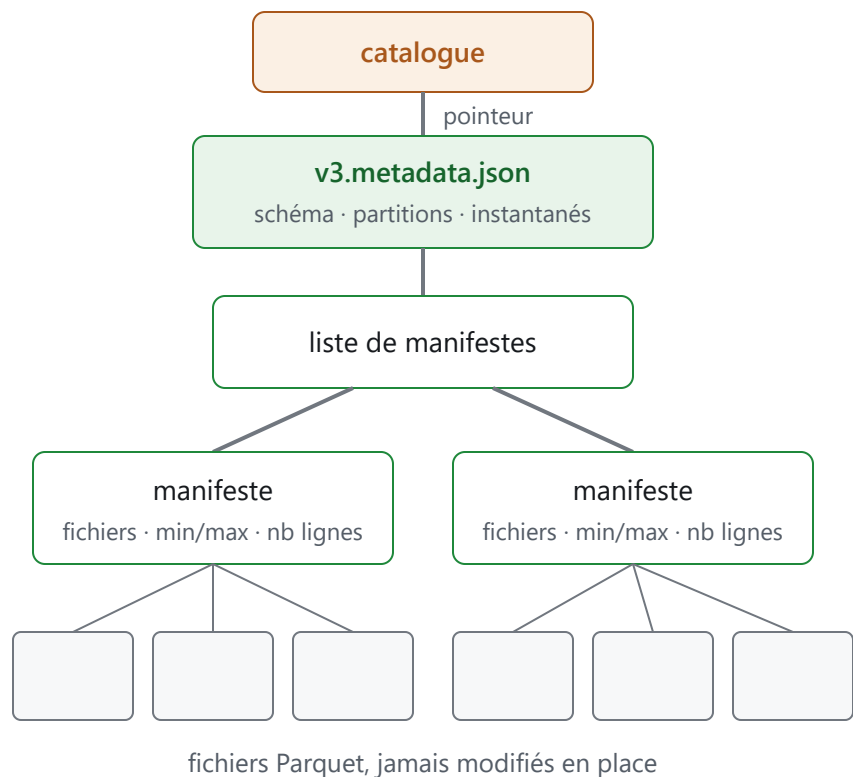
Spark n'est pas transactionnel

Il traite des données ; il ne garantit rien sur le dossier qu'il écrit.
Les garanties viennent du **format de table**.

Trois formats

Delta Lake, né chez Databricks ; **Apache Iceberg**, né chez Netflix ; **Apache Hudi**, né chez Uber. Tous rangent des fichiers Parquet sous un journal de métadonnées.

Les métadonnées d'une table Iceberg



1

Chaque écriture crée des fichiers neufs

Données, manifestes, et un nouveau `metadata.json`.

2

La validation déplace un pointeur

Le catalogue passe atomiquement à la nouvelle version : c'est tout le « A » d'ACID.

3

Les anciennes versions restent lisibles

Tant qu'on ne les expire pas : c'est le voyage dans le temps.

4

L'élagage se fait sur les métadonnées

Min / max par fichier dans les manifestes, sans lister de dossiers.

Iceberg en SQL, depuis PySpark

```
spark = (SparkSession.builder
    .config("spark.jars.packages",
        "org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.10.1")
    .config("spark.sql.extensions",
        "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
    .config("spark.sql.catalog.loc",
        "org.apache.iceberg.spark.SparkCatalog")
    .config("spark.sql.catalog.loc.type", "hadoop")
    .config("spark.sql.catalog.loc.warehouse", "file:///.../entrepot")
    .getOrCreate())
```

```
CREATE TABLE loc.velo.trajets (
    trajet_id STRING, debut TIMESTAMP,
    duree_min DOUBLE, station_depart STRING)
USING iceberg PARTITIONED BY (days(debut))

MERGE INTO loc.velo.trajets t
USING corrections c ON t.trajet_id = c.trajet_id
WHEN MATCHED THEN UPDATE SET t.duree_min = c.duree_min
WHEN NOT MATCHED THEN INSERT *

DELETE FROM loc.velo.trajets WHERE duree_min > 600
```

```
SELECT snapshot_id, operation
FROM loc.velo.trajets.snapshots

SELECT count(*) FROM loc.velo.trajets
VERSION AS OF 1234567890 -- un snapshot_id

CALL loc.system.rollback_to_snapshot(
    'velo.trajets', 1234567890)
```

OPÉRATION	LIGNES
Insertion des trajets	299 251
Après MERGE INTO	299 252
Après DELETE	297 229
Instantané 1, en VERSION AS OF	299 251
Après rollback_to_snapshot	299 251

Delta Lake ou Iceberg

	DELTA LAKE	APACHE ICEBERG
Métadonnées	Journal de transactions : fichiers JSON, points de contrôle Parquet	<code>metadata.json</code> , listes de manifestes, manifestes Avro
Fichiers de données	Parquet	Parquet, ORC ou Avro
Voyage dans le temps	<code>VERSION AS OF</code> , <code>TIMESTAMP AS OF</code> , <code>RESTORE</code>	<code>VERSION AS OF</code> , <code>TIMESTAMP AS OF</code> , procédure <code>rollback_to_snapshot</code>
Partitionnement	Colonnes classiques ; regroupement par <code>OPTIMIZE ... ZORDER BY</code>	Transformations cachées, et évolution du partitionnement sans réécriture
Entretien	<code>OPTIMIZE</code> , <code>VACUUM</code>	<code>rewrite_data_files</code> , <code>expire_snapshots</code>
Moteurs	Spark d'abord, écosystème Databricks ; lecteurs pour Trino, Flink...	Pensé multi-moteurs : Spark, Trino, Flink, Snowflake, Athena...

Le critère qui décide le plus souvent : **quels autres moteurs** lisent la table. Un seul, Spark ou Databricks : Delta est naturel. Plusieurs :

Avant de toucher à la configuration

Dans le code

- Filtrer et ne garder que les colonnes utiles, **avant** jointures et agrégations.
- Aucune UDF Python là où une fonction native existe.
- Pas de `collect()` ni de `toPandas()` sur autre chose qu'un résultat.
- Diffuser les petites tables ; semi / anti jointures pour les tests d'existence.
- Un `cache()` seulement si réutilisé, et son `unpersist()`.

Dans les données

- Parquet, fichiers de 128 Mo à 1 Go, partitionné par la colonne filtrée.
- Arrow activé si l'on passe par pandas.

Dans l'interface web

- L'étape la plus longue, et sa tâche max contre la médiane.
- Spill, shuffle, GC : lequel des cinq S ?
- Un seul changement, puis remesurer.

Cinq questions sur les cinq S

Q1 Une étape de 200 tâches : 199 finissent en 2 s, une en 3 min, avec du spill. Doublez-vous la mémoire ?

Non : c'est un déséquilibre. Jointure : vérifier qu'AQE le découpe ; sinon diffuser l'autre côté ou saler la clé.

Q2 Pourquoi activer Kryo n'accélère-t-il pas un job PySpark fait de DataFrames ?

Les lignes d'un DataFrame sont déjà au format binaire de Spark, et les objets Python sont sérialisés par pickle : Kryo ne sérialise que des objets JVM.

Q3 Un `partitionBy("jour")` sur 181 jours peut écrire jusqu'à 36 200 fichiers. Pourquoi, et que faire ?

Chaque tâche écrit un fichier par jour rencontré : 200 tâches × 181 jours. Un `repartition("jour")` avant l'écriture en fait un par jour.

Q4 `count()` fait-il courir un risque de mémoire au driver ?

Non : chaque exécuteur compte sa partition, seul un nombre remonte. Il coûte une lecture complète, pas de la mémoire.

Q5 Une jointure sans condition passe sans erreur en 3.5. Est-ce normal ?

Oui depuis Spark 3.0 : `crossJoin.enabled` vaut `true`. C'est un produit cartésien, à repérer dans le plan.

FIN **AVANCÉ**

Le code d'abord, les données ensuite, la configuration en dernier

Et entre chaque changement : l'interface web.